Introduction to Data Structures: Where Data Lives and How Programs Use It

These study notes are based on the provided transcript.

Lesson overview

This lesson introduces the basic idea of **data structures**. The instructor explains that every program works on data. Because of that, data must be stored and organized in a useful way while the program is running. That organization in **main memory (RAM)** is called a **data structure**. The lesson also gives a beginner-level comparison between **data structures**, **databases**, **data warehouses**, and **big data**.

Main topics

1. What data structure means
2. Why data is necessary in every program
3. CPU, RAM, and storage roles
4. How a program and its data move into memory
5. Why data structures are needed in running applications
6. Difference between data structure and database
7. What a data warehouse is
8. What big data means
9. Final comparison of all four terms

---

1. What is a data structure?

Definition

A **data structure** is the **arrangement of a collection of data items** so that the data can be used efficiently and operations on that data can also be done efficiently.

Detailed explanation

- A program does not only need instructions.
- A program also needs **data**.
- That data must be organized properly.
- Good organization makes processing easier and faster.
- The instructor says data structures are about:
  - **arranging data**
  - **performing efficient operations on data**

Where does a data structure exist?

A data structure exists **inside main memory** while a program is running. It is not mainly described here as something stored on disk. It is part of the program during execution.

Key idea

- **Data structure = arrangement of data in main memory during program execution**

---

2. Why data is necessary in every program

Core idea

The instructor explains that a program is a **set of instructions that perform operations on data to produce results**. Without data, instructions have no real purpose.

Important points

- Programs work on data.
- Data is an essential part of applications.
- Without data:
  - there is no meaningful processing
  - instructions are not useful
- Because every application uses data, every application needs some way to organize that data.

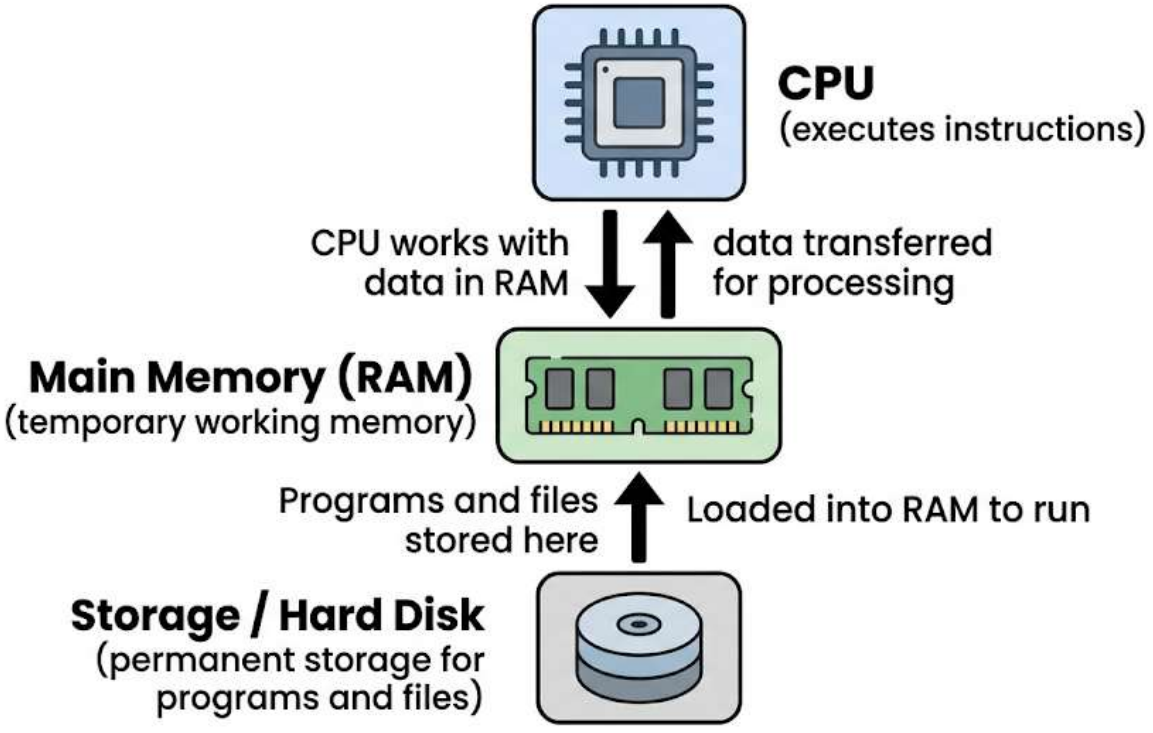
Beginner-friendly explanation

Think of a program like a worker and data like the material the worker uses.
If there is no material, the worker cannot do useful work.

---

3. CPU, RAM, and storage

The instructor uses a simple computer model to explain where things happen.



CPU

- CPU stands for **Central Processing Unit**
- It is the **processor**
- It executes instructions

Main memory

- Main memory means **RAM**
- It is **temporary memory**
- It is also called **working memory**
- It is also called **primary memory**

Storage

- Storage means things like:
  - hard disk
  - external storage
  - phone storage
- It is **permanent storage**

Device examples

The instructor gives examples from:

- **PC**
  - processor
  - RAM
  - hard disk
- **Mobile phone**
  - processor

- RAM
- storage such as 16 GB, 32 GB, 128 GB

Comparison

RAM

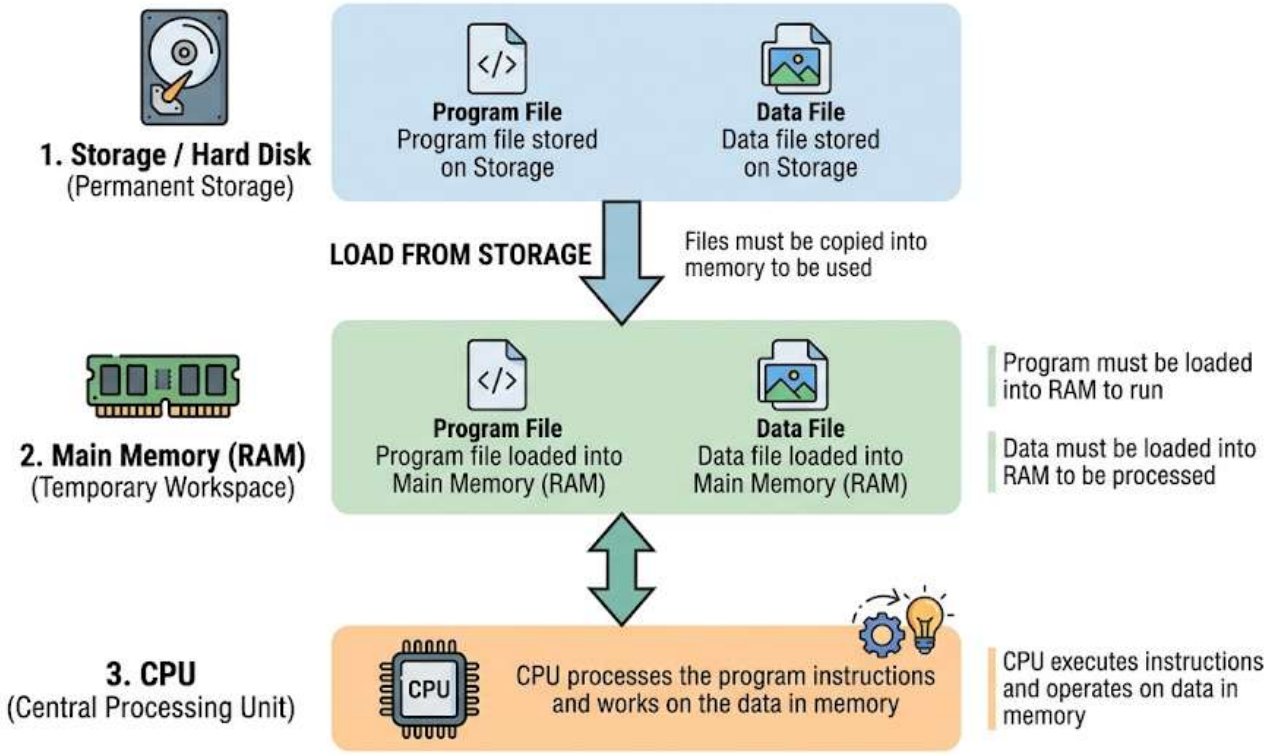
- temporary
- used while the program is running
- holds program code and active data

Storage

- permanent
- keeps installed applications and saved files

---

4. How programs and data move into memory

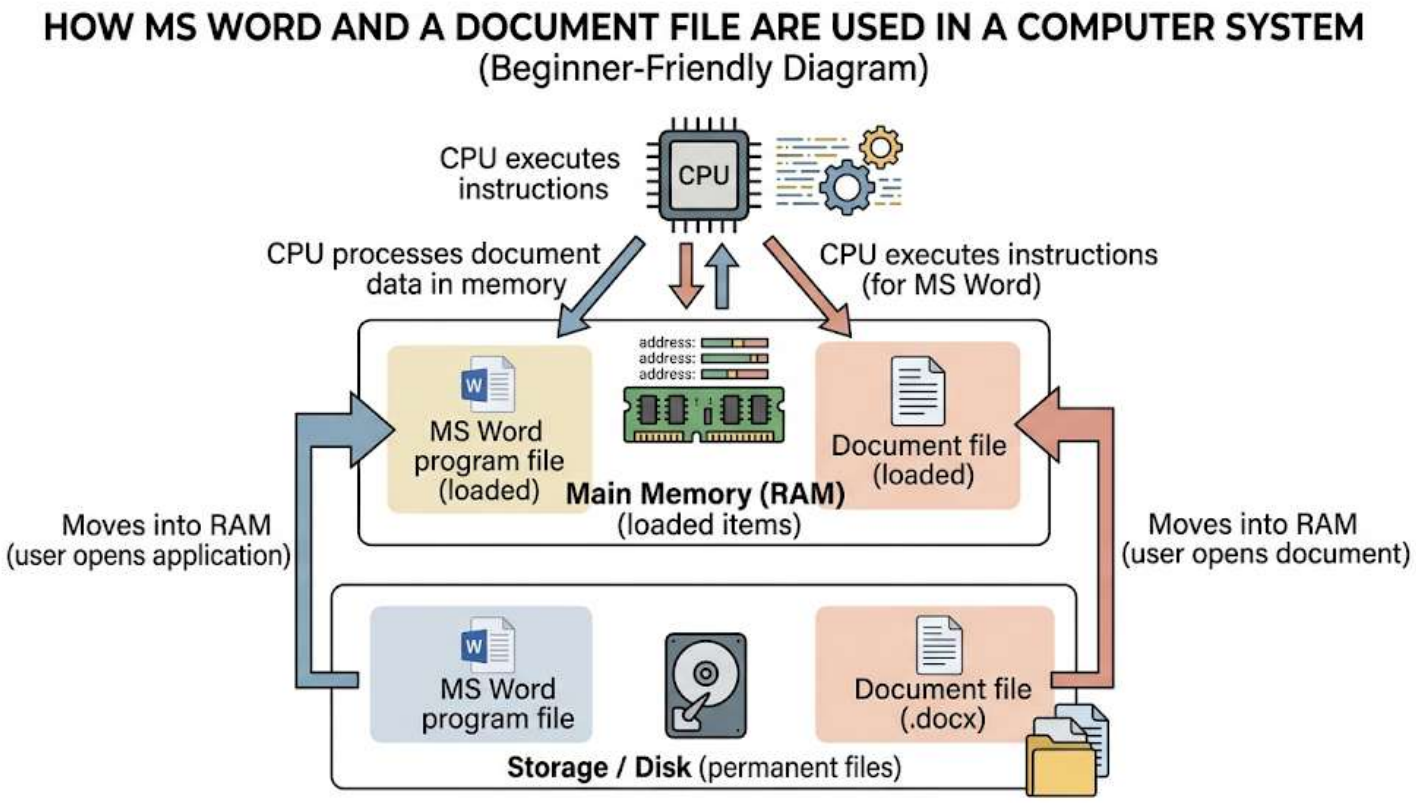


The instructor explains this through the example of **MS Word**.

Example used

- MS Word program file
- Word document file (.docx)

Step-by-step process



Step 1: The program is stored on storage

- When you install an application, it is stored on disk or phone storage.

Step 2: You open the application

- You click or tap the icon.

Step 3: The program is loaded into main memory

- The MS Word program code is brought into RAM.

Step 4: The CPU starts executing the program

- The processor runs the instructions.
- The application window appears.

- Menus and features become available.

Step 5: You open a data file

- Example: a Word document.

Step 6: The file's data is also brought into main memory

- The program cannot process the file directly while it stays only on storage.
- The data must be loaded into RAM.

Step 7: The program processes the data in memory

- The instructions of the running program operate on the data now present in RAM.

Important conclusion

Both of these must come into main memory:

- the **program**
- the **data**

Only then can processing happen.

---

5. Why data structures are needed

Main reason

Every application works on data, and that data must be organized in memory in a useful way. That organization is the data structure.

Instructor's explanation in simple form

- Applications like MS Word, Notepad, or a web browser all use data.
- That data may be:
    - text

    - images
    - videos
    - web pages
    - documents
- All of that must be placed in main memory while the application uses it.
- The way it is arranged affects efficiency.

Why organization matters

A good data structure helps the application:

- use data easily
- process data faster
- perform operations efficiently

Types of data structures mentioned

The instructor briefly mentions examples such as:

- arrays
- linked lists
- trees
- hash tables

These are examples of structures a program may choose depending on what kind of work it needs to do.

Key statement

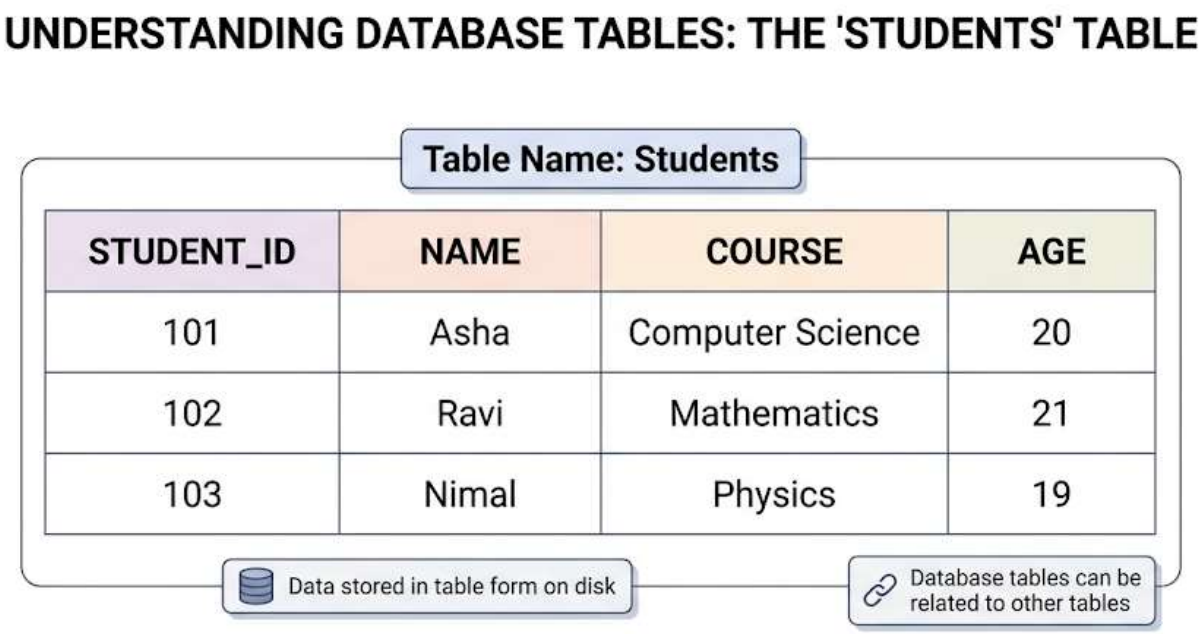
A data structure is **part of a running program**. It is created while the program executes and needs data.

---

6. Data structure vs database

The instructor then compares a data structure with a database.

What is a database?



A **database** is an arrangement of data in **permanent storage**, usually in some organized model, so applications can retrieve and use it easily.

The instructor specifically refers to **relational data** stored in **tables** on disk.

Main characteristics of a database

- stored on disk
- often used for commercial or business data
- organized into tables
- tables may have relationships with each other
- built for easy retrieval and access by applications

Business examples mentioned

- banks
- retail stores
- manufacturing organizations

Important connection to data structures

Even when data is stored in a database on disk, an application still has to bring that data into main memory to use it. Once the data is in memory, it still needs a **data structure** there.

Very important comparison

Data structure

- arrangement of data in main memory
- used during execution

Database

- arrangement of data on disk
- stored permanently
- often organized as relational tables

---

7. Data warehouse

Next, the instructor explains the idea of a **data warehouse**.

Why data warehouse is needed

Businesses collect a huge amount of data over time.

Examples:

- customer transactions
- manufactured goods

- sold goods
- old business records

As time passes, the total data becomes very large.

Two categories of business data

The instructor divides commercial data into:

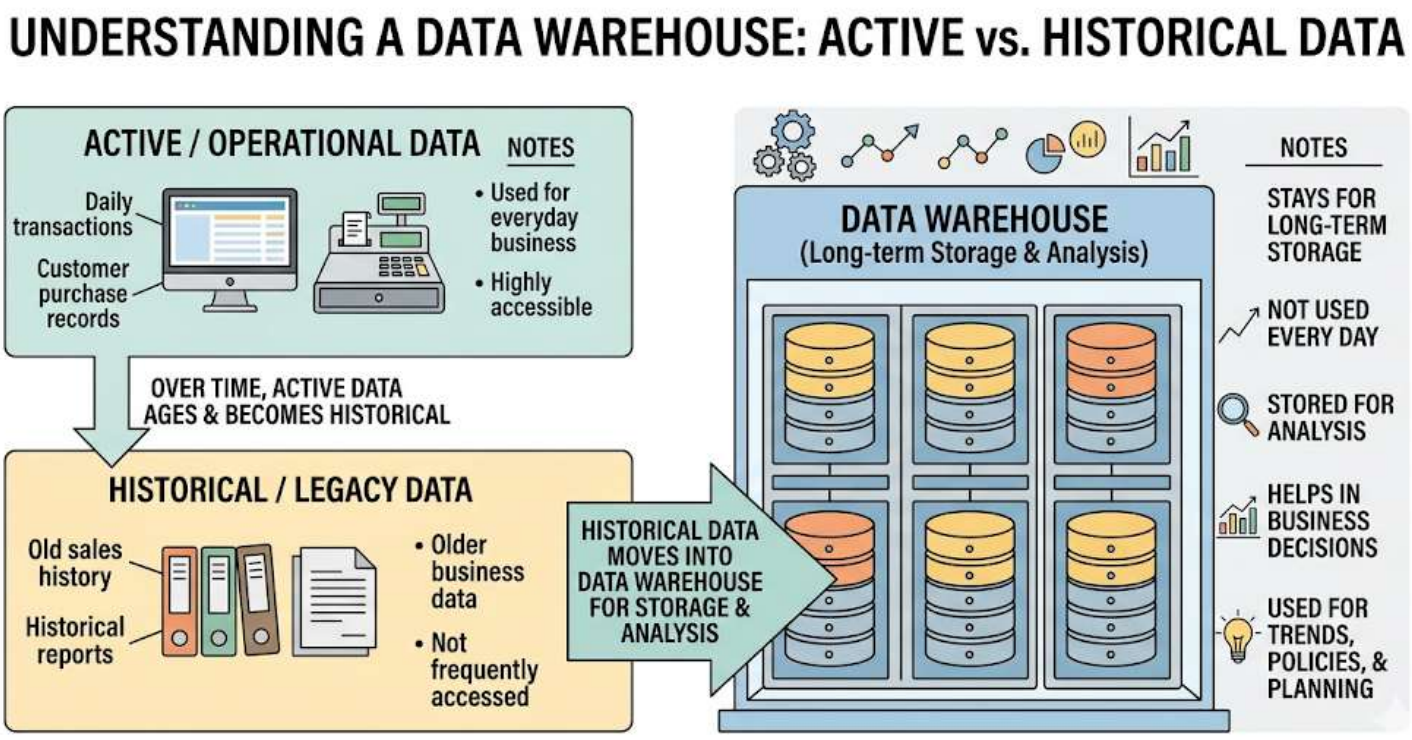
1. Operational data

- used daily
- active data

2. Legacy or historical data

- old data
- not used day to day
- may be used later when needed

What is a data warehouse?



A **data warehouse** is large historical data stored over an **array of disks**. It is mainly older, inactive data kept for future analysis and decision-making.

Why organizations keep it

According to the instructor, this old data helps with:

- analyzing business performance
- making policies
- starting new trends
- creating offers for customers
- decision-making

Related term mentioned

The instructor says that algorithms used to analyze warehouse data are called **data mining algorithms**.

Simple comparison

Database

- active stored data
- can support regular application use

Data warehouse

- very large historical data
- mostly not operational every day
- used for analysis and decisions

---

8. Big data

The lesson also introduces **big data** in a basic way.

Instructor's explanation

With the growth of the internet, a huge amount of data is being accumulated every day.

This data can be about:

- people
- places
- things
- many other topics

Why it matters

Analyzing this data can help with:

- management
- governance
- business decisions

Definition in this lesson

The instructor describes **big data** as the study of storing and using very large amounts of data, especially the kind growing through the internet.

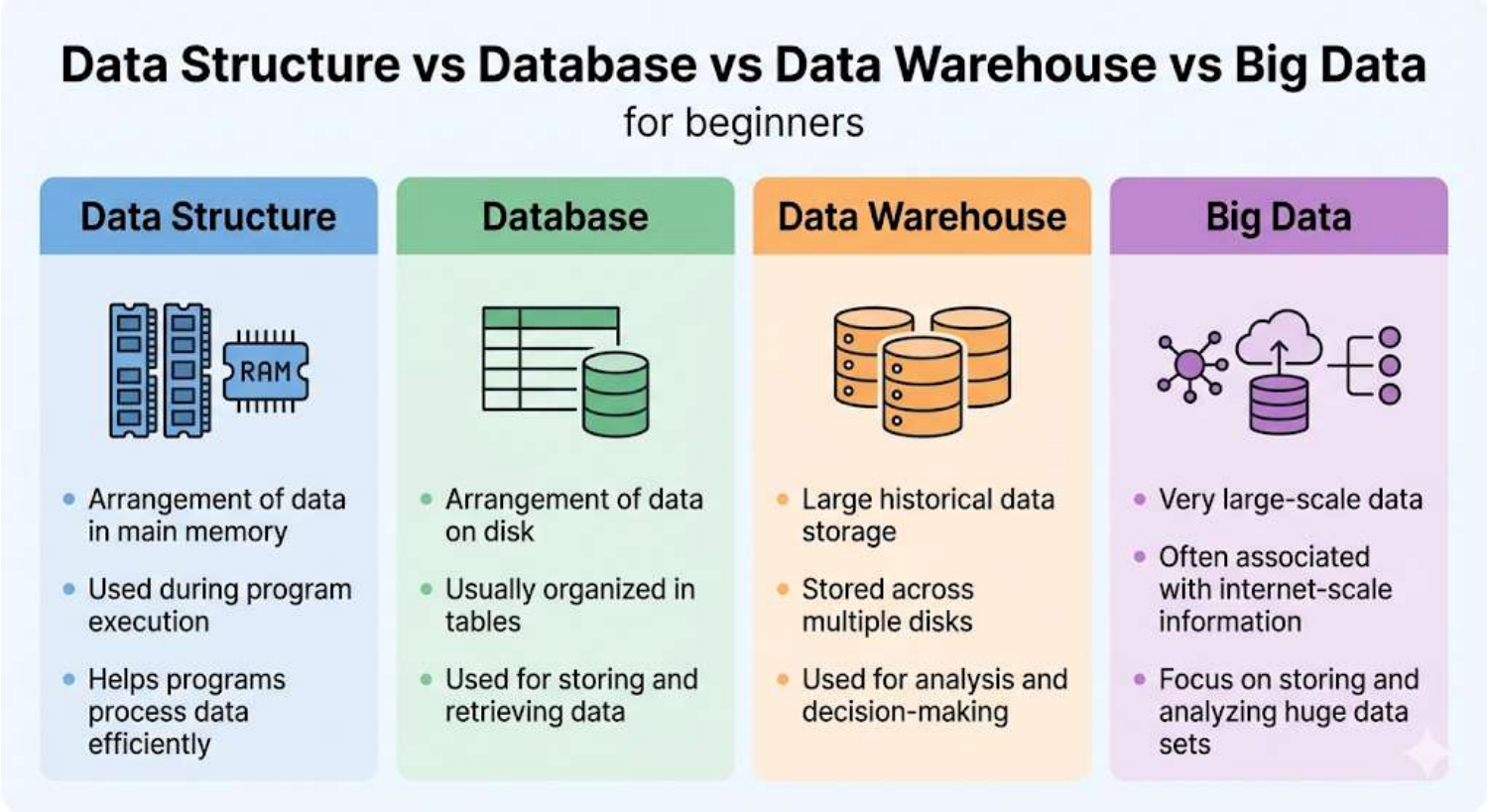
Beginner note

This is only a brief introductory explanation. The lesson does not go into technical big data tools or systems.

---

9. Final comparison of the four terms

The instructor repeats the differences clearly at the end.



Data Structure

- arrangement of data in **main memory**
- exists during **program execution**

Database

- arrangement of data on **disk**
- usually structured for access and retrieval
- often stored as relational tables

Data Warehouse

- arrangement of large **historical** data on an **array of disks**
- not usually used day to day
- useful for analysis and decision-making

Big Data

- study of storing and using very large data sets

- especially associated here with internet-scale data

Detailed explanation of the MS Word example

This is the most important example in the lesson because it shows where data structures are actually used.

What the instructor is trying to prove

The goal of the example is to show that:

- a program on disk cannot run until loaded into RAM
- a file on disk cannot be processed until loaded into RAM
- once data enters RAM, it must be arranged properly
- that arrangement is the data structure

Example flow

- MS Word is installed on storage.
- A document file is also stored on storage.
- You open MS Word.
- MS Word code is loaded into main memory.
- CPU executes the code.
- You open a document.
- The document data is loaded into main memory.
- MS Word now processes that data in memory.
- To do this efficiently, the data must be organized properly.

Why this matters

This example teaches the core principle:

Storage keeps data permanently, but processing happens in main memory.

That is why data structures are central to program design.

Code examples related to each topic

Code examples

No code was shown or explained in this lesson.

Important instructor tips, warnings, or common mistakes

Tips

- Always remember **where** each concept exists:
  - data structure → main memory
  - database → disk
  - data warehouse → array of disks
  - big data → very large-scale data, especially internet-related in this lesson
- When learning data structures, do not think only about definitions.
  Also think about **why programs need them**.
- Connect data structures to real applications like:
  - Word processors
  - Notepad
  - Web browsers

Warnings / common mistakes

- **Mistake:** Thinking data structures are mainly stored on disk.
  Correction: In this lesson, data structures are explained as arrangements in **main memory during execution**.
- **Mistake:** Thinking a program can directly process data while it stays only on storage.
  Correction: The data must first be brought into main memory.
- **Mistake:** Confusing database with data structure.
  Correction: Database is storage-side organization. Data structure is memory-side organization.
- **Mistake:** Thinking all stored business data is used every day.
  Correction: Some data is operational. Some is historical and belongs in a data warehouse.

Final recap

This lesson builds the foundation for studying data structures.

Main takeaways:

- Every program works on data.
- A program is a set of instructions that operates on data.
- For execution, the program must come into main memory.
- For processing, the data must also come into main memory.
- The arrangement of data in main memory during execution is called a **data structure**.
- Data structures help programs use data efficiently and run faster.
- A **database** stores organized data on disk.

- A **data warehouse** stores large historical business data, usually across arrays of disks.
- **Big data** refers to storing and using very large data sets, especially from internet-scale sources.
- The course focus from the next lesson onward will be **data structures**.

Unclear or incomplete transcript parts

The overall lesson is clear, but a few transcript parts are noisy or unclear:

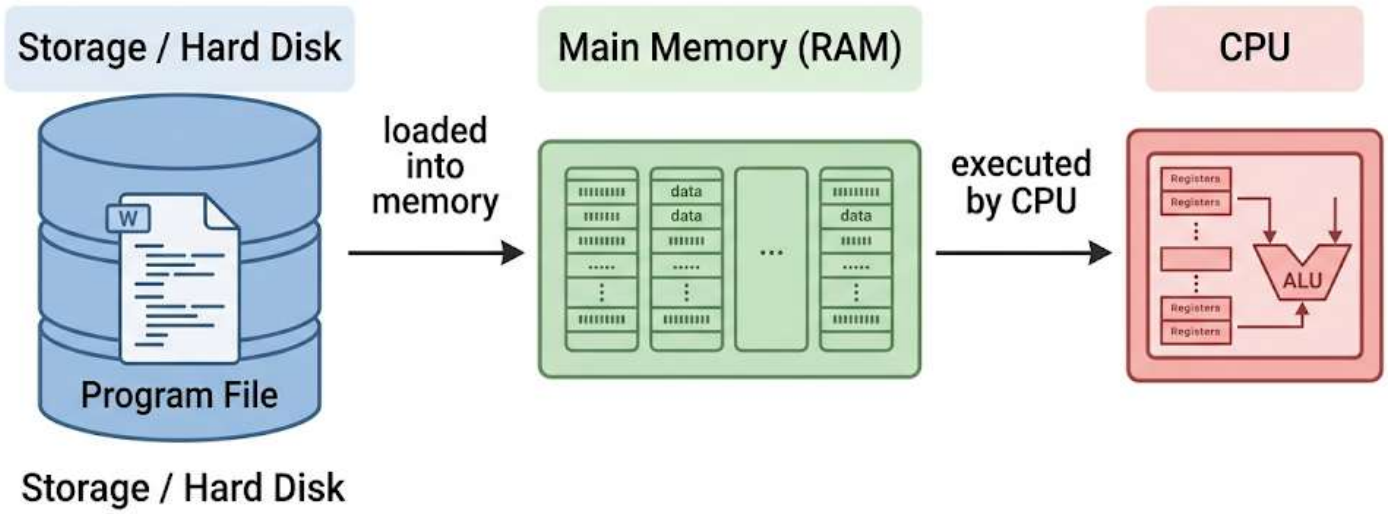
- “then the data term, term data” → **[unclear]**
- “commercial farms” may possibly mean **commercial firms** → **[unclear]**
- Some repeated phrases were transcript noise and were cleaned in these notes.

Static and Dynamic Memory Allocation

These notes are based on the provided transcript.

1. Lesson overview

In this lesson, the instructor introduces the idea of **static memory allocation** and **dynamic memory allocation**.



Before explaining the difference, the instructor first explains:

- what memory is
- how memory is organized
- how a program uses main memory
- what the **code section**, **stack**, and **heap** are
- why memory used for local variables is called **static allocation**

The lesson mainly builds the foundation needed to understand static vs dynamic memory allocation.

---

2. Main topics covered

- Understanding main memory
- Memory as bytes with addresses
- Why the instructor assumes a 64 KB memory model
- Memory segmentation
- How a program uses main memory
- Code section
- Stack
- Heap
- Example of local variables inside main()
- Stack frame / activation record
- Static memory allocation
- Why it is called “static”

---

3. Understanding main memory

3.1 Memory is divided into bytes

- Main memory is made of many small units.
- These small units are called **bytes**.
- Each byte has its own **address**.

3.2 Memory addresses are linear

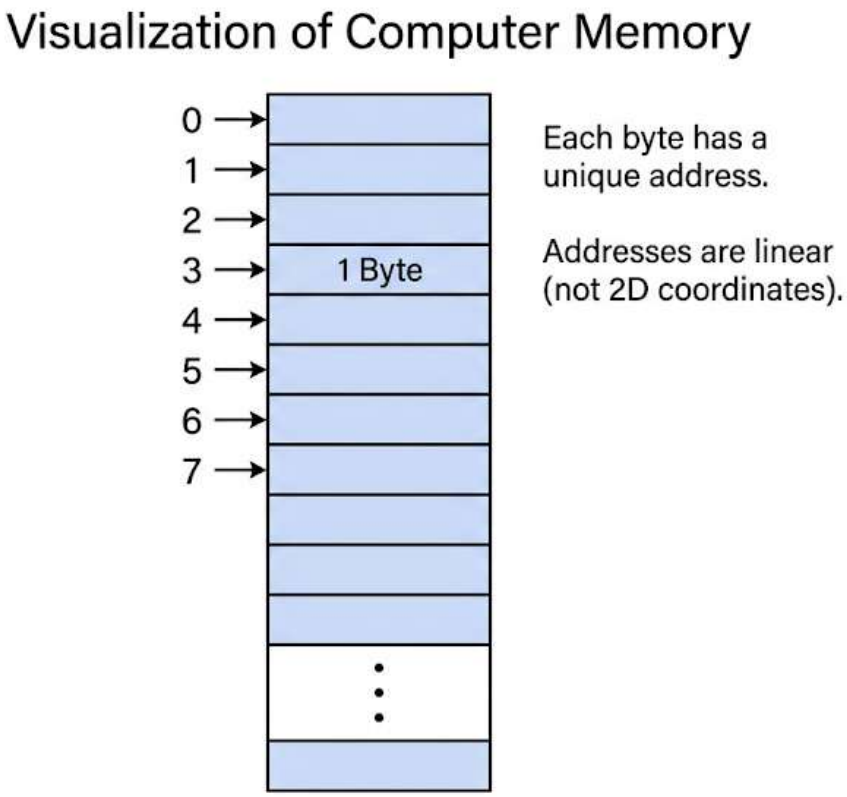
- Even if memory is drawn like a 2D box in a diagram, actual addresses are not 2D.
- Memory addresses are **linear**.

- That means each byte has only **one address value**.
- It is not like coordinates such as (x, y).

3.3 Example of byte addresses

The instructor gives a simple idea like this:

- first byte → address 0
- next byte → address 1
- then 2, 3, 4, 5, and so on

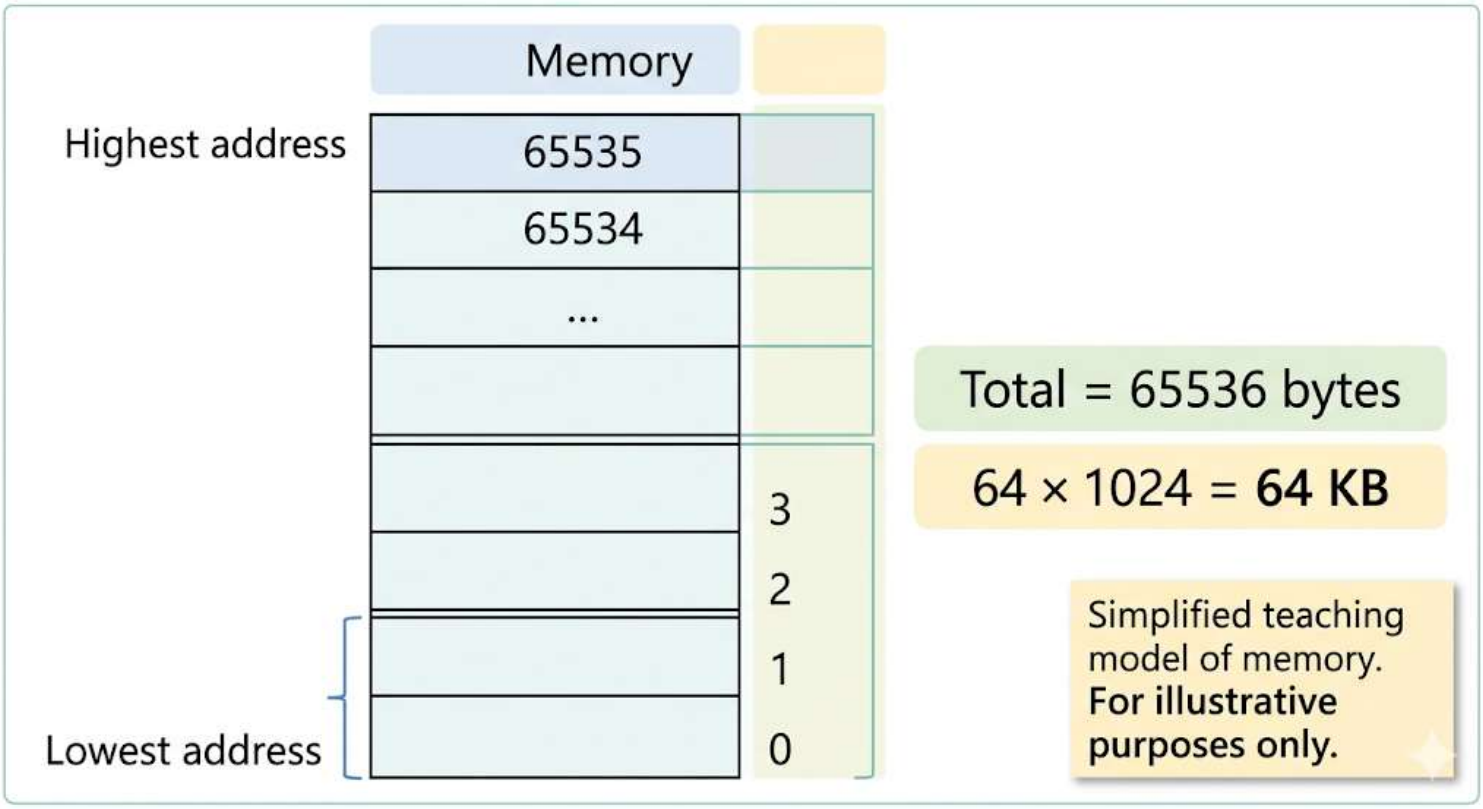


Important idea:

- memory addresses increase one by one
- every byte can be uniquely identified by its address

---

4. Assumed memory size in the lesson



4.1 Why the instructor uses 64 KB

To make the explanation easy, the instructor assumes that the memory size is:

- from address 0 to address 65535

That gives:

- total bytes = 65536
- which is 64 × 1024
- so the total memory size is **64 kilobytes (64 KB)**

4.2 Why this is only for learning

The instructor clearly says:

- real computers today use much larger RAM sizes
- such as **4 GB** or **8 GB**
- but for learning, a small memory model is easier to understand

So throughout this lesson, the instructor assumes:

- memory size = **64 KB**

---

5. Memory segments

5.1 Large memory is divided into smaller parts

The instructor explains that in real systems:

- the whole RAM is not always treated as one single block
- it is divided into smaller manageable parts
- these parts are called **segments**

5.2 Segment size in the discussion

- A segment is explained as usually being **64 KB**
- So when the instructor talks about memory in this lesson, he is basically discussing **one memory segment**

Important note

This is part of the instructor's simplified teaching model.
It is used to help understand how memory works.

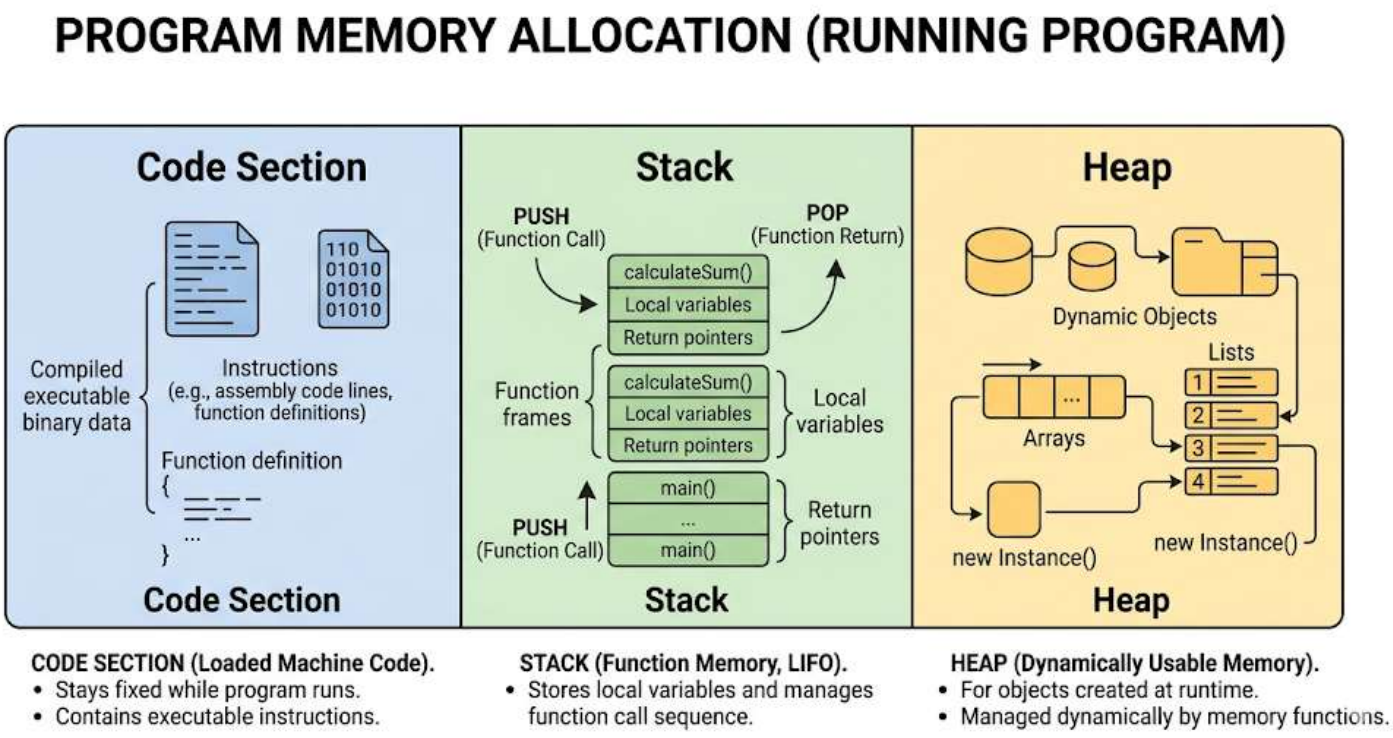
---

6. How a program uses main memory

The instructor explains that when a program runs, main memory is divided into **three sections**:

- **Code section**
- **Stack**
- **Heap**

These are the main memory areas used by a program.



---

7. Code section

7.1 What it is

The **code section** is the part of memory where the program's **machine code** is loaded.

7.2 How it is used

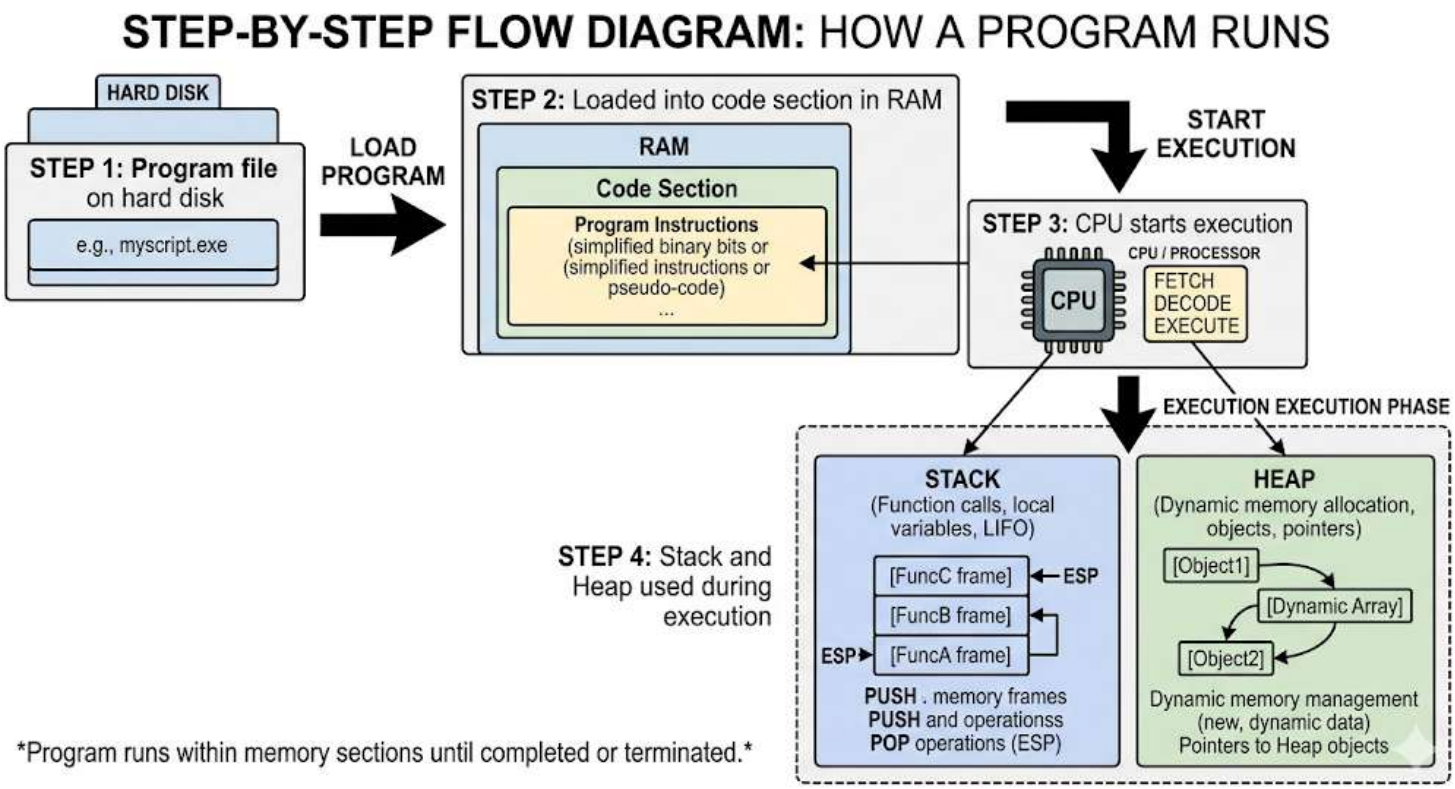
- A program file is stored on the hard disk
- To run the program, its machine code must first be loaded into main memory
- That loaded part goes into the **code section**

7.3 Important detail

- The size of the code section is **not fixed**
- It depends on the size of the program

Key idea

The code section contains the actual instructions that the CPU executes.



---

8. Stack and heap

After the program code is loaded:

- the CPU starts executing the program
- the remaining memory is used mainly as:
    - **stack**
    - **heap**

The instructor says the diagram position may be changed just for ease of explanation, but the important idea is that the program uses memory through these three regions:

- code

- stack
- heap

---

9. Example program used by the instructor

The instructor uses a simple main() function with two variables:

int a;

float b;

What this code means

- a is an integer variable
- b is a floating-point variable

Why this code is used

The instructor uses this example to explain:

- how variables inside a function use memory
- where that memory is allocated
- why this is called static memory allocation

---

10. Assumed size of data types

The instructor makes a teaching assumption:

- int takes **2 bytes**
- float takes **4 bytes**

So total memory needed is:

- 2 + 4 = 6 bytes

Important warning

The instructor clearly warns that this is **not always the same in all systems**.

For int:

- it may take **2 bytes** or **4 bytes**
- it depends on:
    - compiler
    - operating system
    - hardware

Compiler comparison mentioned

- In **Turbo C** (16-bit compiler), int may take **2 bytes**
- In tools like **Dev Studio** or **CodeBlocks**, int commonly takes **4 bytes**
- These are associated with 32-bit environments in the instructor's explanation

Main learning point

Do not assume that the size of int is always the same everywhere.

---

11. Memory allocation inside the stack

11.1 Local variables go into the stack

The instructor explains that the variables declared inside a function are stored in the **stack**.

For the example:

int a;

float b;

Memory required:

- a ⟶ 2 bytes
- b ⟶ 4 bytes
- total ⟶ **6 bytes**

These 6 bytes are allocated in the **stack**.

11.2 Memory belongs to the function

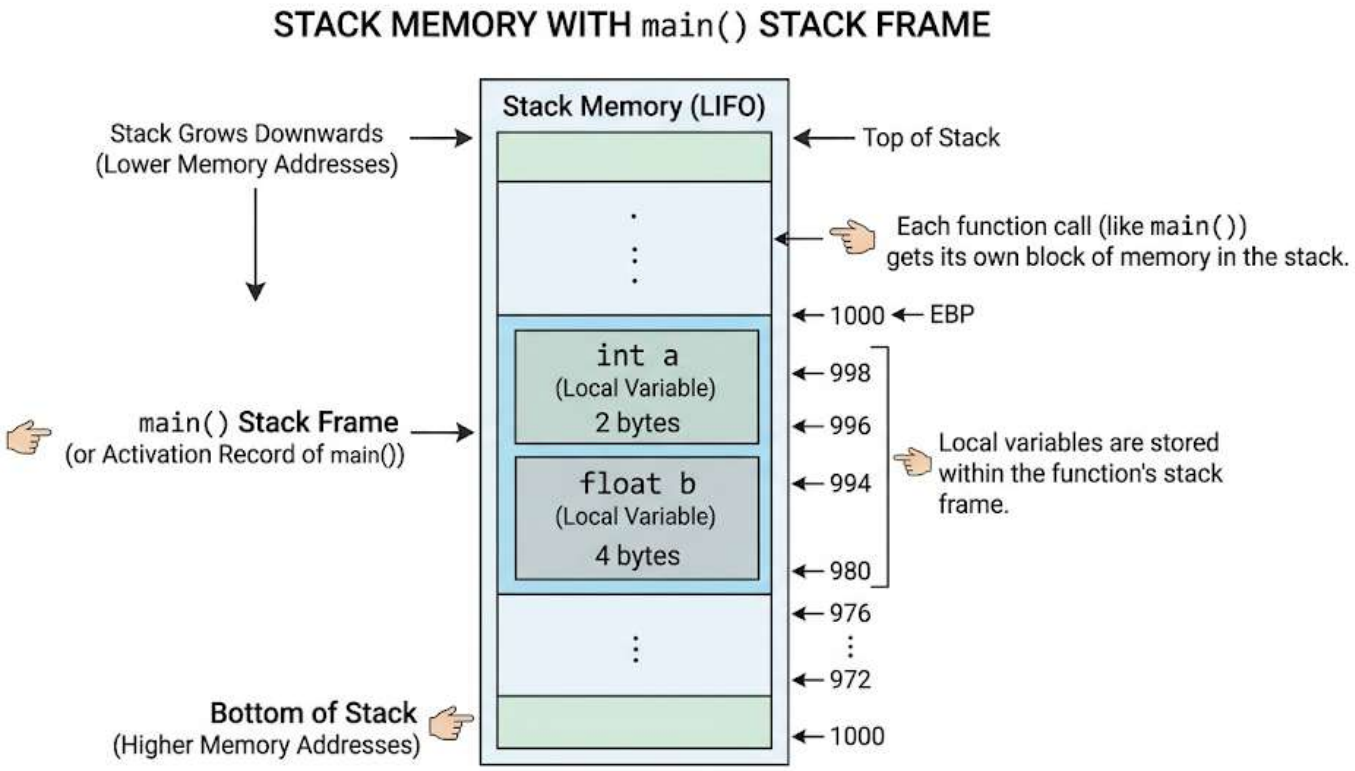
The memory given to a function for its local variables is treated as a block belonging to that function.

That block is called:

- **stack frame**, or
- **activation record**

So for main(), the allocated memory block is called:

- **stack frame of main function**
- or **activation record of main function**



---

12. Stack frame / activation record

Definition

A **stack frame** (or **activation record**) is the block of memory in the stack that belongs to a function.

It contains

From this transcript, the instructor specifically explains that it contains memory for:

- variables declared inside that function

Example

For main():

- integer variable memory
- float variable memory

Together these form the activation record of main() in the example.

Key idea

Every function gets its own stack-related memory area for its local needs.

---

13. How the compiler decides stack memory

The instructor explains an important point:

- the compiler can examine the function before the program runs
- it can see how many variables are declared
- it can calculate how many bytes are needed

So:

- the amount of memory needed by that function is decided **at compile time**

Why?

Because the declarations are already written in the code.

Example:

int a;

float b;

The compiler already knows:

- one integer
- one float
- total required memory

So it can decide the size in advance.

---

14. Static memory allocation

14.1 What “static” means here

The instructor explains that this memory allocation is called **static memory allocation** because:

- the required memory size is decided **before runtime**
- specifically, it is decided at **compile time**

14.2 What is static?

The instructor says the “static” part refers to the fact that:

- the **size of memory** is fixed in advance
- it does not need to be figured out later while the program is running

14.3 In this example

For the function:

int a;

float b;

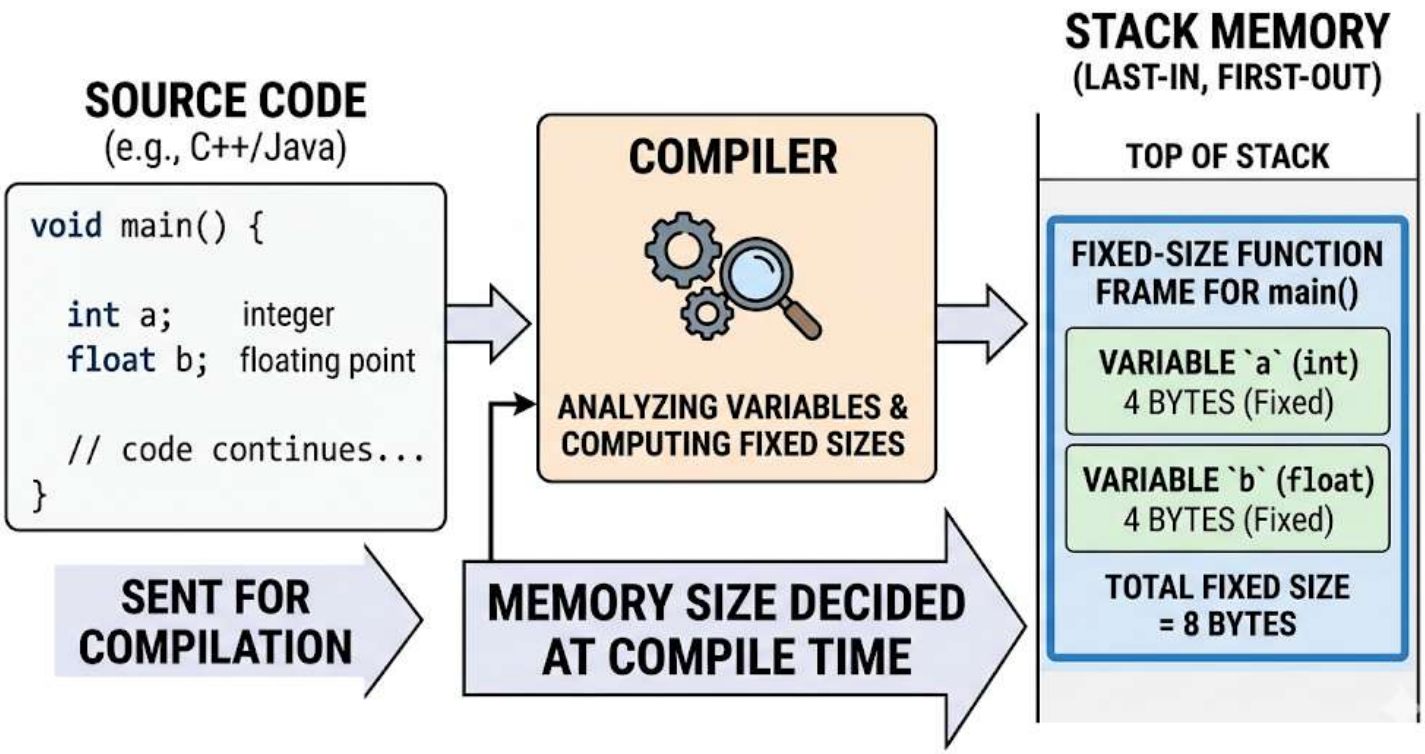
the compiler already knows:

- a needs 2 bytes
- b needs 4 bytes
- total is 6 bytes

So this is static allocation.

Simple definition

Static memory allocation means memory whose size is decided before program execution, usually at compile time.



15. Static vs dynamic memory allocation (based on this transcript portion)

From the provided transcript, the instructor has mainly explained the **static** side.

Static memory allocation

- memory size is known earlier
- decided at compile time
- used for declared local variables in the shown example
- allocated in the stack in this explanation

Dynamic memory allocation

- introduced as a lesson topic
- but not yet fully explained in the provided transcript portion

---

16. Code example with explanation

Example 1

int a;

float b;

What it does

- creates one integer variable
- creates one float variable

Memory used in the instructor's example

- int a ⟶ 2 bytes
- float b ⟶ 4 bytes

- total ⟶ 6 bytes

Where this memory is allocated

- in the **stack**

What concept this example teaches

- local variable memory allocation
- stack usage
- stack frame / activation record
- static memory allocation

Why the instructor used it

The example is very small and easy to understand.
It helps show how the compiler can calculate memory needs in advance.

---

17. Important tips, warnings, and common mistakes

Tip 1: Memory diagrams are simplified

- Memory may be drawn as boxes in 2D
- But real addressing is linear
- Do not think memory addresses work like row and column coordinates

Tip 2: Every byte has its own address

- This is the basic idea of memory organization
- Understanding this helps in learning pointers and memory access later

Tip 3: The 64 KB model is for understanding

- Real systems use much larger memory
- The instructor uses 64 KB only to keep the explanation simple

Warning 1: Data type sizes are not always fixed

- int is not always 2 bytes
- it may be 4 bytes too
- this depends on compiler and system

Warning 2: Do not assume all memory belongs to one simple block in practice

- Real memory may be divided into segments
- the lesson uses one segment for explanation

Common mistake 1: Confusing code section, stack, and heap

Remember:

- **code section** → program instructions
- **stack** → function-related memory such as local variables
- **heap** → mentioned as another memory area, but not yet explained in detail in this part

Common mistake 2: Thinking static means "never changes at all"

In this lesson, "static" is being used in the sense that:

- the **size requirement** is known in advance
- that is why the allocation is called static

---

18. Step-by-step flow explained in the lesson

How a program uses memory

1. A program file exists on the hard disk.
2. To run it, the machine code is loaded into main memory.

3. The loaded instructions go into the **code section**.
4. The CPU begins executing the program.
5. The program uses the remaining memory areas, mainly:
    - **stack**
    - **heap**
6. If a function has local variables, memory for them is allocated in the **stack**.
7. That function's memory block is called a **stack frame** or **activation record**.
8. If the memory size was already known at compile time, that is **static memory allocation**.

---

19. Final recap

- Memory is divided into **bytes**
- Each byte has a unique **linear address**
- The instructor uses a **64 KB** memory model for easy explanation
- A program uses main memory in three main sections:
    - **code section**
    - **stack**
    - **heap**
- The **code section** stores the loaded machine code of the program
- Local variables of a function are stored in the **stack**
- The memory block given to a function is called a:
    - **stack frame**

    - or **activation record**
- The compiler can calculate memory needed for local variables before the program runs
- Because that memory size is decided at **compile time**, it is called **static memory allocation**
- The transcript mainly explains the static part; dynamic memory allocation is introduced but not yet covered in detail in the provided portion

---

20. Unclear or incomplete transcript parts

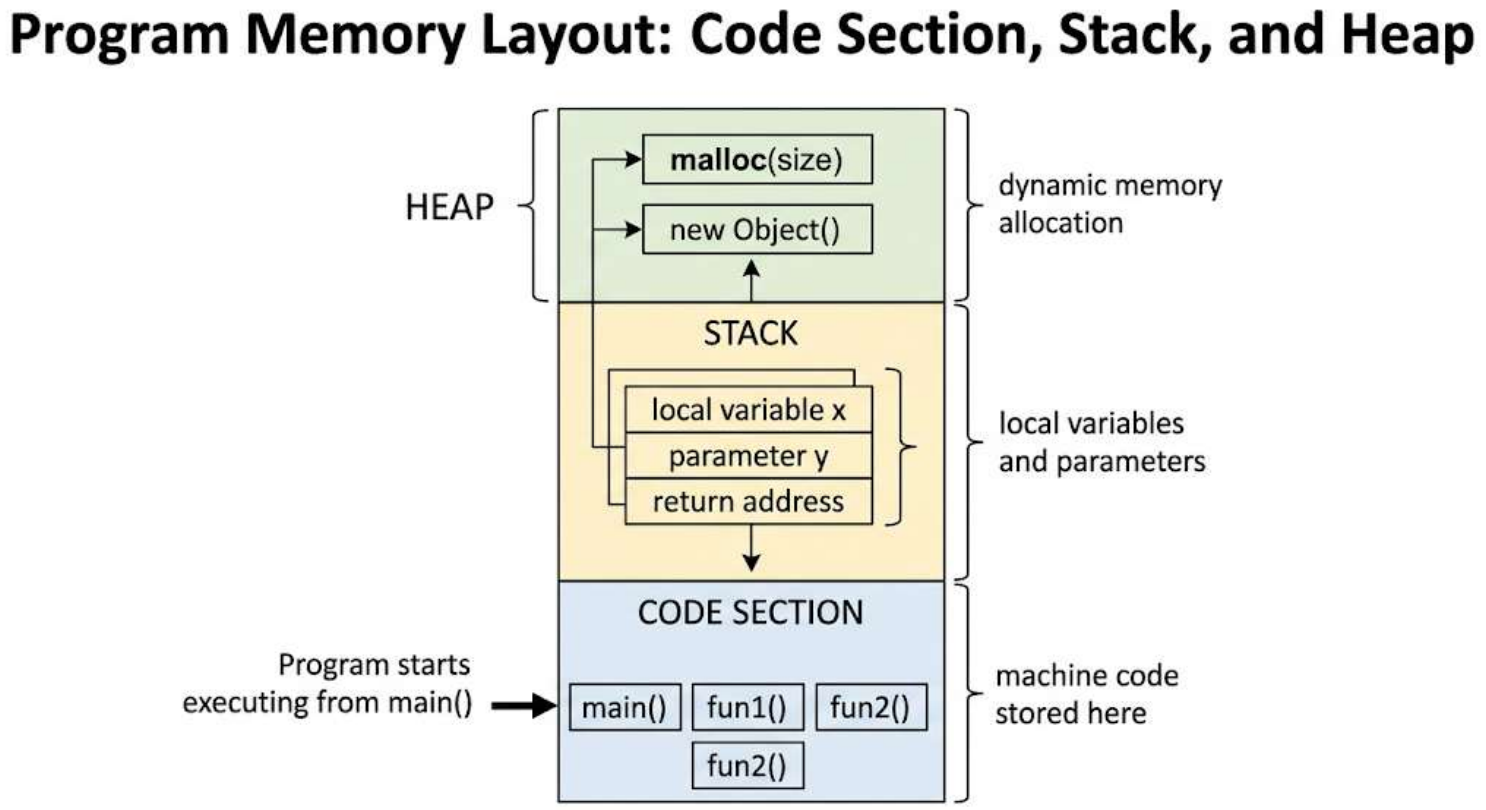
- The transcript ends while discussing static memory allocation.
- The **dynamic memory allocation** part is not included in the provided text.
- The **heap** is introduced, but its actual use is not yet explained in this transcript portion.
- No dynamic allocation code example appears in the provided part.
- No full comparison table between static and dynamic allocation is given yet because the transcript seems incomplete.

Stack Memory and Heap Memory in Function Calls and Dynamic Allocation

These notes are based on the provided transcript. They explain how memory is used during function calls, how stack memory works, how heap memory works, and how static and dynamic memory allocation differ.

1. Lesson Overview

This lesson explains how a program uses main memory while it runs.



The instructor focuses on two major memory areas:

- **Stack memory**
- **Heap memory**

The lesson first shows how memory is allocated in the **stack** during a sequence of function calls.
Then it explains how memory is allocated in the **heap** using pointers.

The lesson ends by comparing:

- **Static memory allocation**
- **Dynamic memory allocation**

---

2. Main Topics

- Memory allocation during function calls
- Activation records in stack memory
- Why stack memory is called a stack
- How memory for local variables and parameters is created
- Heap memory and its meaning
- Heap as a resource
- Why programs use pointers to access heap memory
- Dynamic memory allocation in C++ using new
- Dynamic memory allocation in C using malloc()
- Releasing heap memory
- Memory leaks
- Difference between stack and heap

---

3. Function Calls and Stack Memory

3.1 What happens when a program starts?

When a program starts running:

- The **machine code** of the program is copied into the **code section**.
- Execution begins from the **main function**.
- As the program enters functions, memory is allocated for their variables.

The instructor explains that the code for functions like main(), fun1(), and fun2() exists in the code section. But the variables used inside those functions are stored elsewhere, in the **stack**.

3.2 Sample function call sequence

The instructor describes a simple example:

- main() has two variables
- main() calls fun1()
- fun1() has one local variable x
- fun1() calls fun2(x)
- fun2() receives a parameter and has its own local variable

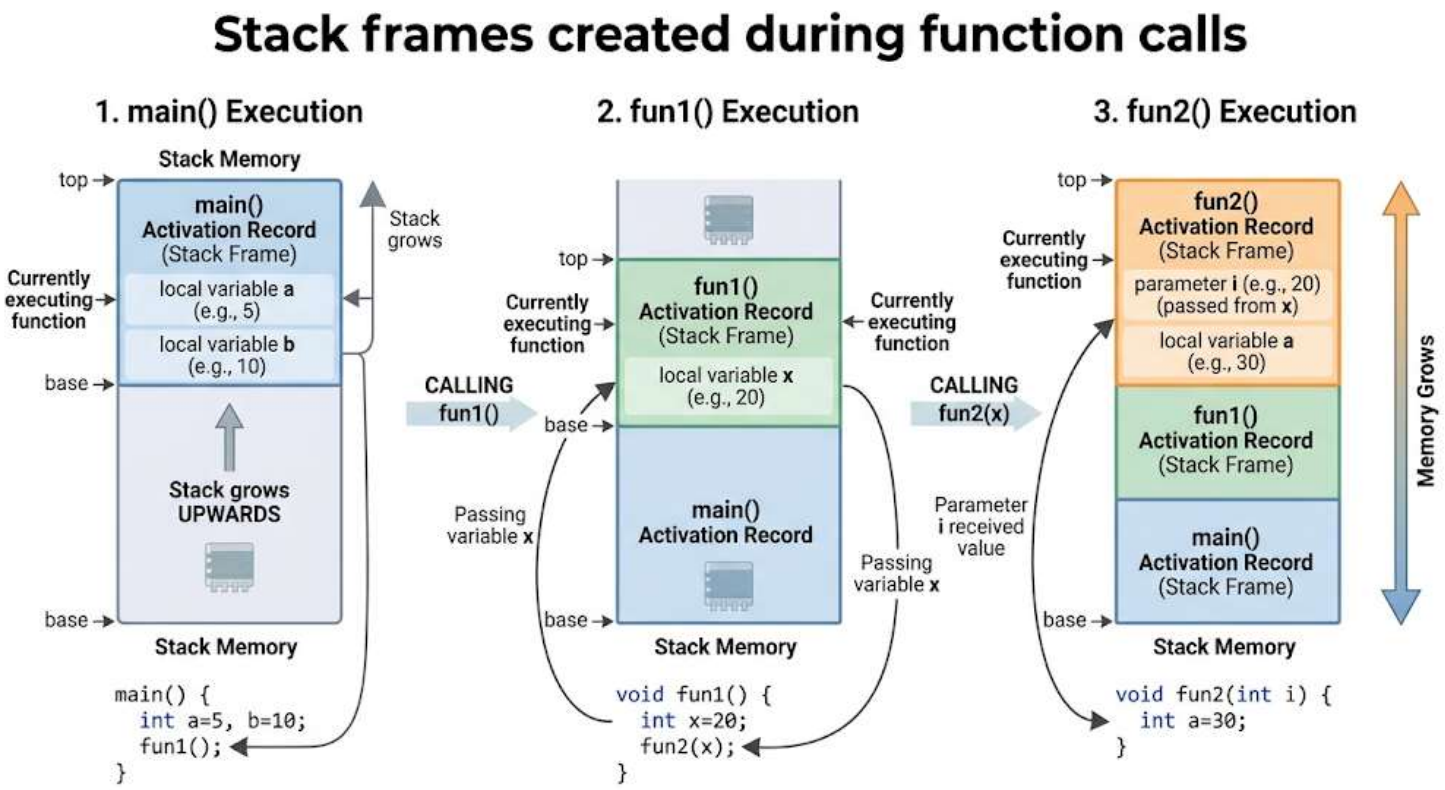
A clean version of the example, based on the explanation, is:

```
void fun2(int i) {
    int a;
}

void fun1() {
    int x;
    fun2(x);
}

int main() {
    int a, b;
```

```
    fun1();
}
```



What this code shows

- main() creates its own local variables.
- fun1() creates its own local variable.
- fun2() creates memory for:
    - its parameter
    - its local variable

Why the instructor used this code

The instructor used this example to show how memory is allocated in the stack when one function calls another.

3.3 Activation record

Each function gets its own block of memory in the stack.

This block is called an **activation record**.

An activation record stores:

- local variables
- function parameters
- other information needed while the function is running

Example from the lesson

When main() starts:

- memory is allocated for its variables a and b

When main() calls fun1():

- memory for fun1() is created above main() in the stack

When fun1() calls fun2():

- memory for fun2() is created above fun1() in the stack

So the stack grows like this:

1. main()
2. fun1()
3. fun2()

The most recently called function is at the **top** of the stack.

3.4 Which function is currently executing?

The currently executing function is the one whose activation record is at the **top of the stack**.

So in the example:

- after entering main(), main() is executing
- after main() calls fun1(), fun1() is executing
- after fun1() calls fun2(), fun2() is executing

This means:

- the **topmost activation record** belongs to the function that is currently running

---

3.5 What happens when a function finishes?

When a function ends:

- its activation record is removed from the stack
- control returns to the function that called it

In the example

Step 1: fun2() finishes

- fun2() ends
- its activation record is deleted
- control returns to fun1()

Step 2: fun1() finishes

- fun1() ends
- its activation record is deleted
- control returns to main()

Step 3: main() finishes

- main() ends
- its activation record is deleted

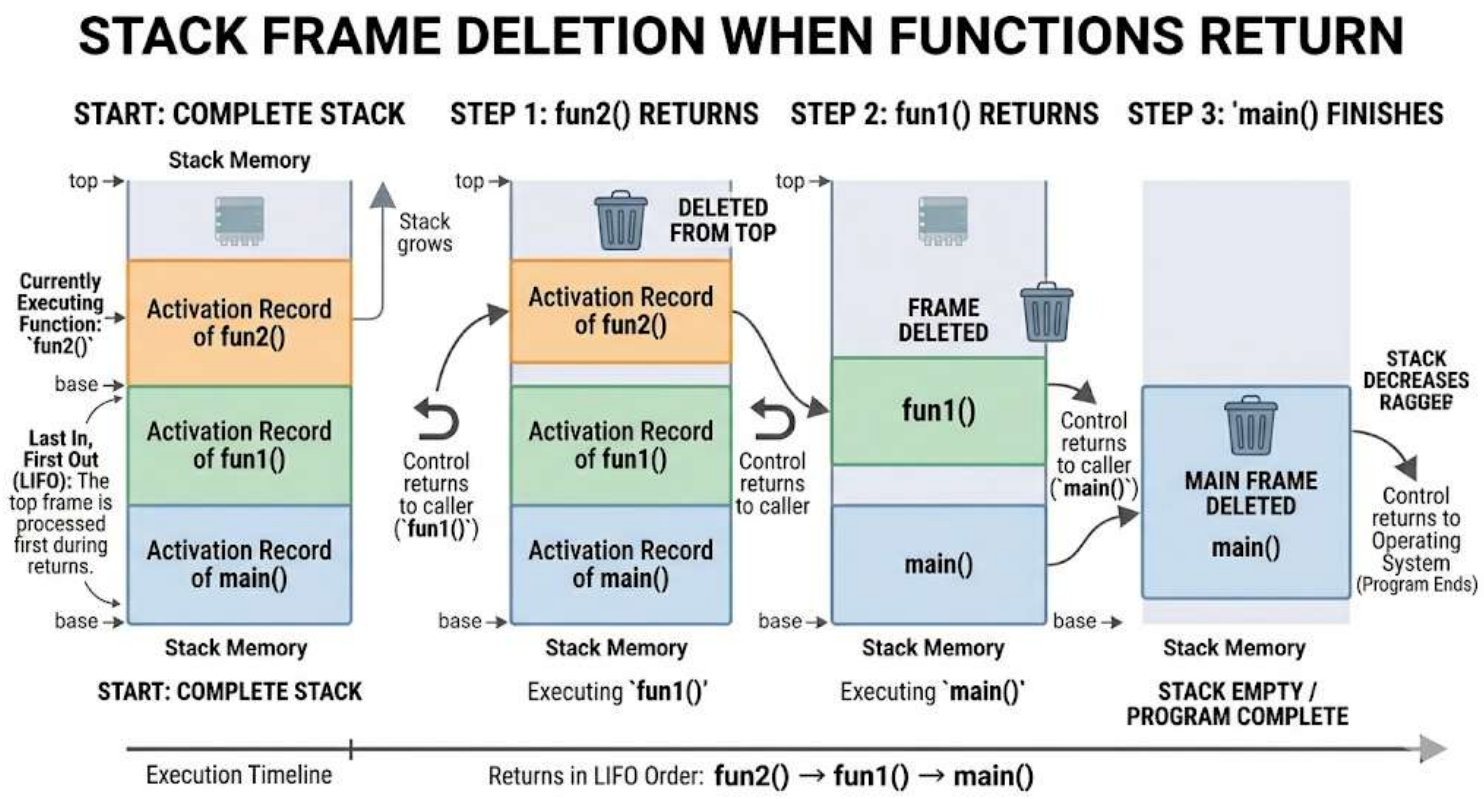
- the program ends

So the deletion happens in reverse order:

1. fun2()
2. fun1()
3. main()

This behavior matches the rule of a stack:

- **Last In, First Out (LIFO)**



3.6 Why this memory area is called a stack

The instructor explains that function memory is created and removed in stack order:

- first pushed
- then pushed again

- then pushed again
- then removed from the top
- then removed from the top
- then removed from the top

That is exactly how a **stack** works.

So this memory section is called **stack memory** because function calls behave like stack operations.

3.7 Important conclusion about stack memory

The instructor makes these important points:

- Memory for local variables is allocated in the **stack**
- Memory for function parameters is also allocated in the **stack**
- This memory is created **automatically**
- This memory is destroyed **automatically** when the function ends
- The programmer does not manually allocate or delete this memory

What decides how much stack memory a function needs?

It depends on:

- number of variables
- sizes of variables

The instructor says this is decided by the **compiler**.

---

4. Static Memory Allocation

4.1 What is static memory allocation in this lesson?

In this lesson, the instructor uses the term **static memory allocation** for memory that is automatically given to variables declared inside functions.

This includes:

- local variables
- function parameters

This allocation happens in the **stack**.

Key points

- happens automatically
- managed by the compiler/runtime
- no manual request from the programmer
- automatically destroyed when function execution ends

---

5. Heap Memory

5.1 Introduction to heap memory

After stack memory, the instructor moves to **heap memory**.

Heap memory is used for **dynamic memory allocation**.

This means:

- memory is requested when needed during program execution
- memory is not automatically removed like stack memory
- the programmer must manage it

5.2 Meaning of the word “heap”

The instructor explains the everyday meaning of the word **heap**:

- things piled one above another
- may be organized
- may be unorganized

Then he connects this idea to memory.

In this lesson

Heap memory is described as:

- **unorganized memory**

This is different from stack memory, which is organized in a strict order.

5.3 Heap should be treated like a resource

One of the most important ideas in the lesson is this:

Heap memory should be treated like a resource.

The instructor compares it to using a printer:

- if a program needs a printer, it requests the printer
- after using it, it releases the printer
- then other programs can use it

Heap memory should be handled in the same way:

- request memory when needed
- release memory when it is no longer needed

This is a very important programming habit.

5.4 Can a program directly access heap memory?

The instructor says:

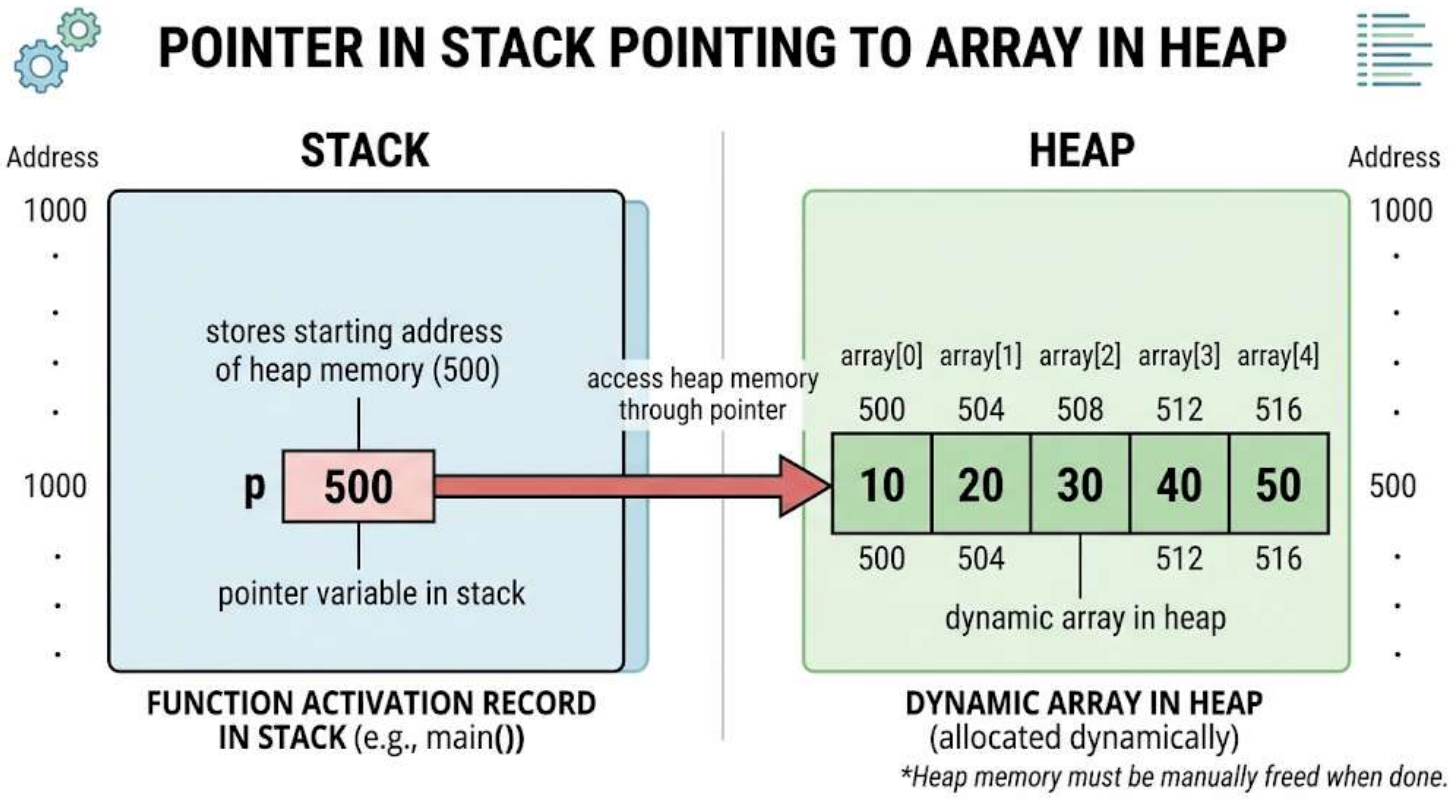
- programs do **not** directly access heap memory in normal practice
- heap memory is accessed **through pointers**

So the proper way to use heap memory is:

1. declare a pointer
2. allocate memory in the heap
3. store the address of that memory in the pointer
4. use the pointer to reach the allocated memory

---

6. Pointer and Heap Allocation



6.1 Where is the pointer stored?

The pointer itself is just a variable.

So its memory is allocated in the **stack**, inside the function's activation record.

The heap memory it points to is allocated separately in the **heap**.

Important idea

- **Pointer variable** → stored in stack
- **Dynamic array or object** → stored in heap

This is a key concept.

6.2 Pointer size

The instructor simplifies the discussion by assuming:

- pointer size = 2 bytes

He also says that pointer size usually depends on the integer size in the machine model being discussed.

This is a simplification used for explanation.

Important note

Modern systems often use larger pointer sizes, but the instructor is using a simplified model here for teaching.

---

7. Dynamic Memory Allocation in C++

7.1 C++ example using new

The instructor explains dynamic allocation of an integer array of size 5 using C++.

A clean version of the code is:

```
int *p;

p = new int[5];
```

What this code does

- p is a pointer variable

- p is stored in the stack
- new int[5] allocates an array of 5 integers in the heap
- the starting address of that heap memory is stored in p

Why the instructor used this code

The instructor used this example to show how heap memory is requested in C++.

Concept related to this code

- dynamic memory allocation
- pointer-based access to heap memory

7.2 How access works

The program does not directly work with heap memory by itself.

Instead:

- p stores the address of the allocated memory
- the program uses p to access the array in the heap

So the pointer acts like a bridge between the stack and the heap.

---

8. Dynamic Memory Allocation in C

8.1 C example using malloc()

The instructor also explains the equivalent idea in C.

A clean version is:

```
int *p;

p = (int *)malloc(5 * sizeof(int));
```

What this code does

- malloc() allocates memory in the heap
- memory is enough for 5 integers
- the returned address is typecast to int *
- that address is stored in p

Why the instructor used this code

The instructor used this to compare C++ and C styles of heap allocation.

Concept related to this code

- dynamic memory allocation in C
- use of malloc()
- storing heap addresses in pointers

Instructor note

The instructor says the C version is more lengthy, so he prefers to use the C++ new form in later discussion.

---

9. Releasing Heap Memory

9.1 Why allocated heap memory must be released

The instructor strongly emphasizes this point:

If heap memory is allocated and later no longer needed, it must be released explicitly.

Heap memory is **not** automatically destroyed like stack memory.

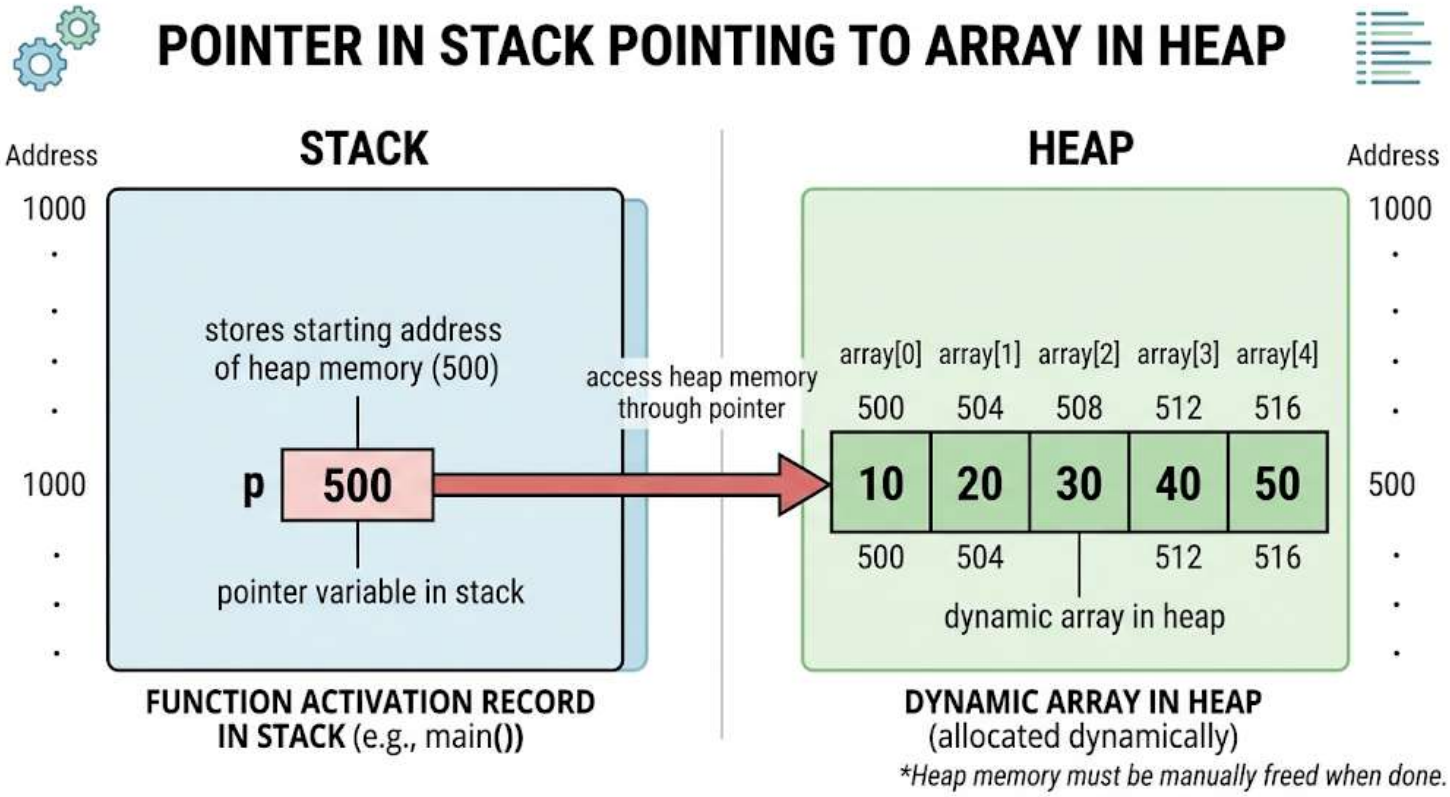
9.2 What happens if a pointer is set to null without deleting memory?

The instructor explains an important mistake:

Suppose p points to heap memory.
If you simply make p = null or p = 0 without releasing the heap memory first:

- the pointer stops pointing to that memory
- but the heap memory is still allocated
- no pointer may remain to reach it
- the memory is wasted

This leads to memory loss.



9.3 Correct way in C++

A clean version of the release process is:

int *p;

p = new int[5];

```
// use the array


delete[] p;

p = nullptr;
```

What this code does

- delete[] p; releases the array allocated with new int[5]
- p = nullptr; removes the pointer reference safely after deletion

Why this matters

This prevents memory from remaining allocated when it is no longer needed.

9.4 Correct way in C

Equivalent C version:

```
int *p;

p = (int *)malloc(5 * sizeof(int));


// use the array


free(p);

p = NULL;
```

What this code does

- free(p); releases the heap memory
- p = NULL; clears the pointer

---

10. Memory Leak

10.1 What is a memory leak?

The instructor defines the loss of memory as a **memory leak**.

A memory leak happens when:

- memory is allocated in the heap
- the program no longer uses it
- but it is not released

As a result:

- that memory still belongs to the program
- other parts of the program cannot use it properly
- the memory becomes wasted

10.2 Why memory leaks are dangerous

If this happens repeatedly:

- heap memory keeps getting filled
- free memory becomes less and less
- eventually, heap memory may become full

Then the program may not get more memory when it needs it.

This can cause serious problems in long-running programs.

---

11. Stack vs Heap

11.1 Main differences

Stack	Heap
Used for local variables and function parameters	Used for dynamically allocated memory
Memory is automatically created	Memory must be explicitly requested
Memory is automatically destroyed	Memory must be explicitly released
Organized structure	Unorganized memory area
Works like a stack during function calls	Accessed through pointers
Managed automatically by compiler/runtime	Managed by programmer

---

11.2 Simple comparison in words

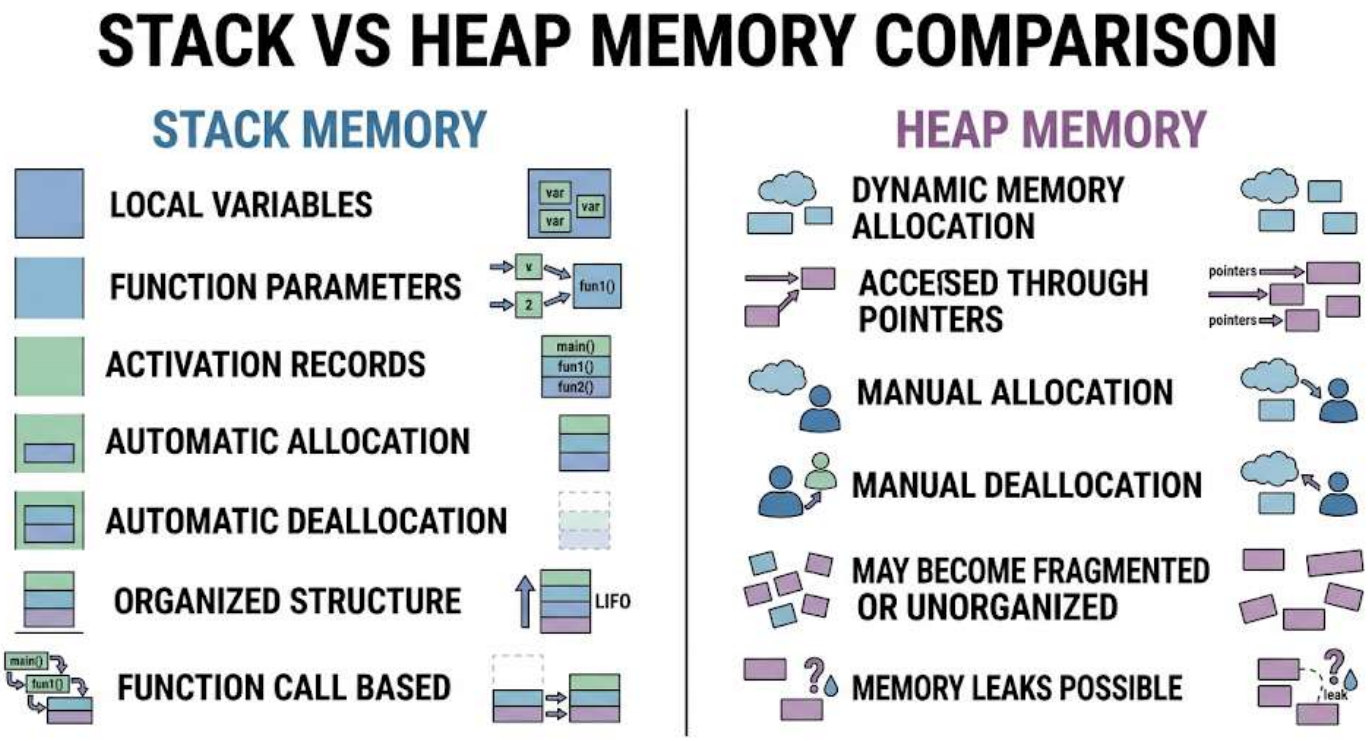
Stack memory

- used during function calls
- stores activation records
- stores local variables
- stores parameters
- automatic allocation
- automatic destruction

Heap memory

- used when memory is needed dynamically
- allocated using new in C++
- allocated using malloc() in C

- must be released manually
- accessed through pointers
- can cause memory leaks if not released



---

12. Step-by-Step Process Summary

12.1 Stack during function calls

1. Program starts
2. Execution enters main()
3. Memory for main() variables is created in stack
4. main() calls fun1()
5. Memory for fun1() variables is created above main()
6. fun1() calls fun2()

7. Memory for fun2() parameter and local variable is created above fun1()
8. fun2() finishes and its memory is removed
9. Control returns to fun1()
10. fun1() finishes and its memory is removed
11. Control returns to main()
12. main() finishes and its memory is removed

12.2 Heap allocation process

1. Declare a pointer in a function
2. Pointer gets memory in the stack
3. Use new or malloc() to request heap memory
4. Store returned address in the pointer
5. Access heap memory through the pointer
6. When done, release the heap memory
7. Set the pointer to nullptr or NULL

---

13. Important Instructor Tips, Warnings, and Common Mistakes

13.1 Tips

- Treat heap memory like a resource
- Take memory only when needed
- Release memory when it is not needed
- Remember that local variables and parameters use stack memory

- Understand that the currently executing function is the one at the top of the stack

13.2 Warnings

- Do not forget to release heap memory
- Repeated failure to release memory causes memory leaks
- Too many memory leaks can fill the heap

13.3 Common mistakes

- Thinking heap memory is removed automatically
- Setting a pointer to NULL or nullptr before freeing/deleting memory
- Forgetting that the pointer is in stack but the data it points to is in heap
- Confusing stack allocation with dynamic allocation

---

14. Final Recap

This lesson explains how memory works during program execution.

Main conclusions

- Function calls use **stack memory**
- Each function gets an **activation record**
- The currently executing function is at the **top of the stack**
- Stack memory is created and removed automatically
- Local variables and parameters are stored in the stack
- Dynamic memory is allocated from the **heap**
- Heap memory is accessed using **pointers**
- In C++, new is used for heap allocation

- In C, malloc() is used
- Heap memory must be released explicitly
- Failure to release heap memory causes **memory leaks**

The lesson ends by saying that this explains the difference between **stack** and **heap**, and the next lesson will introduce different data structures.

---

15. Unclear or Incomplete Transcript Parts

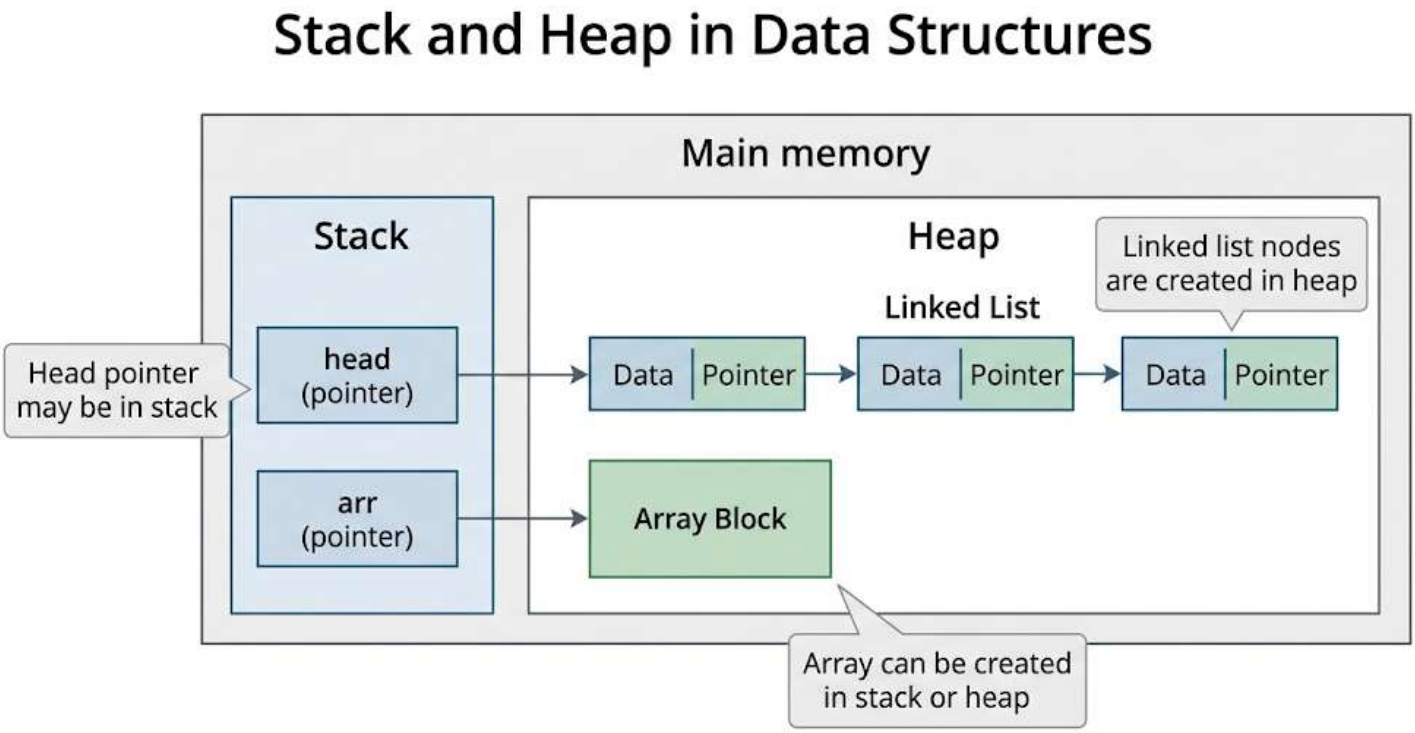
Some parts of the transcript appear to come from a spoken explanation without the full written code visible. These parts should be treated carefully:

- The exact original code shown for the function-call example was not fully visible, but the explanation clearly described:
  - main() with two variables
  - fun1() with local variable x
  - fun2() with parameter i and local variable a
- The C malloc() line was described verbally, but the transcript text around integer size formatting was partially messy.
- The exact delete syntax for the array was not perfectly visible in the transcript text. The lesson clearly intended array deallocation, but the displayed wording was incomplete in places. So the precise original written form should be treated as **[unclear]** in the raw transcript, even though the concept itself is clear.

Introduction to Data Structures — Detailed Study Notes

These notes are based on the provided transcript.

1. **Lesson Overview**



This lesson introduces the basic idea of **data structures**.

The instructor connects this topic to the previous lesson, where memory usage in a program was discussed. That earlier topic included:

- stack and heap memory
- static memory allocation
- dynamic memory allocation

In this lesson, the instructor explains that data structures can be grouped into two broad categories:

- **Physical data structures**
- **Logical data structures**

The main goal of the lesson is to help learners understand:

- what these two categories mean
- how they are different
- how they are connected to each other

---

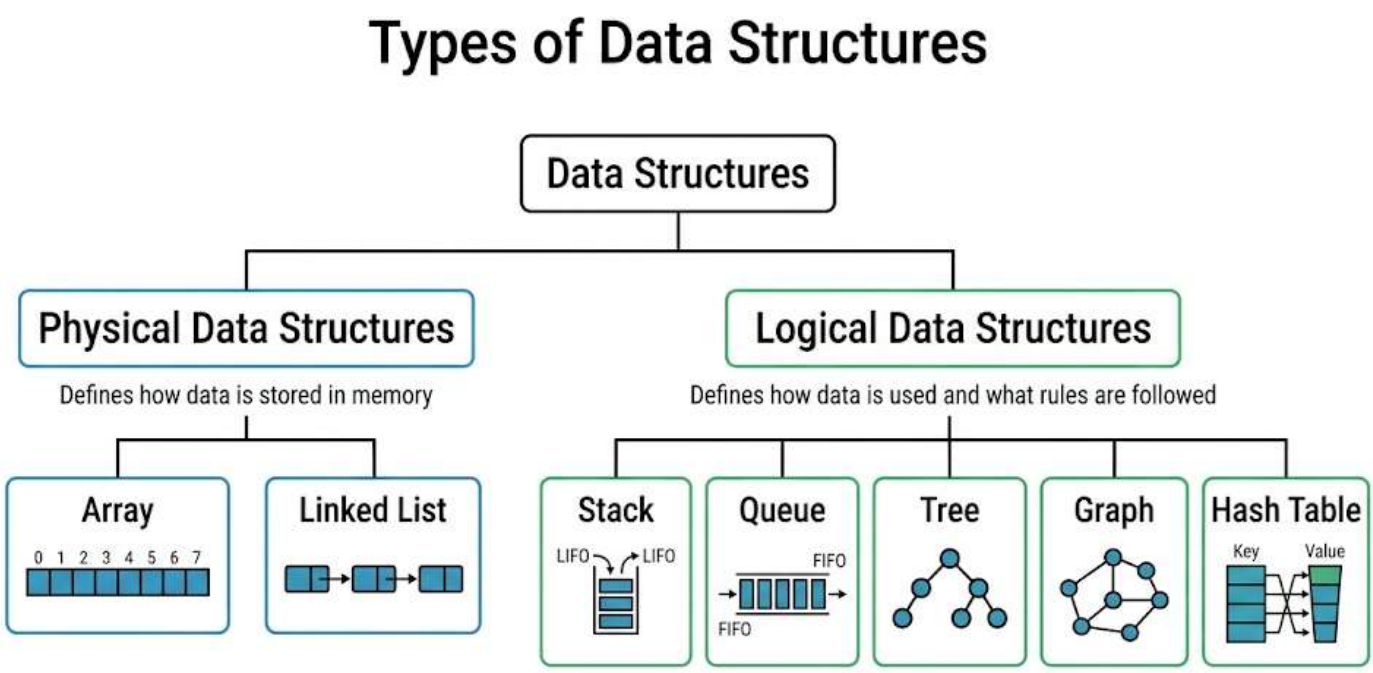
2. Main Topics Covered

Main topics in this lesson:

- Physical data structures
- Array
- Linked List
- Logical data structures
- Stack
- Queue
- Tree
- Graph
- Hash Table
- Relationship between physical and logical data structures
- Course direction for upcoming lessons

---

3. Physical Data Structures



3.1 Meaning of Physical Data Structures

The instructor says that **physical data structures** are called “physical” because they define:

- how memory is organized
- how memory is allocated

So, these data structures are closely related to the actual storage of data in memory.

Physical data structures introduced:

- **Array**
- **Linked List**

The instructor also says that other physical forms can exist as variations or combinations of these two, but these are the two basic ones.

3.2 Array

Definition

An **array** is a data structure that stores elements in **contiguous memory locations**.

That means:

- all elements are stored side by side
- they are placed together in memory
- there are no gaps between them

Example explained by the instructor

If an array stores **7 integers**, then memory for those 7 integers is allocated together in one continuous block.

3.3 Properties of an Array

1. Contiguous memory

- All elements are stored next to each other.
- This is one of the main features of arrays.

2. Fixed size

- Once an array is created with a specific size, that size cannot be changed.
- It cannot be increased or decreased later.
- So, the size of an array is **static**.

3. Direct language support

The instructor says arrays are directly supported in programming languages such as:

- C
- C++
- Java

So arrays are built-in or directly available in many programming languages.

3.4 Where an Array Can Be Created

The instructor explains that an array can be created in:

- **stack**
- **heap**

Also, a pointer can point to the array.

So, arrays are flexible in terms of memory location. They are not restricted to only one area of memory.

3.5 When to Use an Array

The instructor recommends using an array when:

- you know the maximum number of elements in advance
- the length of the list is known
- the size is fixed

Simple idea

Use an array when the number of items is already known or can be safely decided beforehand.

3.6 Important Notes About Arrays

Tips

- Arrays are good when size is fixed.
- Arrays are simple and directly supported in many languages.

Warning

- Do not choose an array if the number of elements will change often.
- Its size cannot grow or shrink after creation.

---

4. Linked List

4.1 Definition

A **linked list** is a **completely dynamic data structure**.

It is a **collection of nodes**, where:

- each node contains data
- each node is linked to the next node

So, unlike an array, the elements are not stored in one continuous block.

4.2 Properties of a Linked List

1. Dynamic size

The size of a linked list can:

- grow dynamically
- reduce dynamically

This means you can add more nodes when needed, or remove nodes when needed.

2. Variable length

A linked list does not have a fixed size like an array.

Its length depends on current need.

3. Node-based structure

Each element is stored inside a node.

Each node has:

- data
- link to the next node

4.3 Where a Linked List Is Created

The instructor clearly states:

- the linked list nodes are always created in the **heap**
- the **head pointer** may be in the **stack**

So:

- actual list nodes → heap
- pointer to the list → may be in stack

This is an important memory-related point.

4.4 When to Use a Linked List

The instructor recommends using a linked list when:

- the size of the list is not known
- the list may grow or shrink
- flexibility is needed

Simple idea

Use a linked list when the number of elements is not fixed in advance.

4.5 Important Notes About Linked Lists

Tips

- Linked lists are suitable for variable-size data.
- They allow dynamic insertion of new nodes.
- They allow size reduction when needed.

Warning

- Do not assume a linked list stores data continuously in memory like an array.
- The instructor presents it as a structure made of separate linked nodes.

---

5. Why Array and Linked List Are Called Physical Data Structures

The instructor repeats the key reason:

They are called **physical** because they define:

- how memory should be organized
- how elements should be stored in memory

So these structures are more about the **physical storage** of data.

Key idea

Physical data structures are directly related to memory layout.

---

6. Logical Data Structures

6.1 Meaning of Logical Data Structures

The instructor then moves to **logical data structures**.

These include:

- Stack
- Queue
- Tree
- Graph
- Hash Table

The instructor explains that logical data structures are not mainly about how memory is physically allocated.

Instead, they define:

- how stored data is used
- what rules or discipline are followed
- how operations such as insertion, deletion, and searching are performed

---

6.2 Purpose of Logical Data Structures

The instructor says that once data is stored, we often perform operations such as:

- insertion
- deletion
- searching
- other operations

At that point, an important question comes up:

How should these values be used?

The answer depends on the logical data structure.

So logical data structures define the **behavior rules** for handling data.

---

7. Types of Logical Data Structures Mentioned

7.1 Stack

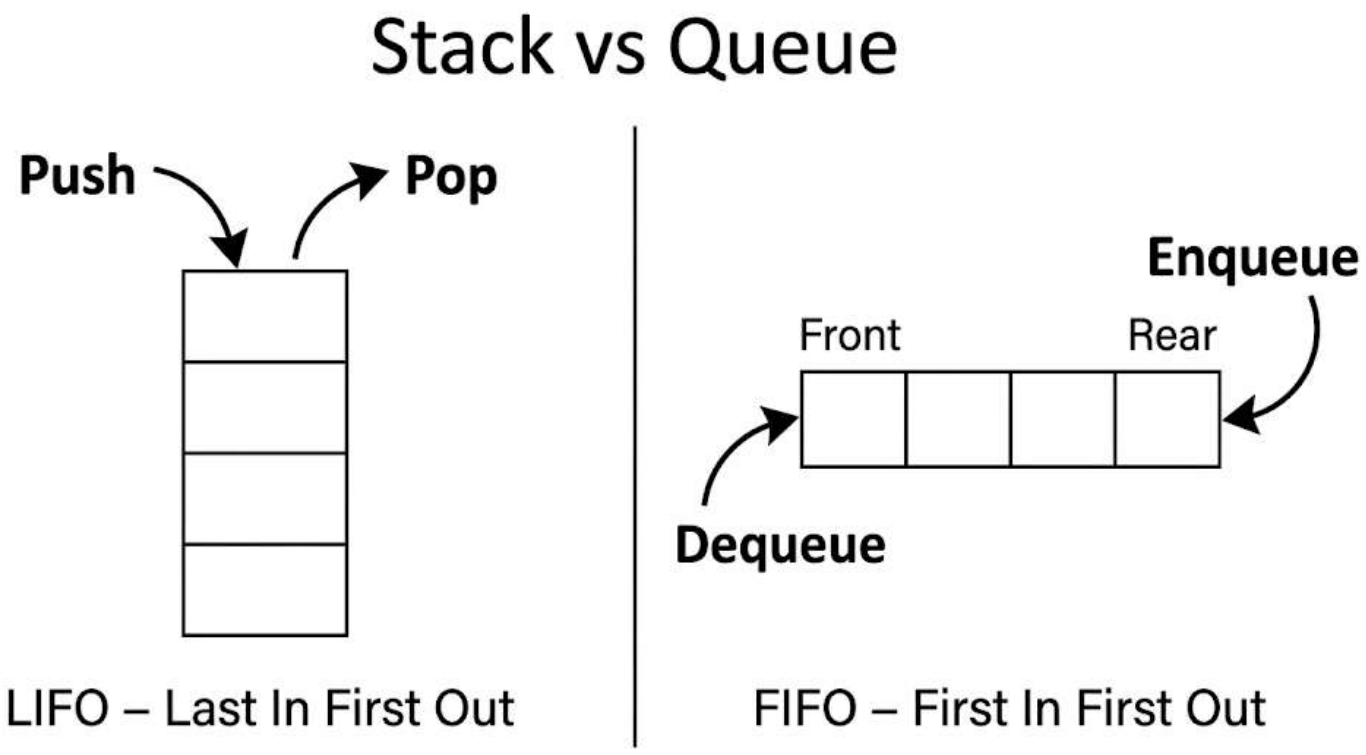
A **stack** follows the discipline:

- **LIFO** = Last In First Out

Meaning

The last element inserted is the first one removed.

This defines the rule of operation for stack behavior.



7.2 Queue

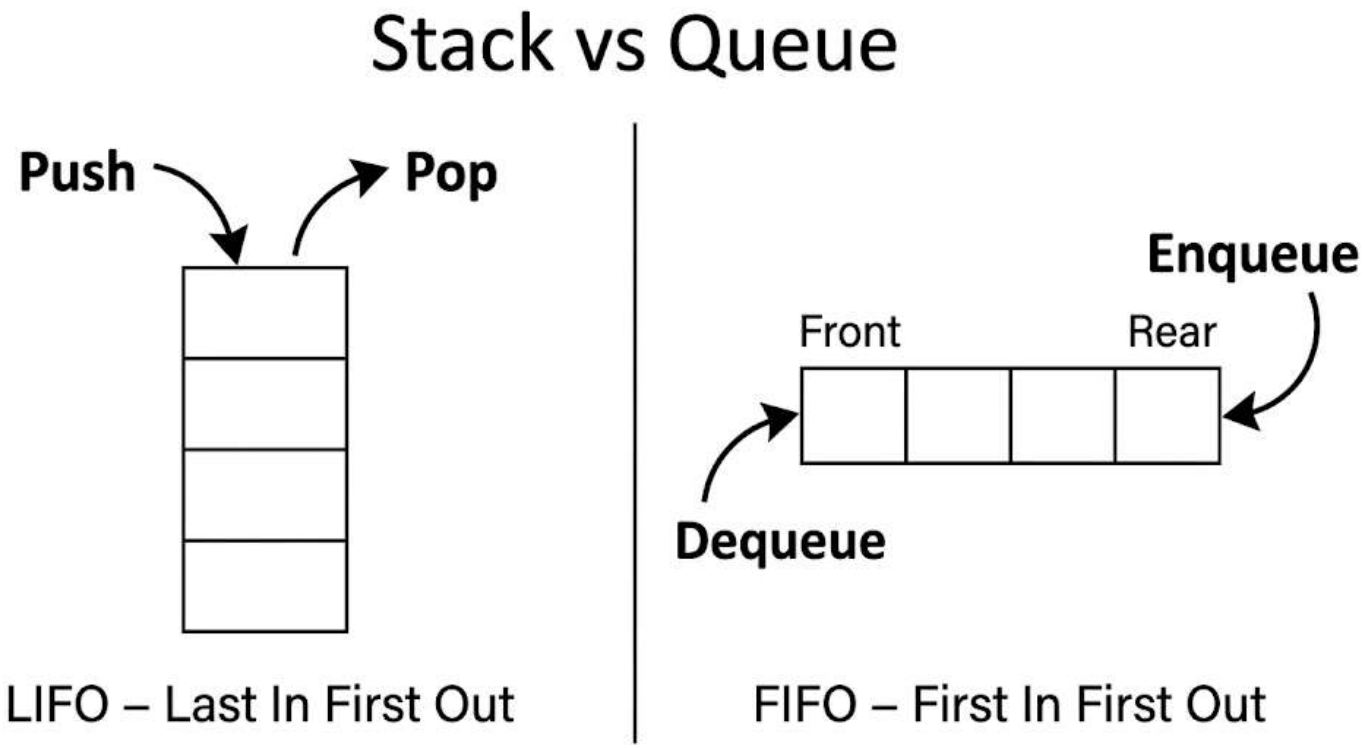
A **queue** follows the discipline:

- **FIFO** = First In First Out

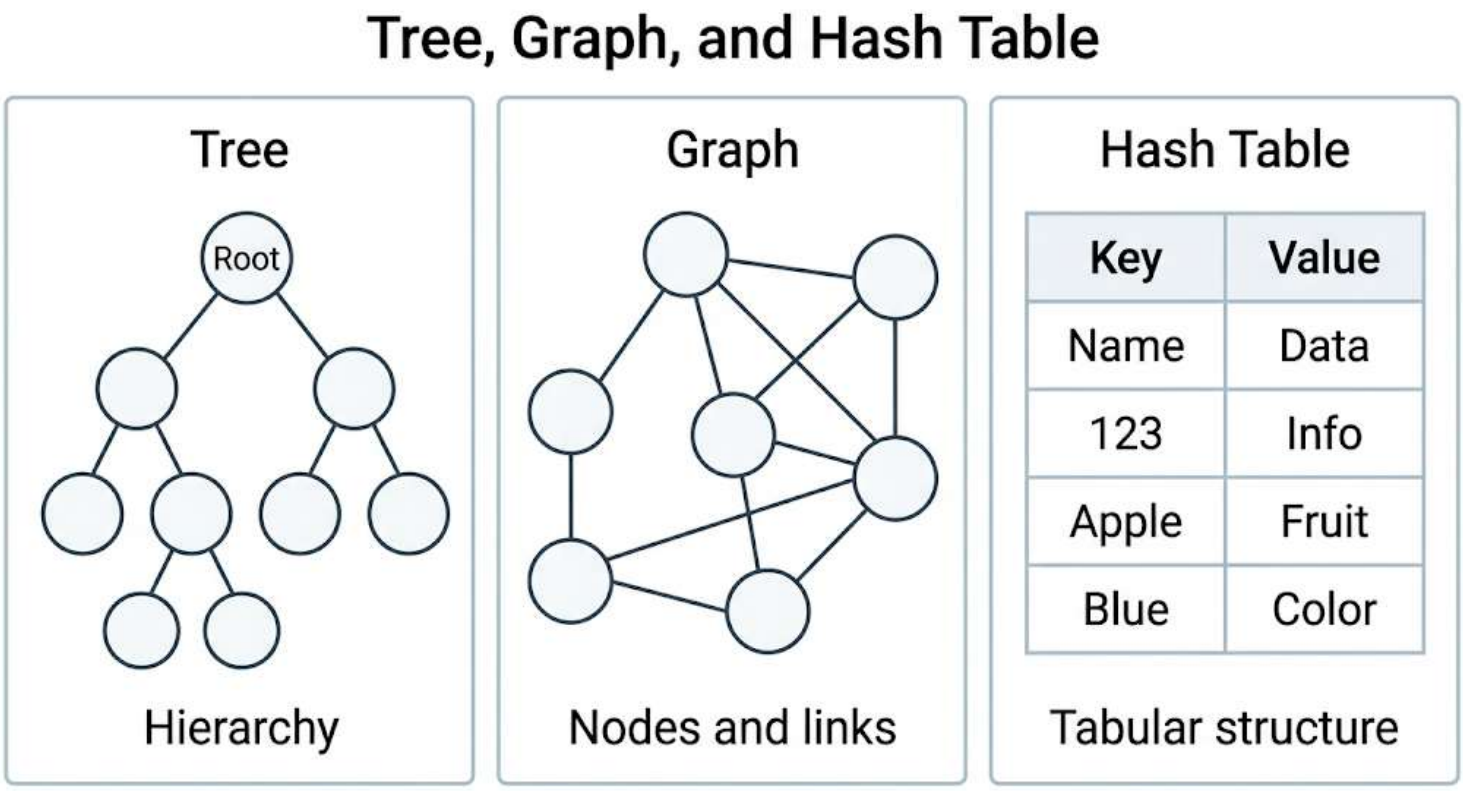
Meaning

The first element inserted is the first one removed.

This defines the rule of operation for queue behavior.



7.3 Tree



A **tree** is described as a **non-linear data structure**.

It is organized like a:

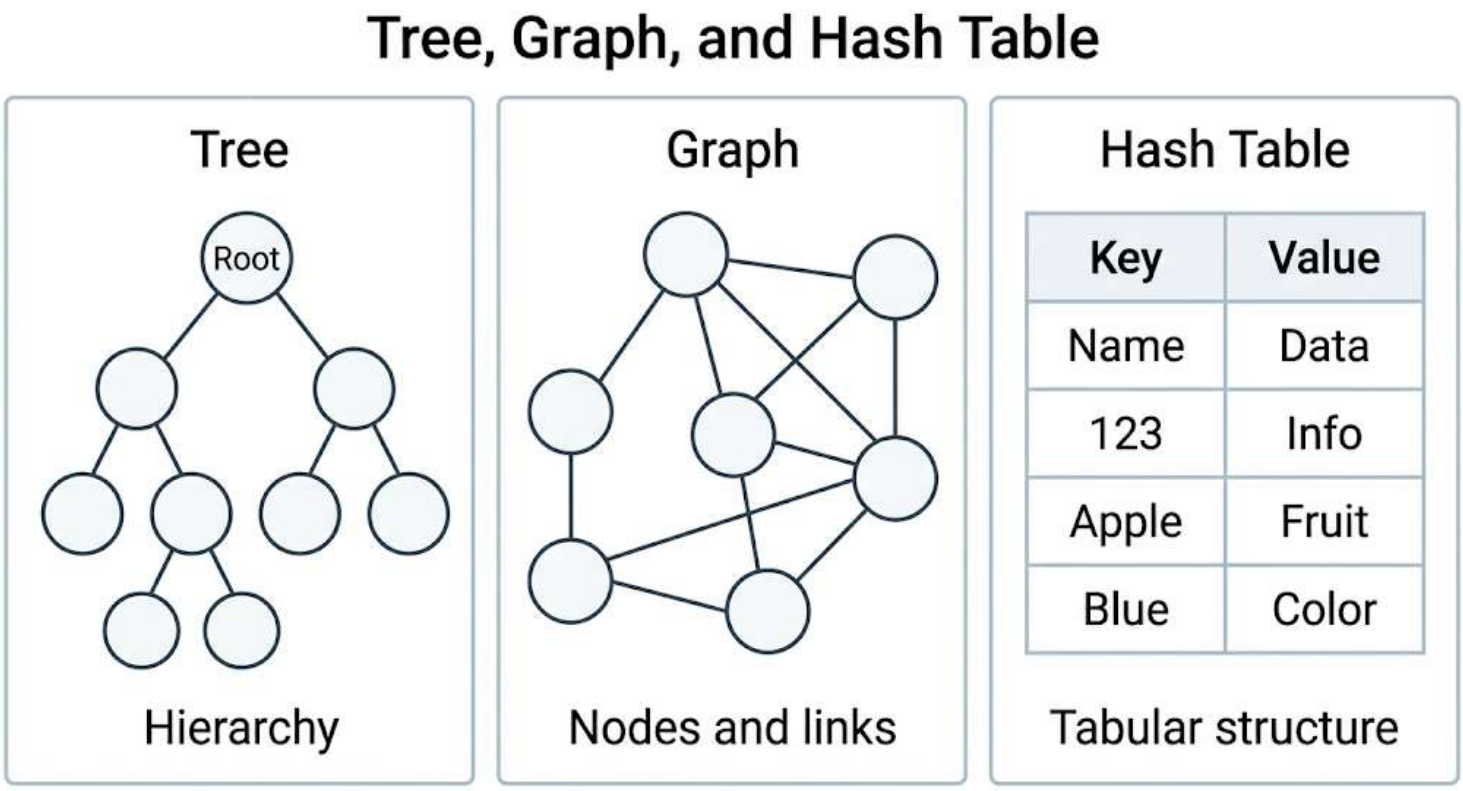
- hierarchy

Meaning

A tree does not store data in a straight sequence like simple linear structures.

Instead, it represents parent-child style organization.

7.4 Graph



A **graph** is also a **non-linear data structure**.

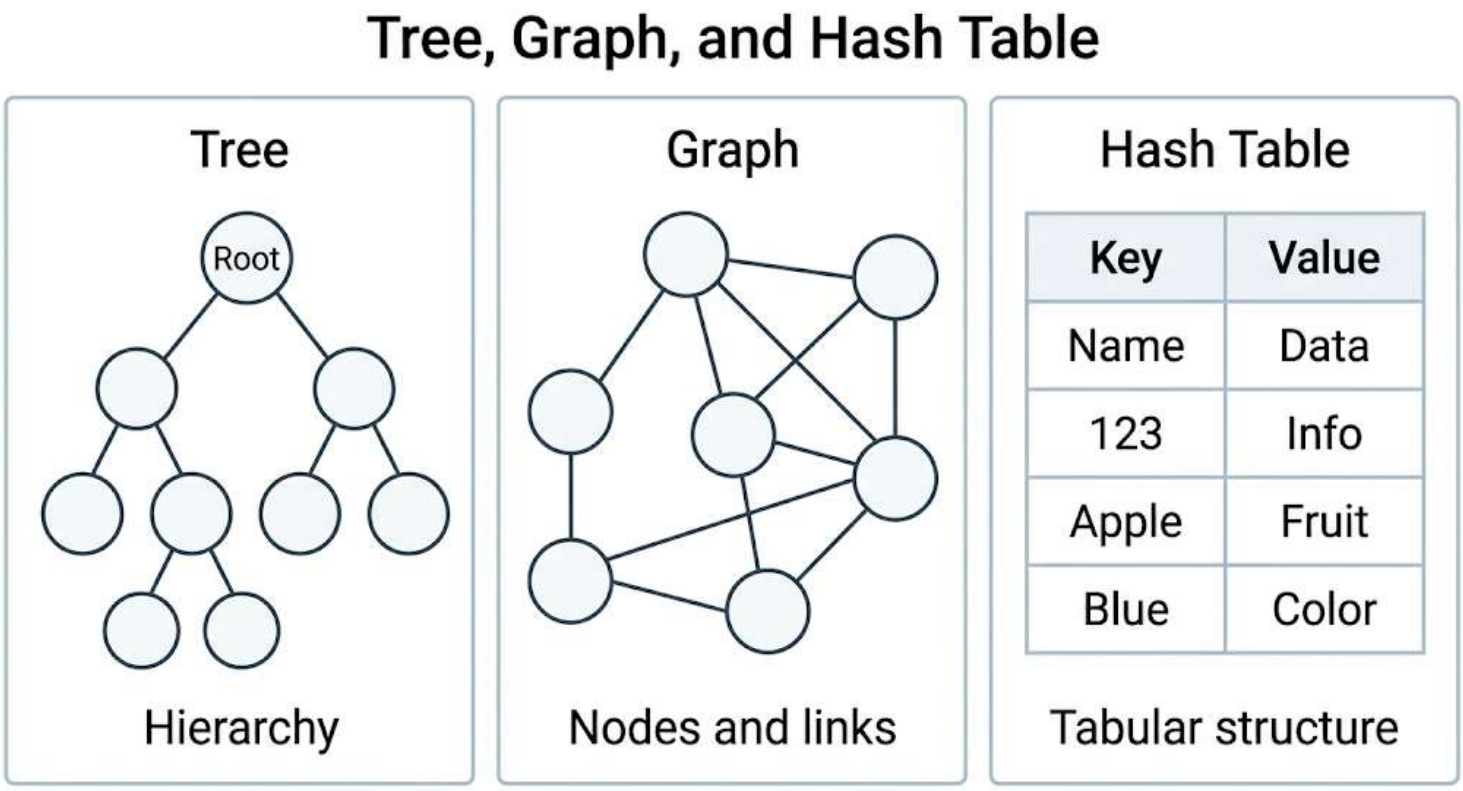
The instructor describes it as:

- a collection of nodes
- links between the nodes

Meaning

Graphs represent relationships between connected nodes.

7.5 Hash Table



The instructor says a **hash table** may be considered:

- linear or
- tabular

Then the instructor specifically refers to it as:

- a **tabular data structure**

So in this lesson, hash table is introduced mainly as a table-like structure.

---

8. Classification Mentioned by the Instructor

8.1 Linear Data Structures

The instructor groups these as linear:

- Stack
- Queue

8.2 Non-linear Data Structures

The instructor groups these as non-linear:

- Tree
- Graph

8.3 Tabular Data Structure

- Hash Table

This helps learners see that logical data structures can also be grouped by shape or arrangement.

---

9. Difference Between Physical and Logical Data Structures

9.1 Physical Data Structures

Physical data structures are used to:

- actually store data
- hold data in memory
- define memory organization

Examples:

- Array
- Linked List

9.2 Logical Data Structures

Logical data structures are used to:

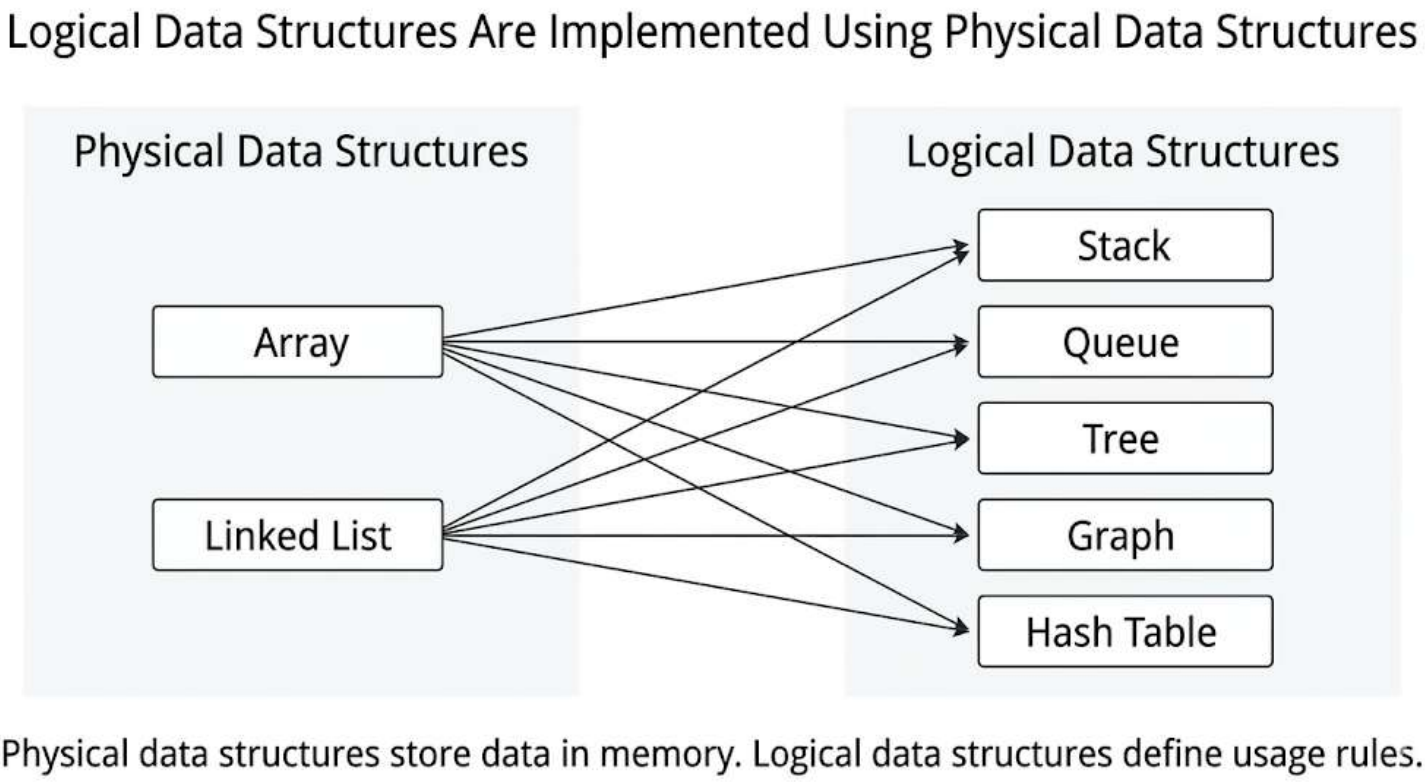
- define how stored data is used
- define rules for insertion, deletion, searching, and other operations
- represent usage discipline or access pattern

Examples:

- Stack
- Queue
- Tree
- Graph
- Hash Table

---

10. Most Important Relationship in This Lesson



Physical data structures store data in memory. Logical data structures define usage rules.

Logical Data Structures Are Implemented Using Physical Data Structures

This is the most important conclusion of the lesson.

The instructor says:

- logical data structures are implemented using physical data structures
- they can be implemented using:
    - array

    - linked list
    - combination of array and linked list

Main idea

Logical data structures describe **how data should behave**.

Physical data structures provide **how data is actually stored in memory**.

So:

- physical data structures = storage foundation
- logical data structures = operational discipline

---

11. Use in Applications and Algorithms

The instructor says that these logical data structures are used in:

- applications
- algorithms

This means they are not just theoretical topics. They are important in practical programming and problem solving.

But to build them, we use physical data structures such as arrays and linked lists.

---

12. Course Direction Mentioned by the Instructor

The instructor explains what will be studied later in the course.

First, the course will cover:

- arrays in detail
- linked lists in detail

This includes:

- implementation
- writing programs for them

After that, the course will move to:

- stack
- queue
- tree
- graph
- hash table
- and more topics under them

The instructor also says that each major topic contains many subtopics.

Examples mentioned:

- different types of queues
- different types of trees
- different types of graphs

So this lesson is only an introduction and not the full coverage of those topics.

---

13. What the Instructor Says About Future Implementations

The instructor says that throughout the course:

- each logical data structure will be implemented using arrays
- each logical data structure will also be implemented using linked lists

This is an important learning plan because it helps students understand that one logical structure can have more than one physical implementation.

---

14. Mention of the Next Topic

The instructor ends by saying that in the next video, the topic will be:

- **ADT**
- various types of lists

ADT likely refers to **Abstract Data Type**, but the instructor only mentions the term here and does not explain it in this transcript.

So in these notes, it should only be recorded as the next lesson topic.

---

15. Comparison: Array vs Linked List

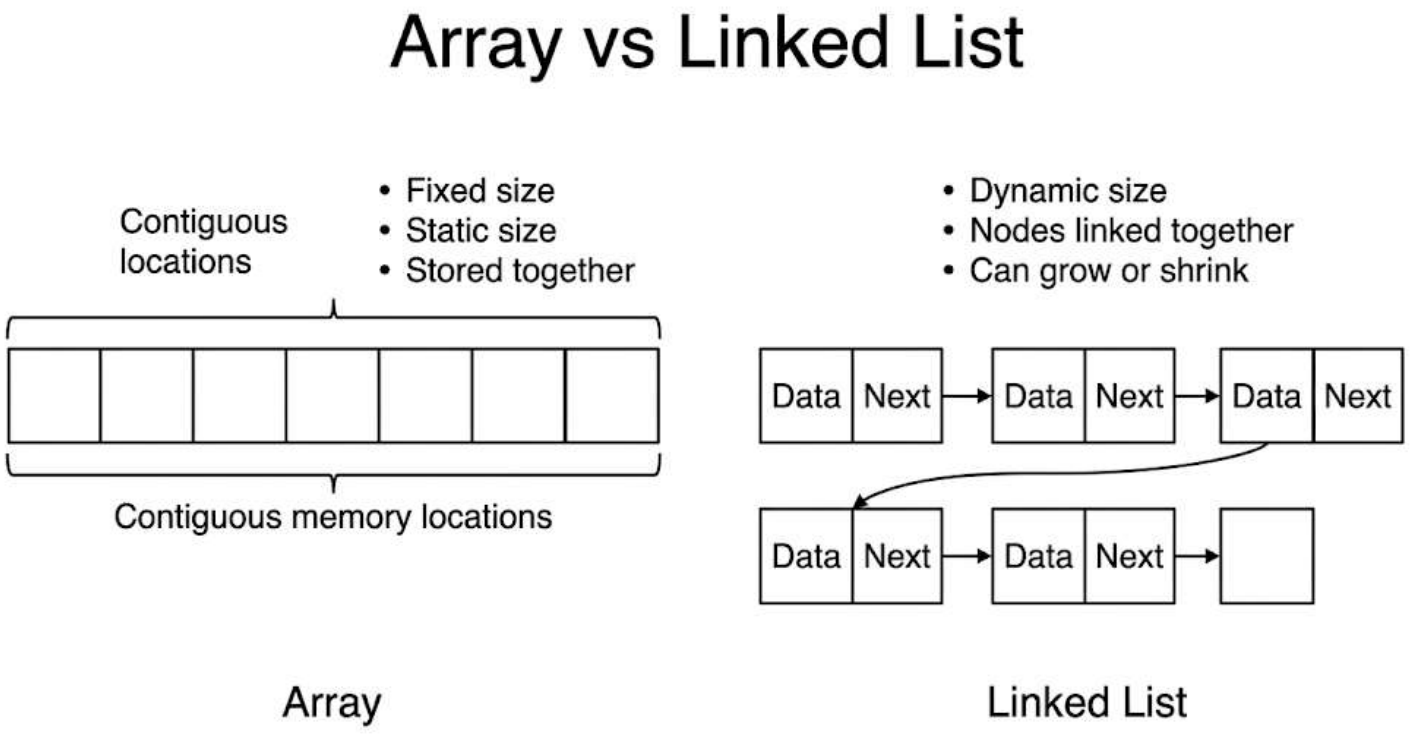
Array

- Stores elements in contiguous memory
- Fixed size
- Static size
- Can be created in stack or heap
- Best when size is known in advance
- Directly supported by languages like C, C++, Java

Linked List

- Stores elements as linked nodes
- Dynamic size
- Variable length
- Nodes are created in heap
- Head pointer may be in stack

- Best when size is not known in advance



---

16. Comparison: Physical vs Logical Data Structures

Physical Data Structures

- Concerned with actual memory organization
- Used to store data physically
- Examples: Array, Linked List

Logical Data Structures

- Concerned with rules for using data
- Define operational discipline
- Examples: Stack, Queue, Tree, Graph, Hash Table

Relationship

- Logical data structures are implemented using physical data structures

---

17. Step-by-Step Understanding of the Lesson

Step 1

Recall that programs use memory through areas like stack and heap.

Step 2

Understand that data structures can be divided into two categories:

- physical
- logical

Step 3

Learn the two basic physical data structures:

- array
- linked list

Step 4

Understand how they differ:

- array = fixed size, contiguous memory
- linked list = dynamic size, linked nodes

Step 5

Learn the main logical data structures:

- stack
- queue
- tree
- graph

- hash table

Step 6

Understand that logical data structures define usage rules such as:

- LIFO
- FIFO
- hierarchy
- node-link relationships
- tabular organization

Step 7

Learn the key conclusion:

- logical data structures are built using physical data structures

---

18. Important Instructor Tips, Warnings, and Common Mistakes

Tips

- Use an array when the size is fixed or known in advance.
- Use a linked list when the size is unknown or changes often.
- Always separate the ideas of **storage** and **usage discipline**.
- Remember that logical structures are often implemented using arrays or linked lists.

Warnings

- Do not confuse physical data structures with logical data structures.
- Do not think array and linked list are the same just because both store data.

- Do not forget that arrays are fixed in size.
- Do not forget that linked list nodes are created in heap.
- Do not assume logical data structures exist independently from physical implementation.

Common Mistakes

- Thinking stack and queue are physical storage formats
- Forgetting that array uses contiguous memory
- Forgetting that linked list is dynamic
- Ignoring the connection between memory organization and data structure choice

---

19. Code Examples Related to This Topic

No actual code was included or discussed in this transcript.

So there are no code examples to record for this lesson.

---

20. Final Recap

This lesson gives a basic introduction to data structures.

The instructor divides data structures into two types:

Physical data structures

- Array
- Linked List

These define how memory is organized and how data is physically stored.

Logical data structures

- Stack
- Queue
- Tree
- Graph
- Hash Table

These define how data is used and what operational discipline is followed.

The most important conclusion

Logical data structures are implemented using physical data structures, such as:

- arrays
- linked lists
- combinations of both

The lesson is only an introductory overview. The course will later cover each structure in detail, starting with arrays and linked lists, and then moving to the logical data structures.

---

21. Unclear or Incomplete Transcript Parts

- The transcript is mostly clear.
- No major unclear section affects the meaning of the lesson.
- The term **ADT** is mentioned as the next topic, but it is not explained in this transcript.
- The instructor says hash table “may be linear or tabular,” but then treats it as tabular. The exact classification detail is not fully expanded here.

Abstract Data Type (ADT) and List ADT Study Notes

Lesson overview

This lesson explains what an **Abstract Data Type (ADT)** is. The instructor first breaks the term into two parts: **data type** and **abstract**. Then the instructor uses a **list** as the main example to show how an ADT is defined and used. The main idea is that an ADT combines:

- how data is represented
- what operations are allowed on that data

The internal working is hidden from the user. That hidden part is what makes it "abstract."

Main topics

1. Meaning of a data type
2. Meaning of "abstract"
3. What an Abstract Data Type (ADT) is
4. Why ADT is important in object-oriented programming
5. List as an example of an ADT
6. Representation of a list in a program
7. Operations on a list
8. Key takeaways for data structures

---

1. Meaning of a data type

Definition of a data type

A **data type** is defined using two things:

- **Representation of data**
- **Operations on data**

This means a data type is not just the value itself. It also includes:

- how the value is stored
- what you are allowed to do with it

Explanation

When we study a data type in programming, we usually learn:

- how memory is used for it
- what operators or actions work on it

The instructor says that every data type in a language has:

- a way to represent data
- a set of operations allowed on that data

Example: integer data type

The instructor uses the example of an **integer** in C/C++.

Representation of integer

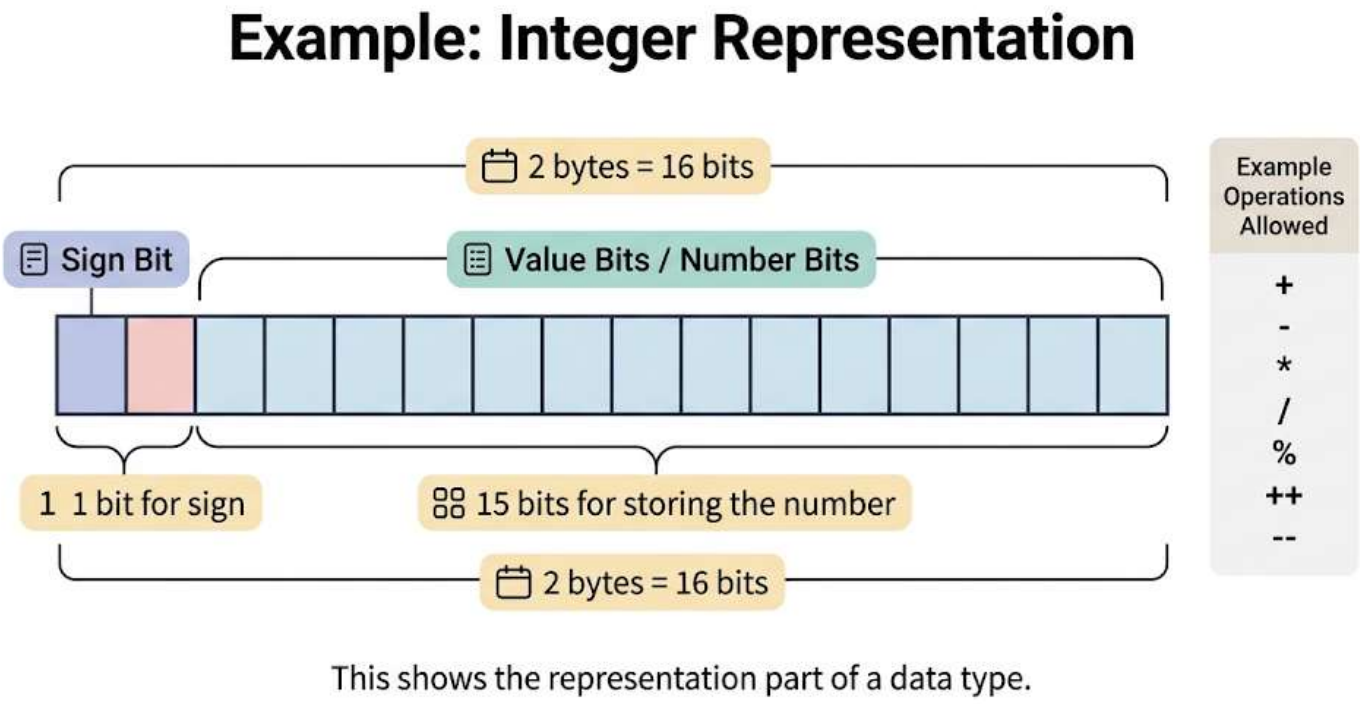
The example assumes an integer takes **2 bytes**, which is:

- 2 bytes = 16 bits

Out of these 16 bits:

- **1 bit** is used as the **sign bit**
- **15 bits** are used to store the number

This shows the **representation** part of a data type.



This shows the representation part of a data type.

Operations on integer

The instructor says integer data supports operations such as:

- arithmetic operations
  - +
  - -
  - *
  - /
  - %
- increment and decrement
  - ++
  - --
- relational operations

- the transcript mentions these, but some operator names are missing due to transcript noise [unclear]

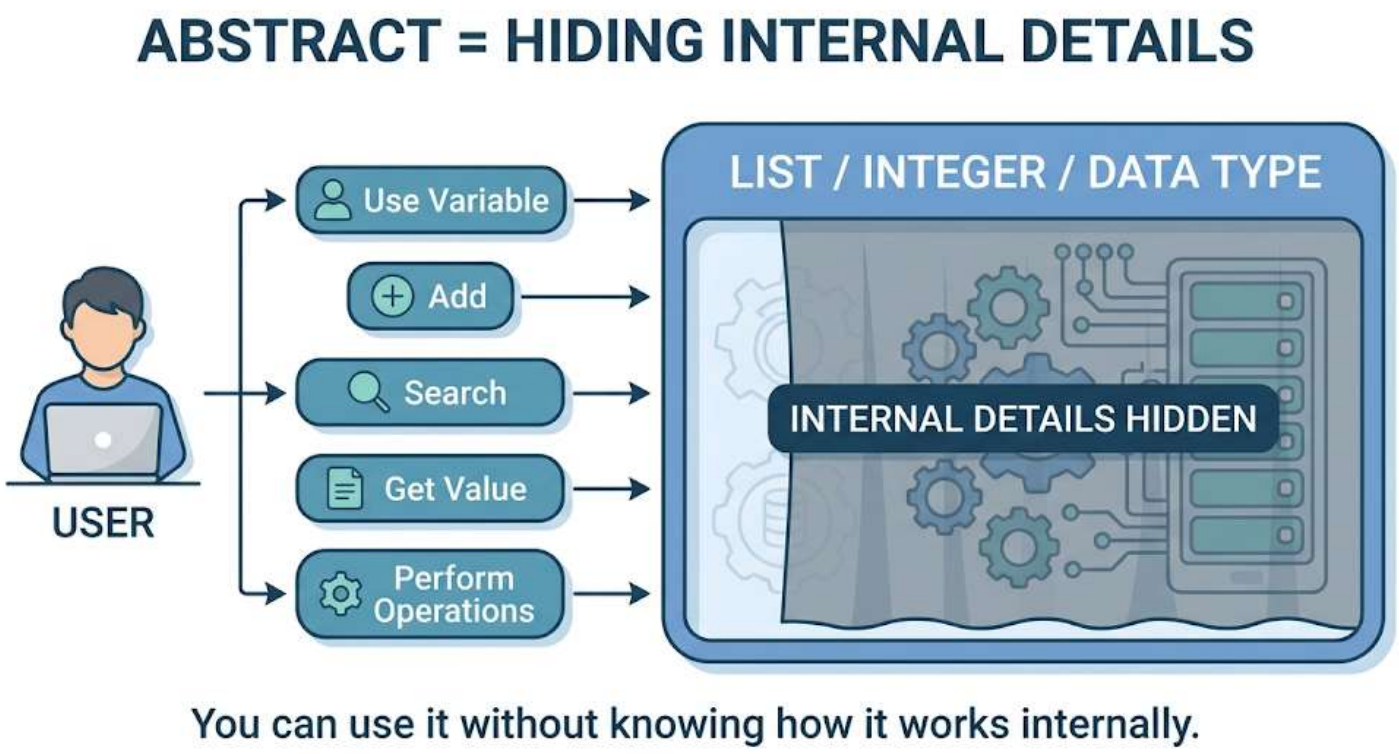
Important idea

A data type is complete only when you know both:

- how it is stored
- what operations can be performed on it

---

2. Meaning of "abstract"



Definition of abstract

Abstract means:

- **hiding internal details**

This means the user can use something without knowing exactly how it works internally.

Explanation using integer example

When you use an integer variable, you usually do not think about:

- how its bits are arranged in memory
- how arithmetic happens internally in binary
- how increment or decrement is implemented inside the machine

You simply:

- declare the variable
- use the operations

So the internal details are hidden. That hidden nature is what the instructor calls **abstract**.

Key point

You do **not** need to know the internal implementation in order to use the data type correctly.

That is the basic meaning of abstraction in this lesson.

---

3. What an Abstract Data Type (ADT) is

Definition of ADT

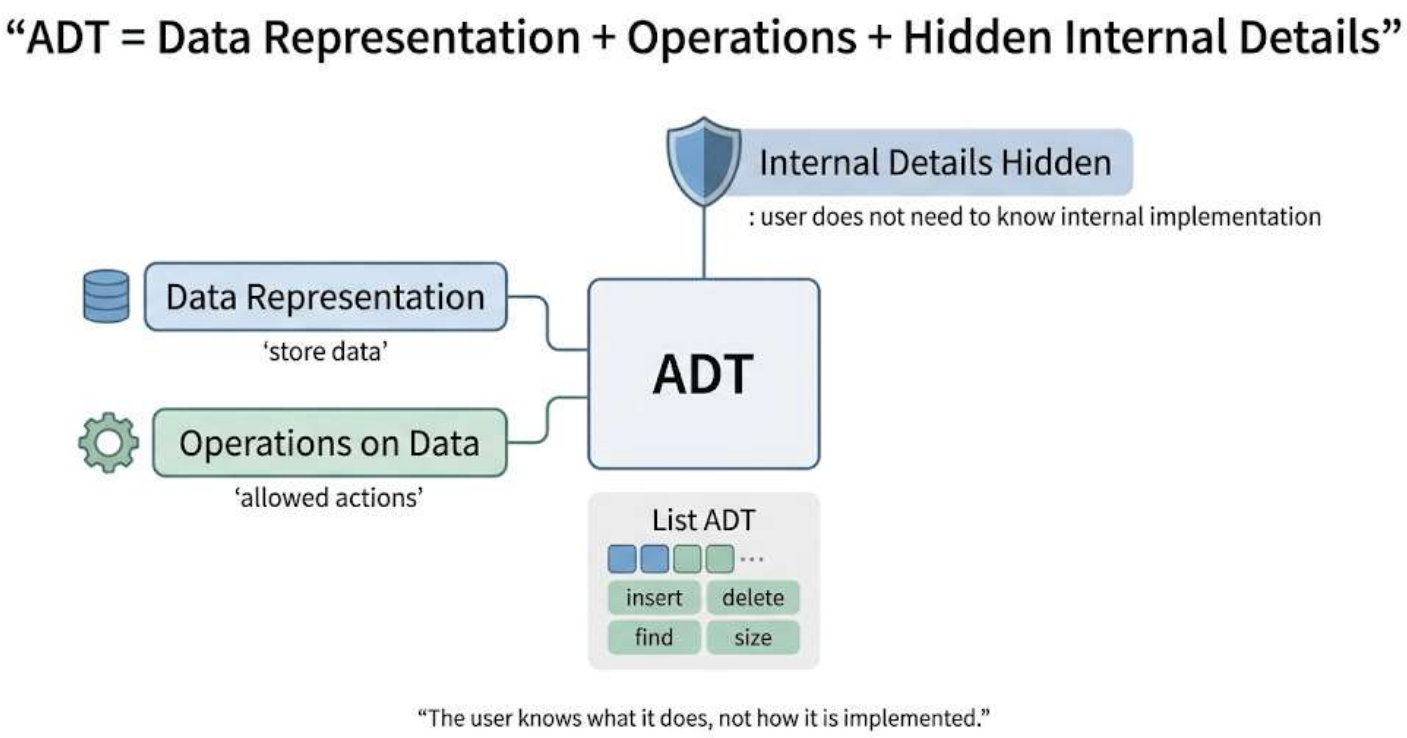
ADT stands for **Abstract Data Type**.

An ADT combines:

- **data representation**
- **operations on that data**

At the same time:

- the **internal implementation** is hidden from the user



Simple meaning

An ADT lets us treat a structure as a usable data type without worrying about how it is built inside.

You only need to know:

- what data it stores
- what operations you can perform

You do not need to know:

- how the data is physically organized inside the program
- how each operation is implemented

Very important comparison

Primitive data type

Example: int

- already provided by the language

- has its own storage format
- has built-in operations

Abstract data type

Example: list ADT

- designed by the programmer
- defines what data is stored
- defines which operations are available
- hides internal implementation details

The instructor clearly says the integer example is a **primitive data type**, not itself the main ADT example being taught.

---

4. Why ADT is important in object-oriented programming

Link with OOP

The instructor says ADT is related to **Object-Oriented Programming (OOP)**.

With OOP, especially using **classes**, we can define our own custom data types. These user-defined types can behave like abstract data types.

Why classes help

A class can contain:

- the data representation
- the functions or operations that work on that data

So when a class is written well, it can define an ADT.

Main benefit

A programmer can:

- create an object
- use the object's operations

without worrying about:

- whether the data is stored using an array
- whether it is stored using a linked list
- how each operation works internally

This is the practical use of abstraction.

Instructor's point

The instructor says that in C++, when a class contains both:

- data representation
- operations on the data

it defines an ADT.

---

5. List as an example of an ADT

What is a list?

A **list** is a collection of elements.

The instructor shows a list of numbers and assigns indices to the elements. The indices can start from:

- 0, or
- 1

depending on the definition or requirement.

In this lesson, the instructor chooses indexing from **0**.

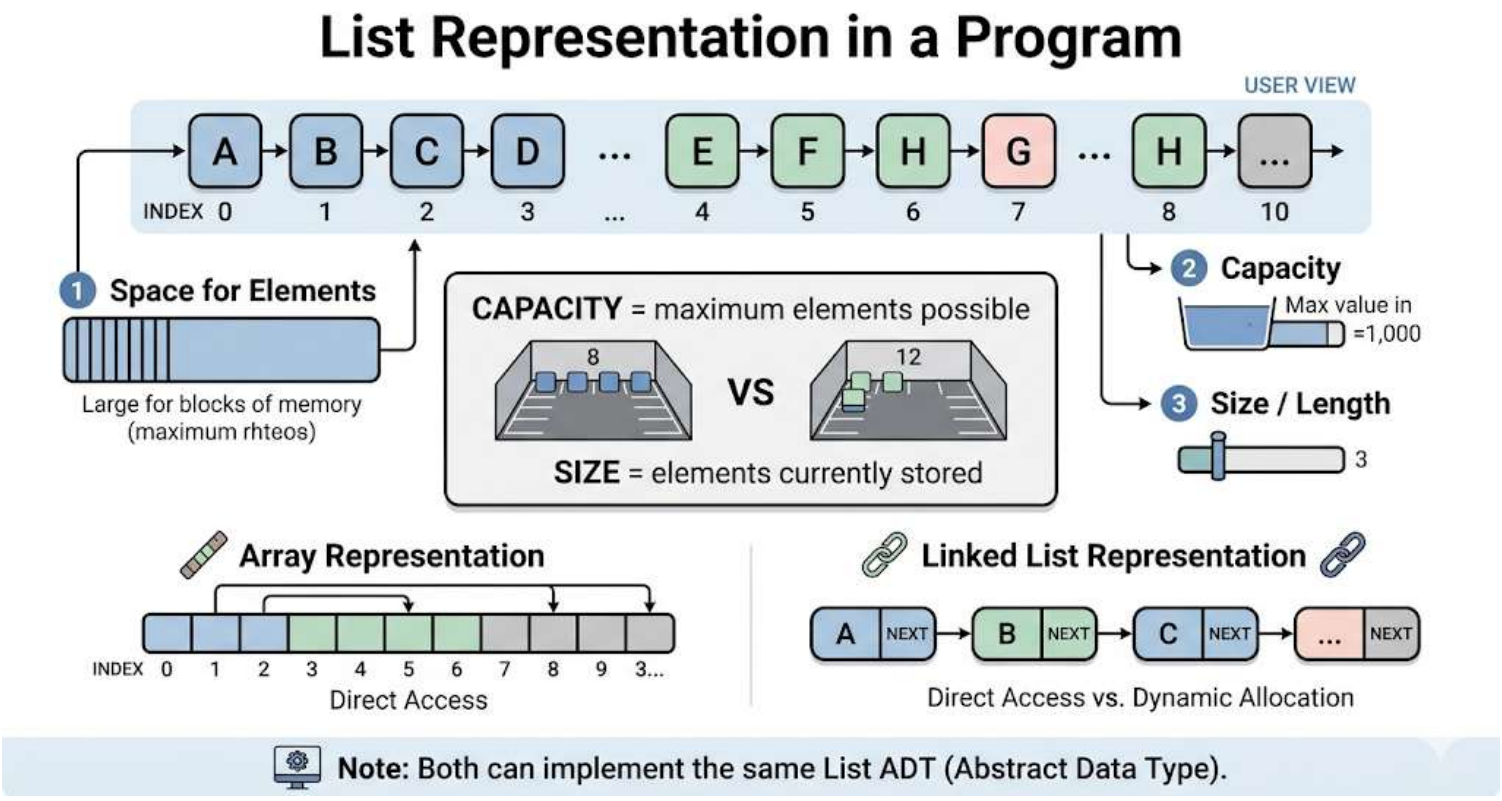
Key idea

The list shown on paper is only a concept.
To use it in a program, we need a proper representation.

That is where the ADT idea becomes useful.

---

6. Representation of a list in a program



What must be stored for a list?

According to the instructor, to represent a list in a program, we need three things:

- **space to store the elements**
- **capacity**
- **size** or **length**

1) Space for elements

This is the actual memory area where list elements are stored.

2) Capacity

Capacity means:

- the maximum number of elements the list can hold

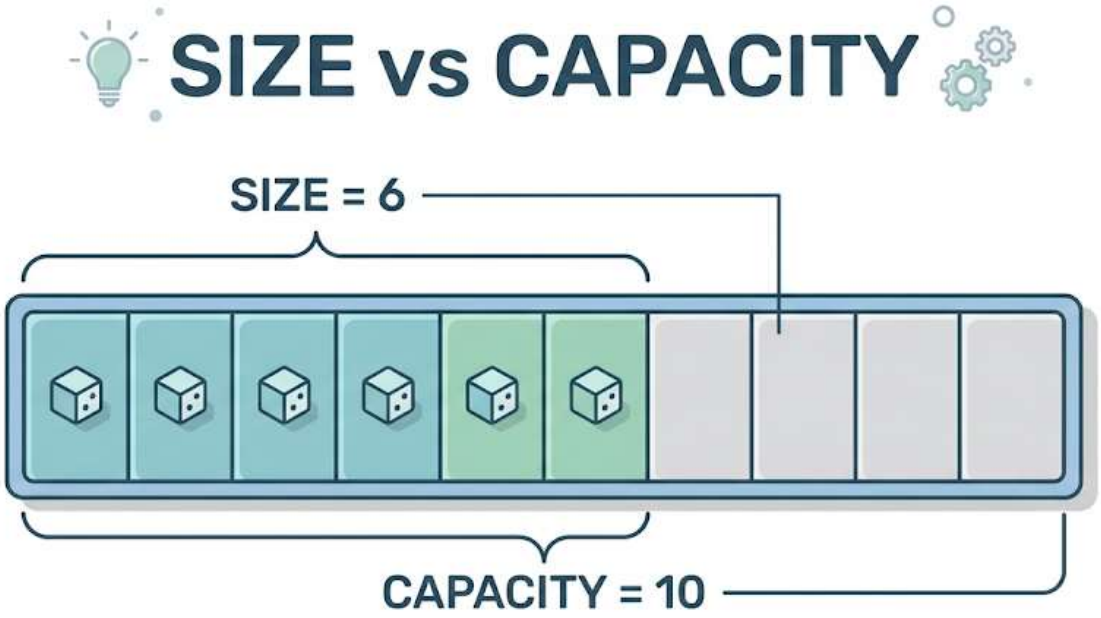
3) Size / length

Size means:

- how many elements are currently in the list

This is different from capacity.

Important comparison: size vs capacity



Capacity is total available space. **Size** is how many elements are currently stored.

Capacity

- total possible space available

Size

- how much of that space is currently used

This is a very important distinction in list implementation.

Possible internal representations

The instructor says a list can be represented in two main ways:

- **array**
- **linked list**

Important ADT concept here

Even if the internal representation changes, the ADT idea stays the same.

That means:

- one implementation may use an array
- another may use a linked list

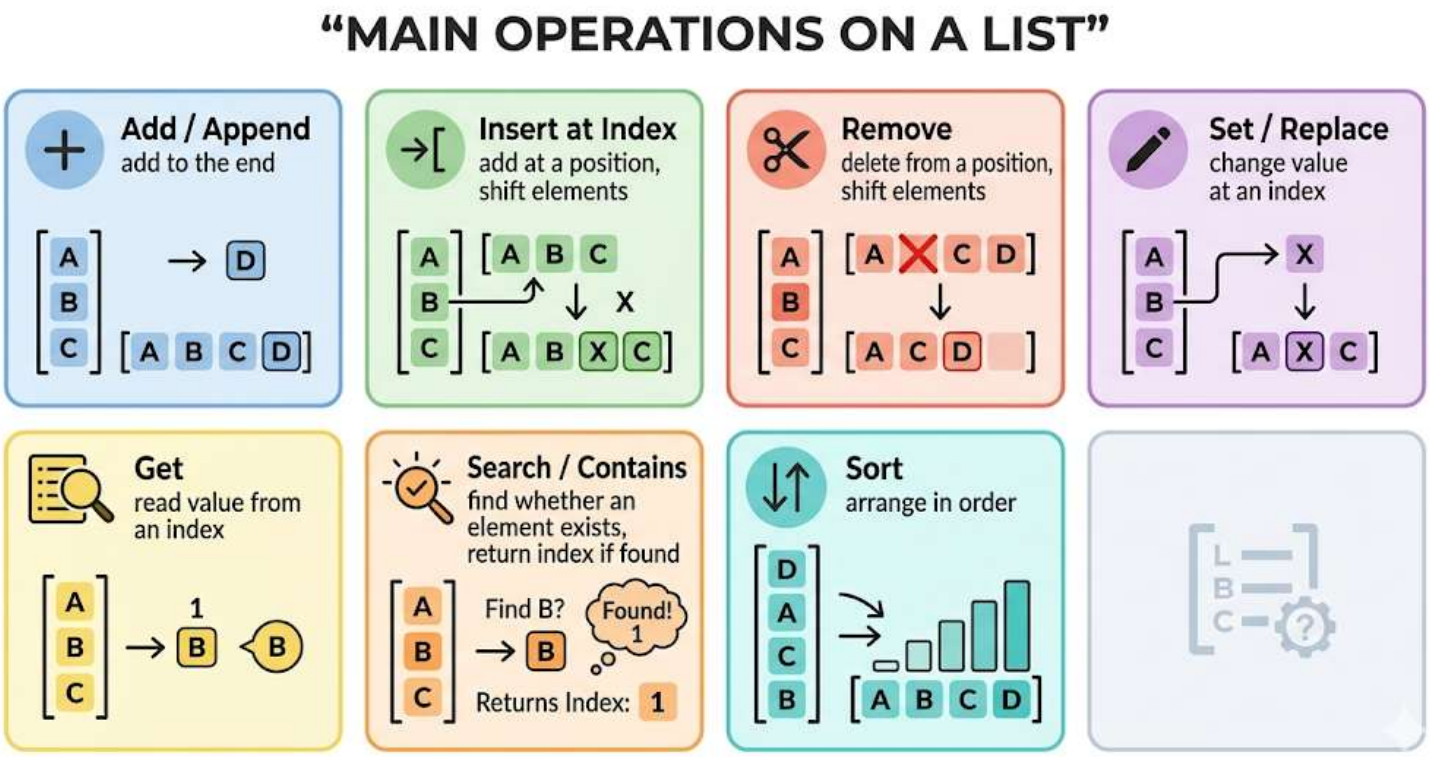
But the user can still use the same list operations.

This shows why ADT is powerful:
the user depends on the **interface and behavior**, not the internal storage.

---

7. Operations on a list

The instructor explains several common list operations. These operations are part of what makes the list a data type.



7.1 Add / Append

Meaning

Add means adding an element to the **end of the list**.

The instructor also says this can be called:

- **append**

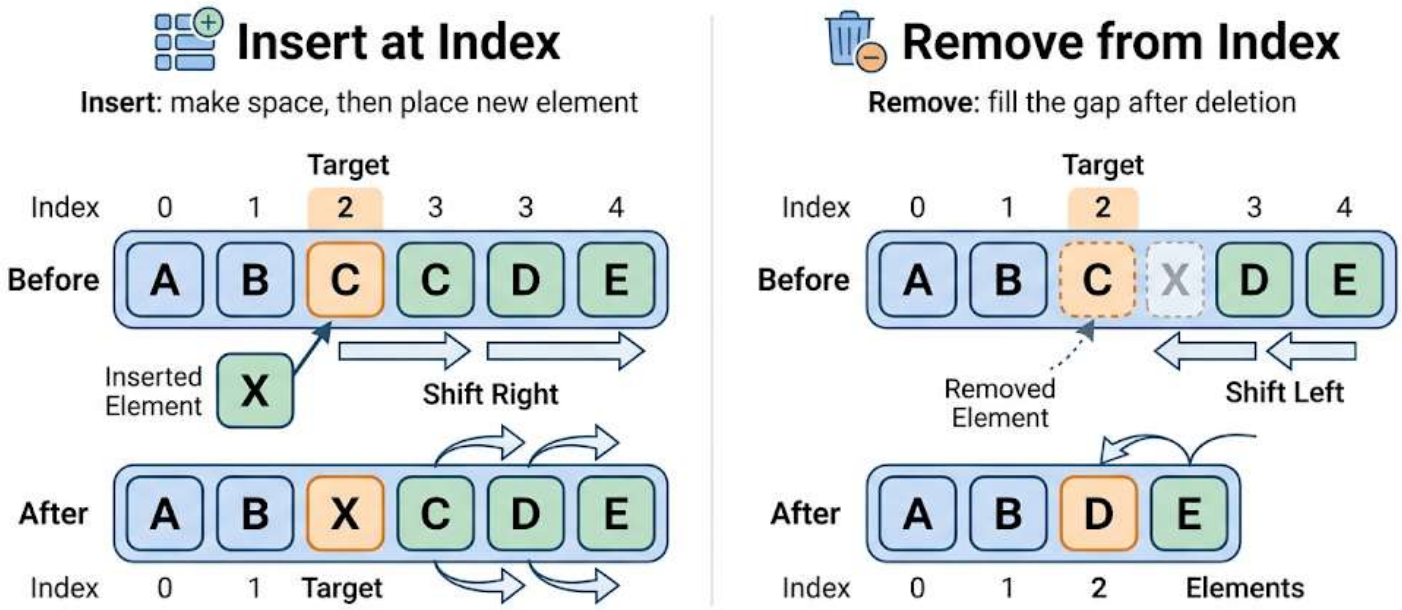
Example idea

If the list currently ends at index 6, and we add 15, then 15 is placed at the next available index.

Important point

Add/append does **not** place the element in the middle.
It places the element at the end.

7.2 Insert at a given index



Meaning

This operation adds an element at a specific position in the list.

This is different from append.

Example from the lesson

The instructor explains inserting 20 at index 6.

If some element is already at index 6, then you cannot simply overwrite it. You must first make space.

Step-by-step process for insert

1. Choose the target index
2. Shift the existing element at that index to the right
3. Shift other following elements to the right as needed
4. Insert the new value into the free space

Why shifting is needed

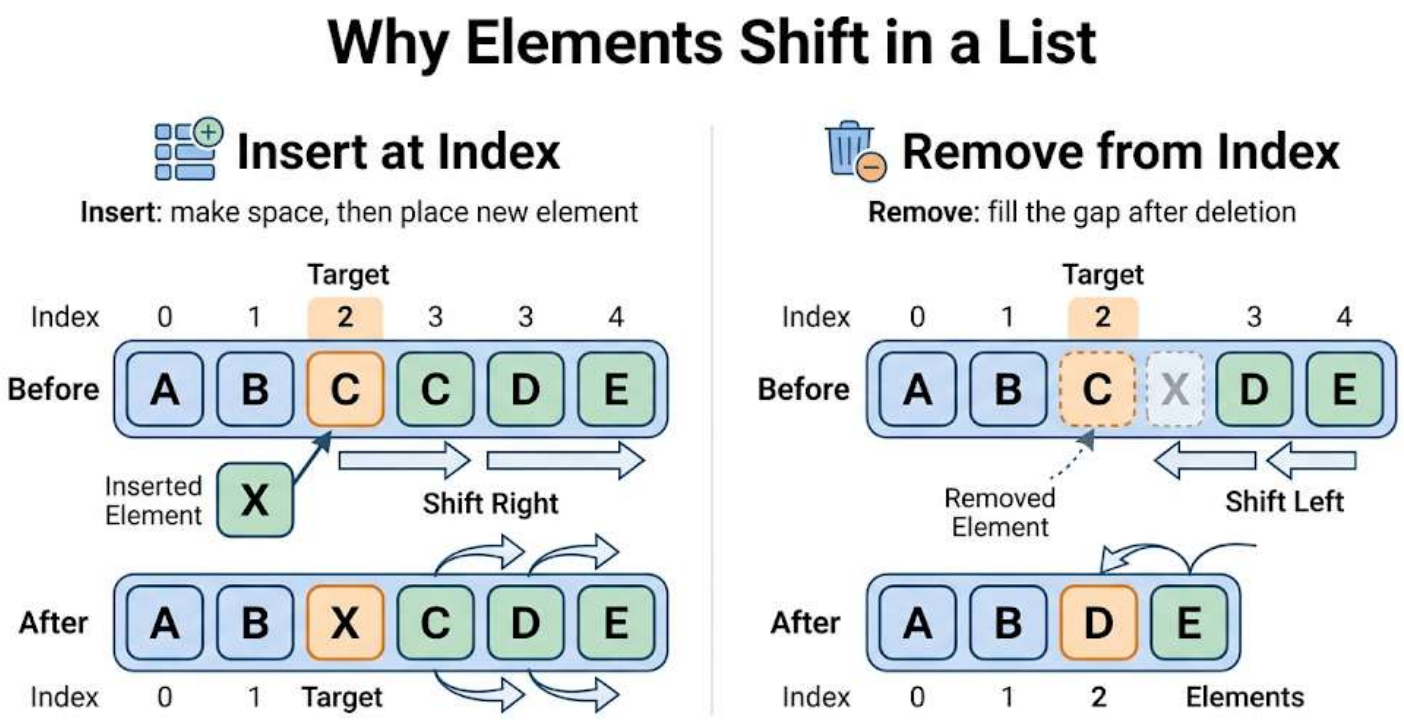
Lists keep order.
If you insert in the middle, the later elements must move.

Another name

The instructor says this operation is also called:

- **insert**

7.3 Remove from a given index



Meaning

To remove an element, you specify the index of the element to remove.

Example from the lesson

The instructor removes the same value that was inserted earlier.

When an element is removed:

- its position becomes empty

- the later elements must shift left to fill the gap

Step-by-step process for remove

1. Select the index to remove
2. Delete that element
3. Shift all later elements one position to the left
4. Reduce the size of the list by one

Important point

After removal:

- the list still stays continuous
- there should not be a gap in the middle

7.4 Set / Replace at a given index

Meaning

This operation changes the value at a particular index.

Example from the lesson

The instructor says to change the element at index 3 to 25.

Difference from insert

This is very important:

- **insert** adds a new element and shifts others
- **set** changes an existing element in place

Another name

The instructor says **set** can also be called:

- **replace**

7.5 Get element at a given index

Meaning

This operation returns the element stored at a specific index.

Example from the lesson

The instructor checks what value is at index 5 and says it is 10.

Use of get

Use this operation when you want to:

- read a value
- inspect a position
- retrieve an element without changing the list

7.6 Search for a key

Meaning

Search means finding whether a given element exists in the list.

If it exists, the result is usually:

- the index where it is found

Example from the lesson

The instructor searches for 9 and says it is found at index 2.

Step-by-step process for search

1. Take the key value to search
2. compare it with list elements
3. if found, return its index

4. if not found, report that it is absent

Another name

The instructor says search can also be called:

- **contains key**

This means checking whether the list contains the given element or not.

7.7 Sort

Meaning

Sorting means arranging the elements in some order.

Usually this means:

- ascending order, or
- descending order

The instructor does not specify the exact sorting order here, only that the list can be made into a sorted list.

7.8 Other possible list operations

The instructor also mentions that many more operations are possible, such as:

- reverse the list
- combine lists
- merge lists
- split a list

This shows that a list ADT can support many operations beyond the basic ones first introduced.

---

8. Why list is an ADT

A list becomes an ADT because it has both:

- **representation**
- **operations**

Representation

A list needs:

- storage for elements
- capacity
- size

and can be implemented using:

- array
- linked list

Operations

A list supports operations such as:

- add / append
- insert
- remove
- set / replace
- get
- search / contains
- sort
- reverse

- merge
- split

Why it is abstract

The user does not need to know:

- how the list is stored internally
- how shifting happens
- whether the implementation uses an array or linked list

The user only needs to know how to use the list and its operations.
That is the essence of the list ADT.

---

9. Detailed beginner-friendly explanation

Think of ADT like a machine with buttons

An ADT is like a machine where:

- you know what buttons exist
- you know what each button does
- you do not need to know the internal mechanism

For a list ADT, the “buttons” are the operations:

- add
- insert
- remove
- search
- sort
- and others

The inside mechanism could be:

- array-based
- linked-list-based

But the user does not need to care.

Why this matters in data structures

In data structures, we often want to think at a high level first.

For example, before worrying about code details, we should ask:

- What data does this structure hold?
- What operations should it support?

This is exactly the ADT viewpoint.

The instructor says many structures will be studied this way, including:

- array
- linked list
- stack
- queue
- graph
- tree
- hash table

---

10. Code examples related to each topic

Code shown in this transcript

No actual code was shown in this transcript.

What the instructor says about code

The instructor says code will be shown later in:

- **C**
- **C++**

The instructor also says these data structures will be implemented as ADTs, especially through C++.

---

11. Important instructor tips, warnings, and common mistakes

Tip 1: Always separate representation and operations

To understand any data type or ADT, ask:

- how is the data stored?
- what operations are allowed?

This is the core framework of the lesson.

Tip 2: Do not confuse primitive data type with ADT

The integer example is used only to explain the basic ideas.
The instructor clearly distinguishes primitive data types from user-defined ADTs.

Tip 3: Abstract does not mean "nothing is defined"

Abstract means:

- internal details are hidden

It does **not** mean the structure is vague or incomplete.

An ADT is still clearly defined by its data and operations.

Tip 4: Size and capacity are different

This is a common beginner mistake.

- **capacity** = maximum number of elements possible

- **size** = number of elements currently present

Tip 5: Insert and set are not the same

Another common mistake:

- **insert** adds a new element and shifts others
- **set/replace** changes an existing element

Tip 6: Remove usually needs shifting

When removing from the middle of a list, later elements must move left to fill the empty space.

Tip 7: Search usually returns position

The instructor explains search as finding the index of the target element if it exists.

---

12. Final recap

- **ADT** stands for **Abstract Data Type**.
- A data type is defined by:
    - representation of data
    - operations on data
- **Abstract** means internal details are hidden.
- Primitive types like int help explain the idea, but ADTs are often user-defined.
- In OOP, especially with **classes**, programmers can define their own ADTs.
- A **list** is used as the main example of an ADT.
- To represent a list in a program, we need:
    - storage for elements

    - capacity
    - size
- A list can be implemented using:
    - an array
    - a linked list
- Common list operations include:
    - add / append
    - insert
    - remove
    - set / replace
    - get
    - search / contains
    - sort
    - reverse
    - merge
    - split
- The user of the ADT should focus on what the list does, not how it is implemented internally.

---

13. Unclear or incomplete transcript parts

The transcript is mostly understandable, but a few parts are unclear or incomplete:

- The exact relational operators mentioned for integer operations are not fully visible in the transcript. This part should be marked as **[unclear]**.

- The exact full list of sample numbers shown by the instructor is not completely reliable from the transcript alone, so it should not be reconstructed beyond the values clearly mentioned in the explanation.
- No code appears in the transcript, though the instructor says code will be shown in later topics.

Time and Space Complexity.

Based on the provided transcript.

1. Lesson Overview

This lesson introduces **time complexity** and **space complexity** in a simple way.

The instructor explains:

- what time complexity means
- how to estimate time based on the work being done
- how to estimate time from code
- common patterns such as:
  - single loop
  - nested loops
  - shrinking work
  - dividing by 2
- how complexity applies to:
  - arrays
  - linked lists
  - matrices
  - trees
- what space complexity means in memory usage

The main idea is this:

- **Time complexity depends on the procedure you use**
- **Space complexity depends on how much memory is used**

---

2. Main Topics

1. Meaning of time complexity
2. Meaning of n
3. Time complexity patterns in arrays
4. Reading complexity from code
5. Complexity in linked lists
6. Complexity in matrices
7. Complexity in combined structures
8. Complexity in binary trees
9. Space complexity basics
10. Important warnings and common mistakes

---

3. Time Complexity

3.1 What Time Complexity Means

Time complexity tells us:

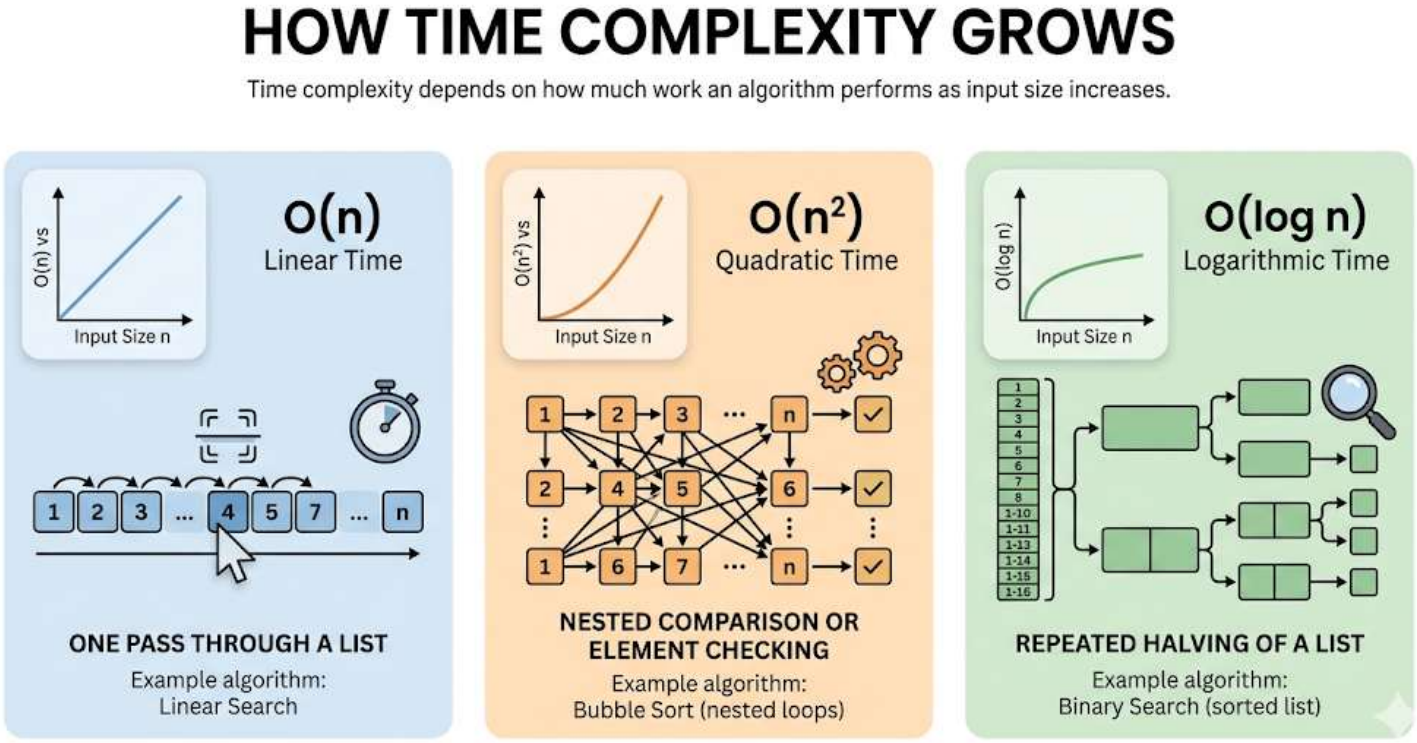
- how much time a computer takes to complete a task
- based on the method or process used

In daily life, we measure how long a task takes.
In programming, we also want to know how long a machine takes to do a task.

The instructor compares this to real life:

- a person may make bread in 15 to 20 minutes

- if a machine takes 4 to 5 hours, then that machine is not useful for that task



So in programming, we ask:

- how much time will the machine take?
- does the procedure make the task fast or slow?

Key idea

Time complexity does **not** mainly depend on the name of the problem.
It depends on **how the problem is solved**.

3.2 Meaning of n

The instructor says students often get confused here.

What n means

n means:

- the number of elements in the input
- some amount of data
- not a fixed number like 5 or 10

It could be:

- 10
- 10,000
- 10,000,000
- or any large number

So when we say an array has n elements, we mean:

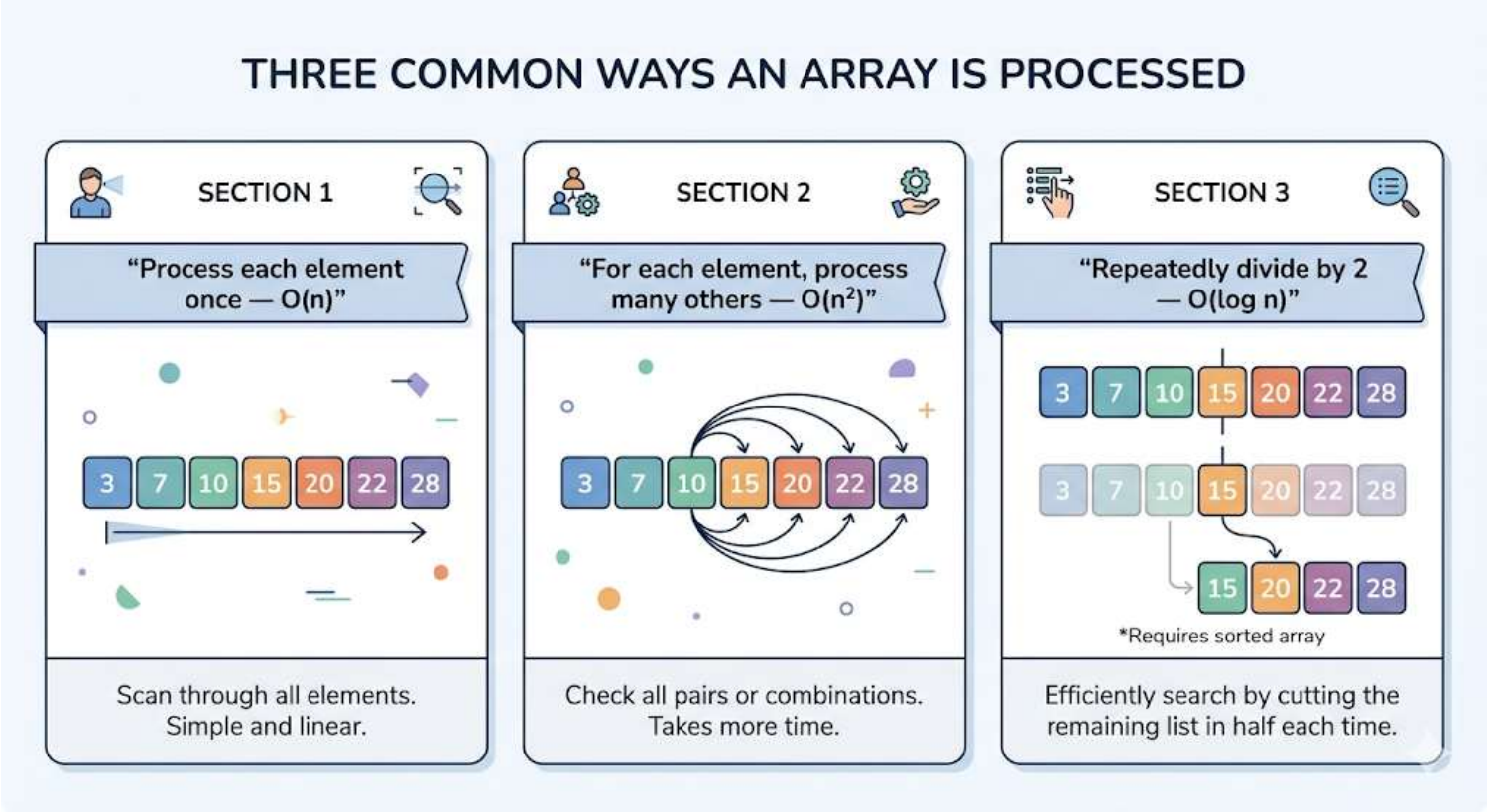
- we do not want to fix the size
- we want a general rule that works for any size

Important note

Do **not** think n always means a small number.
It means **input size**.

---

4. Time Complexity in Arrays



4.1 Case 1: Process every element once

Example tasks:

- add all elements
- search through all elements
- count elements
- find the maximum

If you go through all n elements one time, then the time is:

Order(n)

The instructor uses **Order(n)** instead of Big-O for now.

Why?

Because the work grows directly with the number of elements.

- 10 elements ⟶ about 10 steps
- n elements ⟶ about n steps

Code pattern

```
for(i = 0; i < n; i++)
{
    // processing
}
```

What this code does

- starts from the first element
- visits each element one by one
- repeats the work n times

Why this code is used

This loop is used when you want to handle each element once.

Concept it relates to

This shows **linear time complexity**.

4.2 Example: Searching in an array

If you want to check whether a value exists in an unsorted array, you may need to compare with every element.

Example from the lesson:

- search for 12 → found
- search for 21 → maybe not found, so all elements must be checked

Worst case idea

In the worst case, the algorithm checks all n elements.

So the time is:

Order(n)

Important point

Even if the value is found early sometimes, the full process may still require up to n checks in the worst case.

4.3 Case 2: For each element, process all elements

Now the instructor explains a second pattern.

Suppose:

- for the first element, you compare it with all elements
- for the second element, you again compare it with all elements
- and so on for every element

Then:

- each pass does n work
- and there are n such passes

So total time is:

$n \times n = n^2$

Therefore:

Order(n^2)

Typical use

This kind of pattern often appears in:

- sorting
- pairwise comparisons
- checking all combinations of elements against each other

Code pattern

```
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        // processing
    }
}
```

What this code does

- outer loop runs n times
- inner loop also runs n times for each outer loop step
- total repetitions are about n^2

Why the instructor used this code

To show that **nested loops often lead to quadratic time**

Concept it relates to

This is **quadratic time complexity**

4.4 Case 3: Triangular processing pattern

In this pattern:

- for the first element, process the remaining n - 1 elements
- for the second element, process n - 2
- for the third element, process n - 3
- and so on

This gives a decreasing series:

```
(n − 1) + (n − 2) + (n − 3) + ... + 1
```

The instructor writes this as:

- $(n \times (n - 1)) / 2$
- which becomes $(n^2 - n) / 2$

Final result

The polynomial contains both n^2 and n, but the highest-degree term is n^2.

So the time complexity is:

Order(n^2)

Main lesson here

Even when the work is not exactly n × n, it can still reduce to **Order(n^2)** if the highest-degree term is n^2.

Code pattern

```
for(i = 0; i < n; i++)
{
    for(j = i + 1; j < n; j++)
    {
        // processing
    }
}
```

What this code does

- outer loop picks one element
- inner loop starts after that element
- only the remaining elements are processed

Why this version matters

This is more efficient than comparing every pair twice, but it is still quadratic overall.

Concept it relates to

This also shows **quadratic time**, even though the exact number of operations is smaller than full n^2.

4.5 Case 4: Repeatedly divide the problem by 2

Now the instructor shows a very important pattern.

Suppose you:

- process the middle element
- then go to the middle of one half
- then the middle of the next half
- and continue dividing the list by 2

In this case, you are **not** processing the whole list every time.

You are doing:

- half
- then half again
- then half again

This creates:

log n

More precisely, the instructor mentions this as **log base 2 of n**.

Why?

Because the size keeps getting divided by 2 until it becomes 1.

Code pattern

```
for(i = n; i > 1; i = i / 2)
{
    // processing
}
```

What this code does

- starts from n
- each time divides i by 2
- stops when i becomes 1

Why this matters

A loop is not always Order(n).
You must check **how the loop variable changes**.

If it grows or shrinks by division, the complexity may be logarithmic.

Equivalent while-loop version

```
i = n;
while(i > 1)
{
    // processing
    i = i / 2;
}
```

What this code does

It performs the same logic as the for loop:

- keep processing
- reduce the problem size by half each time

Why the instructor showed both versions

To show that complexity depends on behavior, not on whether the loop is for or while.

Concept it relates to

This is **logarithmic time complexity**

---

5. How to Analyze Complexity

5.1 Two ways to analyze

The instructor says there are two main ways:

1. Analyze from the procedure

Ask:

- what work is being done?
- how many items are being processed?
- how many times is that work repeated?

This is the better method.

2. Analyze from the code

Look at:

- loops
- nested loops
- how counters change
- whether the input size shrinks

This is useful, but students often get confused.

Important advice

Do not look at code blindly.

Instead:

- understand what the code is doing
- then connect it to the amount of work

That makes complexity easier to understand.

5.2 Order and polynomial degree

The instructor says time complexity is written in terms of the **degree of a polynomial**.

Example:

- $(n^2 - n) / 2$

The highest-degree term is n^2, so the complexity is:

Order(n^2)

Important meaning

When writing complexity, we focus on the dominant growth term.

That is why:

- $n^2 + n$ becomes Order(n^2)
- $3n + 5$ becomes Order(n)

The lesson does not formally teach Big-O, Omega, or Theta yet.
The instructor says those will be explained later.

---

6. Linked List Complexity

6.1 Linked list basics in this lesson

The instructor says a linked list is also a kind of list, like an array.

So many of the same time complexity ideas still apply.

Main takeaway

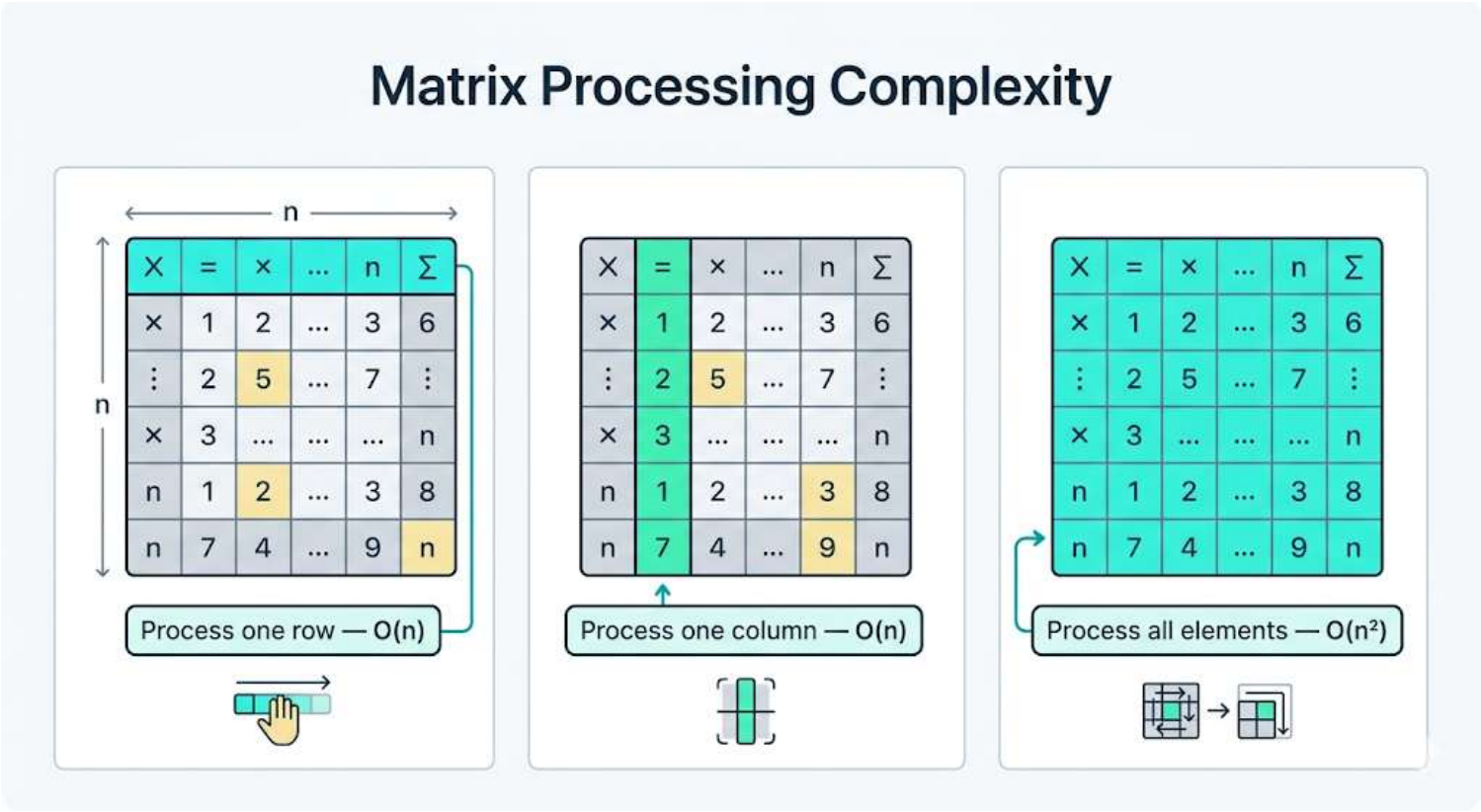
For general processing:

- if you visit all elements once ⟶ **Order(n)**
- if you compare or process in nested fashion ⟶ can become **Order(n^2)**

The lesson does not go deep into linked lists yet.
It only says they behave like lists for many complexity discussions.

---

7. Matrix Complexity



7.1 Number of elements in a matrix

If a matrix is of size n × n, then total elements are:

n^2

So if you process **all elements**, the time is:

Order(n^2)

If you process only:

- one row ⟶ **Order(n)**
- one column ⟶ **Order(n)**

Main idea

The complexity depends on **how much of the matrix you touch**.

7.2 Matrix code pattern

```
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        // processing
    }
}
```

What this code does

- outer loop visits rows
- inner loop visits columns
- together they visit every element in the matrix

Why this code is used

It is the standard pattern for processing all cells of an n × n matrix.

Concept it relates to

This is another example of **Order(n^2)**

7.3 Extra processing inside matrix loops

The instructor adds an important point.

If, for each matrix element, you do more work, then total time can increase.

Example:

- if each element only needs one simple statement → still Order(n^2)
- if each element triggers another loop or a function with a loop → time becomes larger

Example idea

If there are three nested loops:

- loop over rows
- loop over columns
- loop again for extra processing

then the time can become:

Order(n^3)

Important lesson

Do not count only the outer structure.
Also check what happens **inside** the loops.

---

8. Combined Structure: Array of Linked Lists

8.1 General idea

The instructor shows a structure that looks like:

- an array
- where each array entry points to a linked list

This is like an **array of linked lists**.

He says:

- there is an array of size n
- and then there are linked-list elements connected to it

Time idea given by the instructor

He says the total work may be:

m + n

This means:

- some work for the outer array
- some work for the linked-list elements

So depending on what you count, total processing may be:

- m + n
- or just n if you only count certain parts

Important explanation from the instructor

Sometimes complexity analysis depends on what exactly you choose to count.

He gives a real-life example of a party:

- hotel bill
- travel cost
- tips
- other costs

You may include all of them, or only the main cost, depending on the situation.

Meaning of this example

In complexity analysis:

- sometimes you count all parts
- sometimes you focus only on the main part
- you must understand what is relevant

Warning

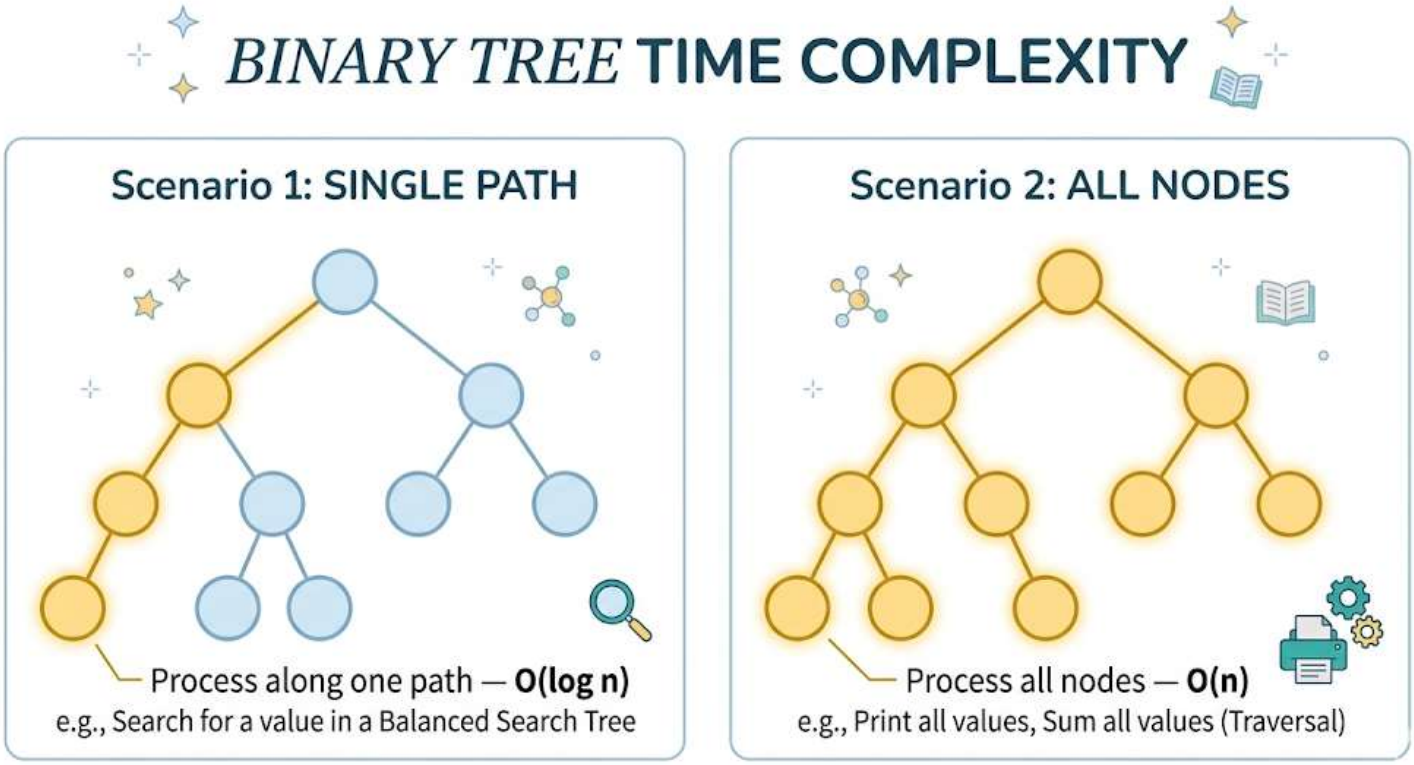
This part of the transcript is a little unclear in notation because the exact meaning of m and n is not fully defined in detail.

So the safe interpretation is:

- total cost depends on both the outer array and the linked elements
- the instructor summarizes that as **m + n**

---

9. Binary Tree Complexity



9.1 Processing only along a path

The instructor then explains a binary tree.

Suppose you only move along one path, such as:

- root
- one child
- one grandchild
- and so on

Then the amount of work depends on the **height** of the tree.

He explains this using the idea that the number of elements halves as you move through tree levels.

That gives:

log n

Meaning

If you only move down one branch in a balanced binary tree, the time is:

Order(log n)

Typical idea this matches

This is common in efficient tree searching where each step reduces the remaining possibilities.

9.2 Processing all nodes in the tree

If instead you process every node in the tree, then the total work is:

Order(n)

because there are n nodes.

Main contrast

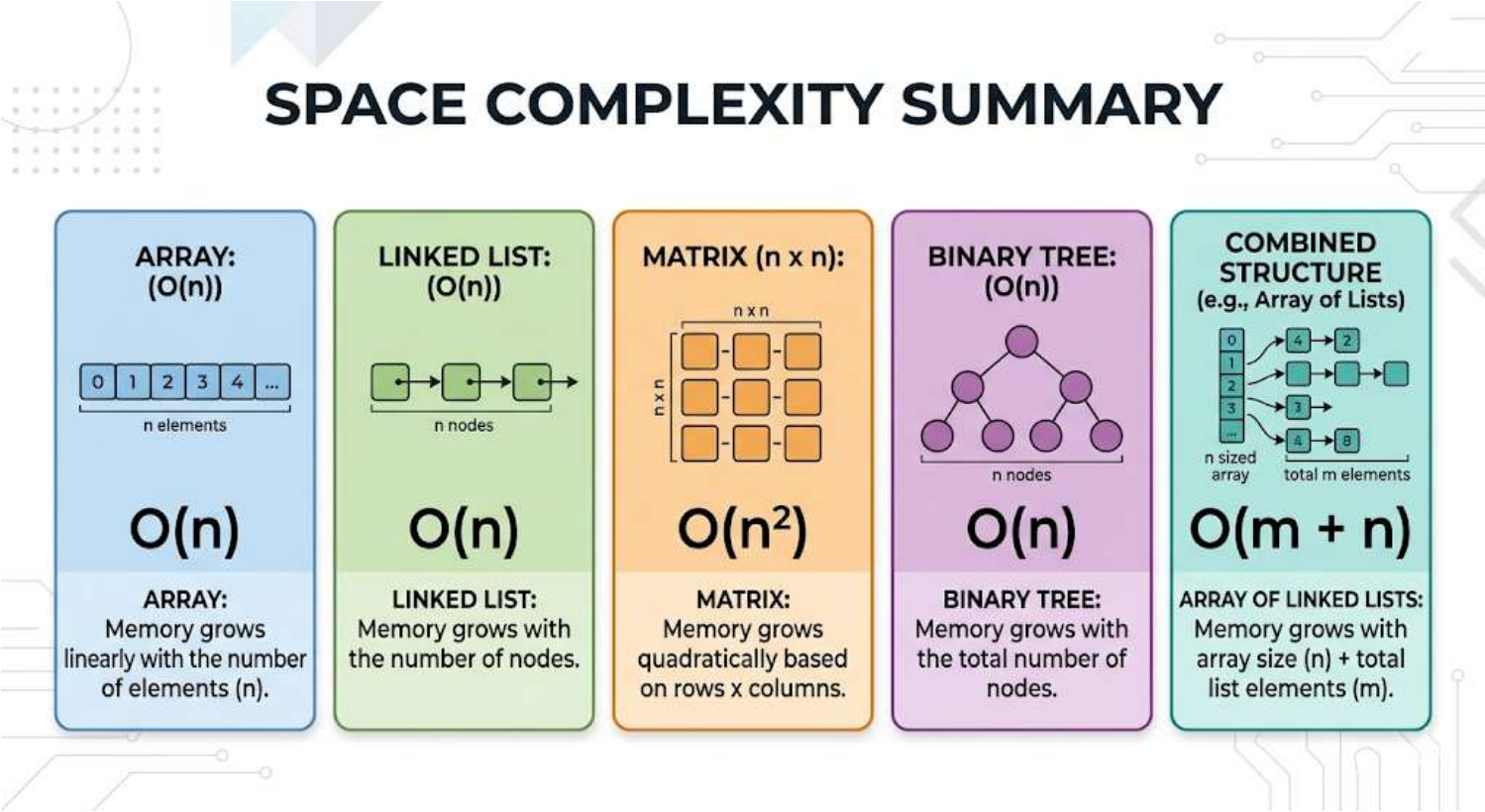
- one path only → Order(log n)

- all nodes ⟶ Order(n)

This is a very important comparison.

---

10. Space Complexity



10.1 What Space Complexity Means

Space complexity tells us:

- how much memory is used during execution

The instructor says we want to know:

- how much space is consumed in main memory while the program runs

Important note

We usually do **not** focus on exact bytes here.

We focus on how memory grows with input size.

So we ask:

- if input size increases, how does memory increase?

10.2 Space complexity of array

An array with n elements uses:

Order(n) space

This could be:

- n integers
- n floats
- n doubles

The exact type does not matter for this discussion.
The key idea is that space grows with n.

10.3 Space complexity of linked list

A linked list also stores n elements.

But it also needs extra space for links or pointers.

The instructor says this can be thought of as:

- about 2n

But the degree still remains:

Order(n)

Meaning

Even with links included, the growth is still linear.

10.4 Space complexity of matrix

An n × n matrix uses:

Order(n^2) space

because it stores n^2 elements.

10.5 Space complexity of array-of-linked-lists structure

The instructor says the space here is:

m + n

This means:

- memory for the outer array
- memory for the connected list elements

Again, the exact notation is not fully explained in the transcript, but the idea is that space depends on both parts of the structure.

10.6 Space complexity of binary tree

A binary tree with n nodes uses:

Order(n) space

because there are n nodes in memory.

---

11. Code Examples by Topic

11.1 Single traversal of an array

```
for(i = 0; i < n; i++)
{
    // processing
}
```

Explanation

- visits each element once
- useful for sum, search, count, max, and similar tasks
- related to **Order(n)**

11.2 Full nested comparison

```
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        // processing
    }
}
```

Explanation

- for every element, all elements are processed
- useful in comparison-heavy tasks
- related to **Order(n^2)**

11.3 Reduced nested comparison

```
for(i = 0; i < n; i++)
{
    for(j = i + 1; j < n; j++)
    {
        // processing
    }
```

```
}
```

Explanation

- avoids reprocessing earlier elements
- still performs a triangular number of comparisons
- related to **Order(n^2)**

11.4 Repeated halving with a for loop

```
for(i = n; i > 1; i = i / 2)
{
    // processing
}
```

Explanation

- each step cuts the size in half
- number of steps is logarithmic
- related to **Order(log n)**

11.5 Repeated halving with a while loop

```
i = n;
while(i > 1)
{
    // processing
    i = i / 2;
}
```

Explanation

- same logic as above

- often easier to read when the counter does not change by 1
- related to **Order(log n)**

11.6 Processing a matrix

```
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        // processing
    }
}
```

Explanation

- visits all cells in an n × n matrix
- related to **Order(n^2)**

---

12. Important Instructor Tips, Warnings, and Common Mistakes

12.1 Do not misunderstand n

Common mistake:

- thinking n means 5 or 10 only

Correct idea:

- n means input size in general

12.2 Do not assume every for loop is Order(n)

Common mistake:

- seeing a loop and immediately saying Order(n)

Correct idea:

- check how the loop variable changes
- if it divides by 2 each time, it may be Order(log n)

12.3 Understand the work, not just the syntax

Common mistake:

- trying to memorize complexity only from code patterns

Correct idea:

- first understand what the code is doing
- then measure how much work is happening

The instructor says this makes analysis much easier.

12.4 Nested loops often mean higher complexity

Common mistake:

- ignoring the effect of an inner loop

Correct idea:

- one loop over n items → Order(n)
- two full nested loops → usually Order(n^2)
- three nested loops → often Order(n^3)

12.5 Focus on the dominant term

When a formula becomes something like:

- $(n^2 - n) / 2$

do not keep all parts equally important.

Focus on the highest-degree term:

- n^2

So complexity becomes:

Order(n^2)

12.6 Decide what should be counted

The instructor points out that in some structures, you may count:

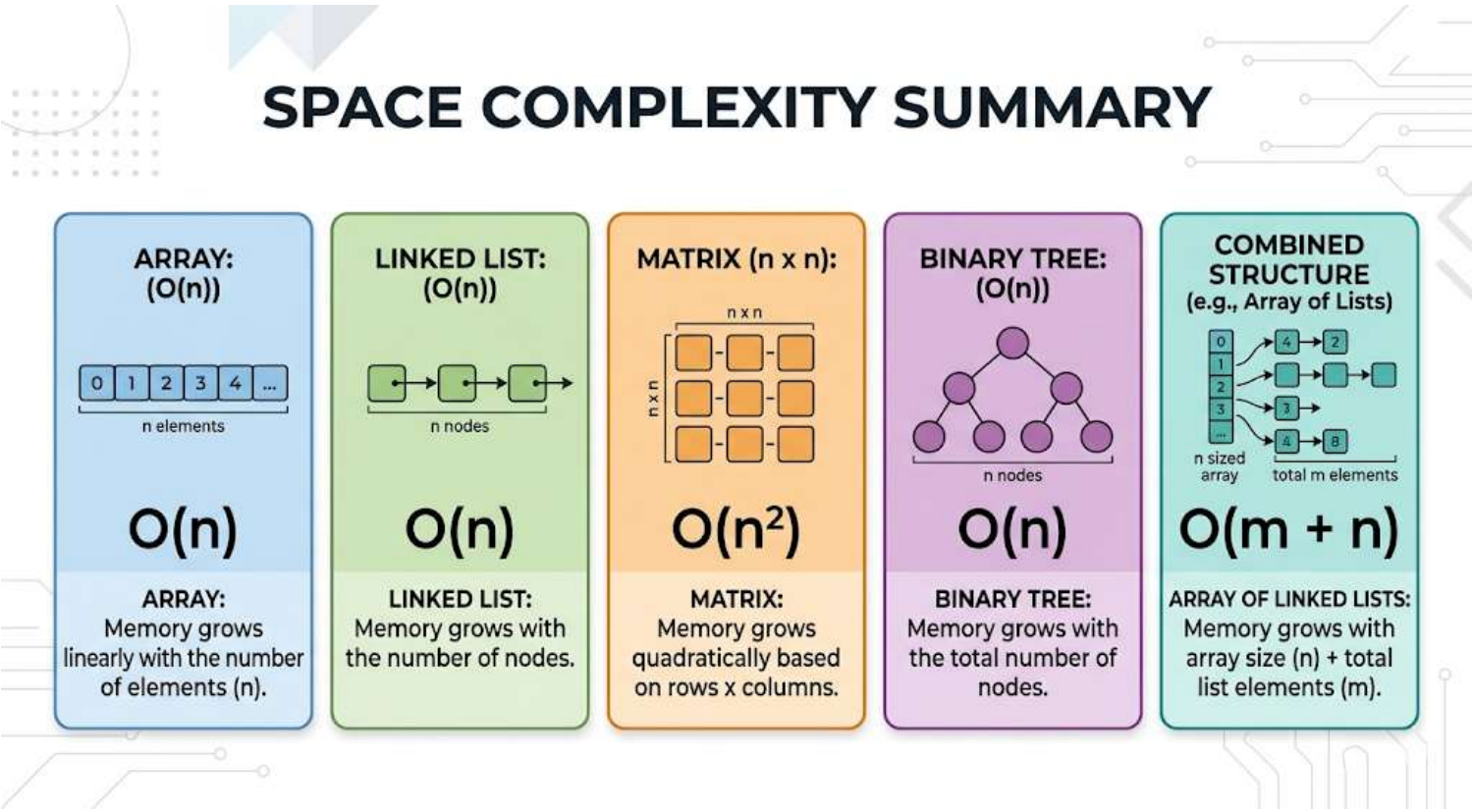
- all parts
- only main parts
- or only relevant parts

So complexity analysis sometimes depends on the exact problem definition.

---

13. Final Recap

This lesson gives a practical introduction to complexity.



Time complexity

It tells us how running time grows as input size grows.

Main patterns covered:

- process all elements once ⟶ **Order(n)**
- for each element, process all elements ⟶ **Order(n^2)**
- process remaining elements only ⟶ still **Order(n^2)**
- divide work by 2 each step ⟶ **Order(log n)**
- process all matrix elements ⟶ **Order(n^2)**
- process along a balanced tree path ⟶ **Order(log n)**
- process all tree nodes ⟶ **Order(n)**

Space complexity

It tells us how memory usage grows.

Examples:

- array → **Order(n)**
- linked list → **Order(n)**
- matrix → **Order(n^2)**
- binary tree with n nodes → **Order(n)**
- combined structures may be counted as **m + n**

Core message of the lesson

- Complexity depends on the **work being done**
- Code should be read carefully
- Loops must be analyzed by behavior, not by appearance alone
- The dominant growth term matters most

---

14. Unclear or Incomplete Transcript Parts

14.1 Array-of-linked-lists notation

The instructor explains a structure that looks like an array of linked lists and says the complexity can be m + n.

This part is slightly unclear because:

- the exact meaning of m is not fully defined
- the number of array elements and linked-list elements is described somewhat loosely

Safe interpretation:

- one part of the cost comes from the outer structure
- another part comes from the connected inner elements
- total complexity may be described as m + n

14.2 “Degree” wording

The instructor says “degree” and “order” in an informal teaching sense.

The full formal meaning using:

- Big-O
- Omega
- Theta

is not taught in this lesson and is postponed for later.

If you want, I can also turn these notes into a **one-page revision sheet** or a **question-answer study guide**.

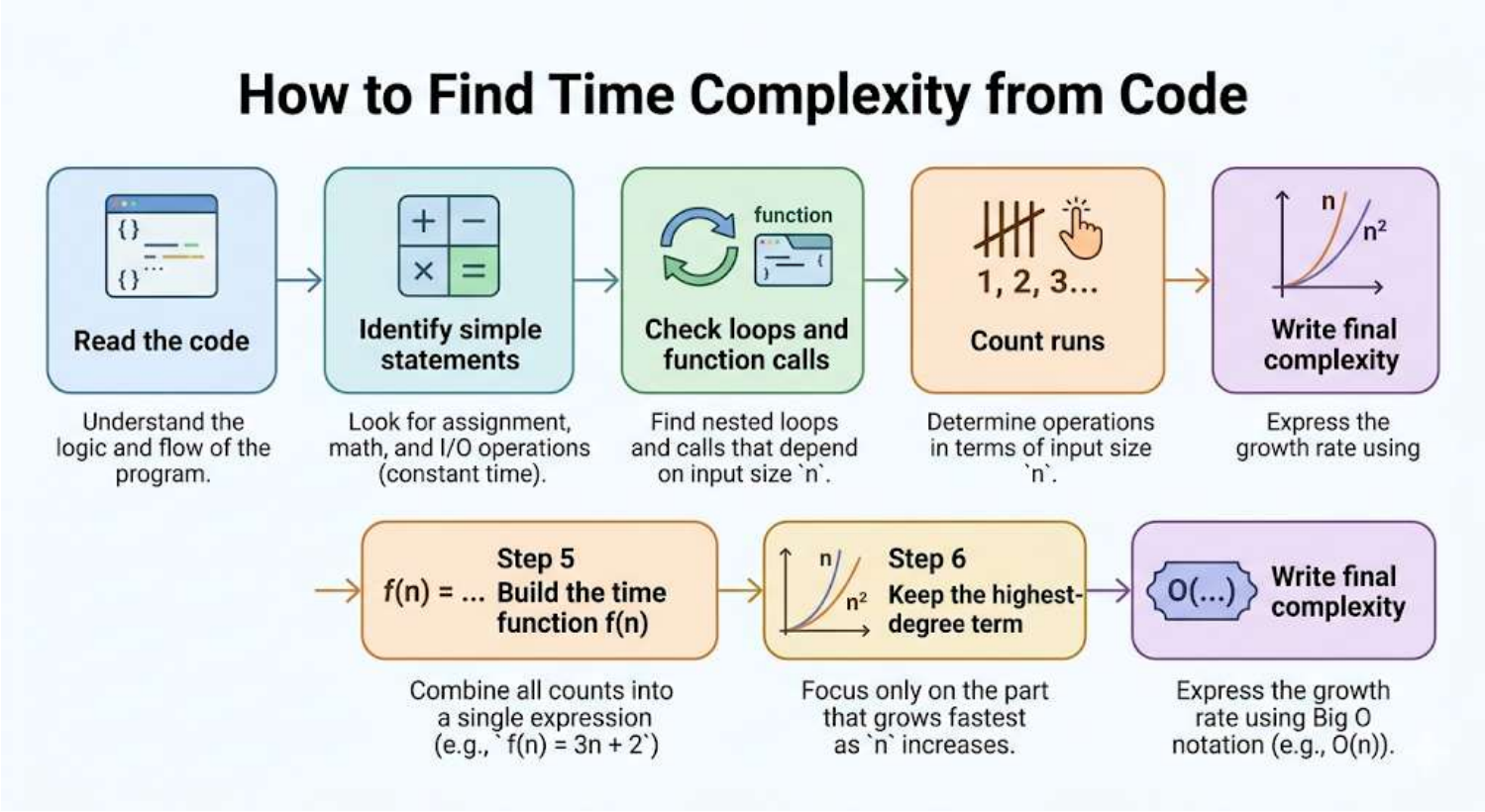
Study Notes: How to Find Time Complexity from Program Code

These notes are based on the transcript you provided.

---

2. Lesson overview

This lesson explains how to find **time complexity** by reading program code step by step. The instructor shows how to build a **time function** first, and then how to get the final complexity from that function. The lesson uses simple examples, a loop example, a nested loop example, and a function-call example. The main idea is to count how many times statements run. Then keep the highest-order term to describe the growth of running time.



---

3. Main topics

Main topics covered

- What a time function is

- The assumption that simple statements take one unit of time
- How to analyze a simple swap function
- How to analyze a loop that sums array elements
- How to analyze nested loops
- How to handle function calls when finding time complexity
- Why loops often determine complexity
- Why the final answer comes from the highest-degree term
- A brief note that space complexity can also be written as a function

---

4. Subtopics under each main topic

Topic 1: Basic idea of time complexity

Subtopics

- Time function
- Unit-time assumption
- Simple vs complex statements

Topic 2: Example 1 - Swap function

Subtopics

- Counting simple assignment statements
- Constant time
- Why the result is O(1)

Topic 3: Example 2 - Single loop for array sum

Subtopics

- Counting loop control operations

- Counting statements inside the loop
- Forming the time function
- Getting O(n)

Topic 4: Example 3 - Nested loops

Subtopics

- Outer loop count
- Inner loop count
- Multiplication in nested loops
- Forming the time function
- Getting O(n^2)

Topic 5: Function calls and hidden cost

Subtopics

- Why one line of code is not always O(1)
- Looking inside a called function
- Time of caller depends on callee

Topic 6: General rules and instructor advice

Subtopics

- Loops usually dominate complexity
- For loop and while loop are analyzed in the same way
- Some loops are not O(n)
- Practice is needed for line-by-line analysis

---

5. Detailed explanation for each subtopic

Topic 1: Basic idea of time complexity

What is a time function?

- A **time function** shows how the running time depends on input size.
- It is written as a function of **n**.
- After getting the time function, we find the **highest-degree term**.
- That highest-degree term gives the final time complexity.

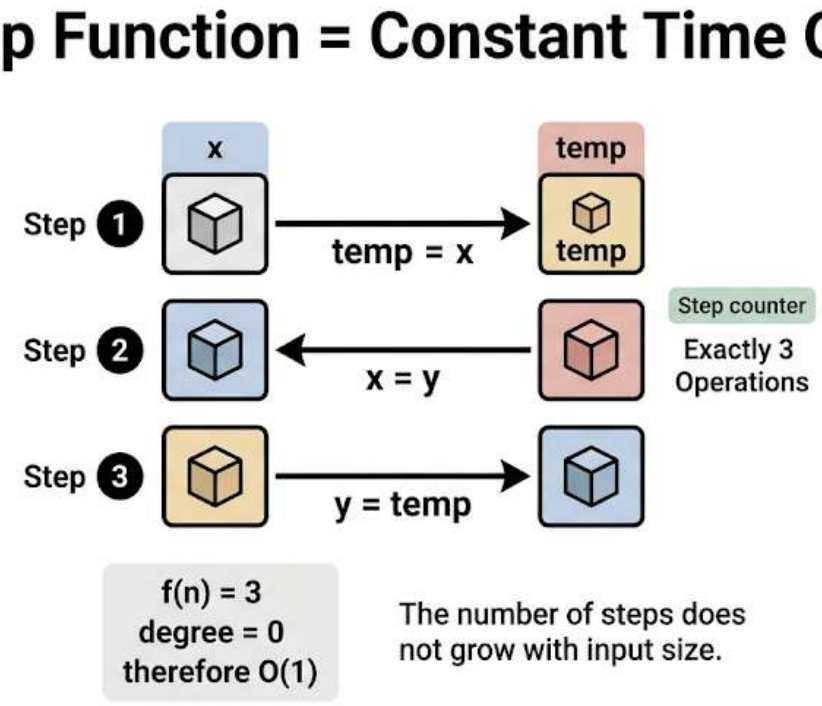
Unit-time assumption

- The instructor assumes that every **simple statement** takes **1 unit of time**.
- Examples of simple statements:
  - Assignment
  - Arithmetic operation
  - Conditional check
- This is a starting rule used to analyze code.

Simple vs complex statements

- A simple statement can be counted directly as 1.
- A complex statement must be analyzed based on what it does.
- A loop is not treated like a normal single statement.
- A function call is also not always just 1.
- You must inspect the actual work being done.

Topic 2: Example 1 - Swap function



Idea

The first example is a swap function that exchanges the values of x and y. The instructor says it contains three assignment statements, and each one takes 1 unit of time. So the total time is 3. That is a constant. Therefore, the complexity is O(1).

Approximate code shown in the lesson

The exact code formatting was not fully visible in the transcript. This is a clean pseudocode reconstruction of what the instructor describes.

```
swap(x, y)
{
    temp = x
    x = y
    y = temp
}
```

What this code does

- It stores x in a temporary variable.
- It copies y into x.
- It copies the old x value into y.

Why the instructor used this code

- To show the simplest possible time-complexity example.
- To show how a fixed number of statements gives **constant time**.

Time analysis

- temp = x ⟶ 1
- x = y ⟶ 1
- y = temp ⟶ 1

So:

```
f(n) = 3
```

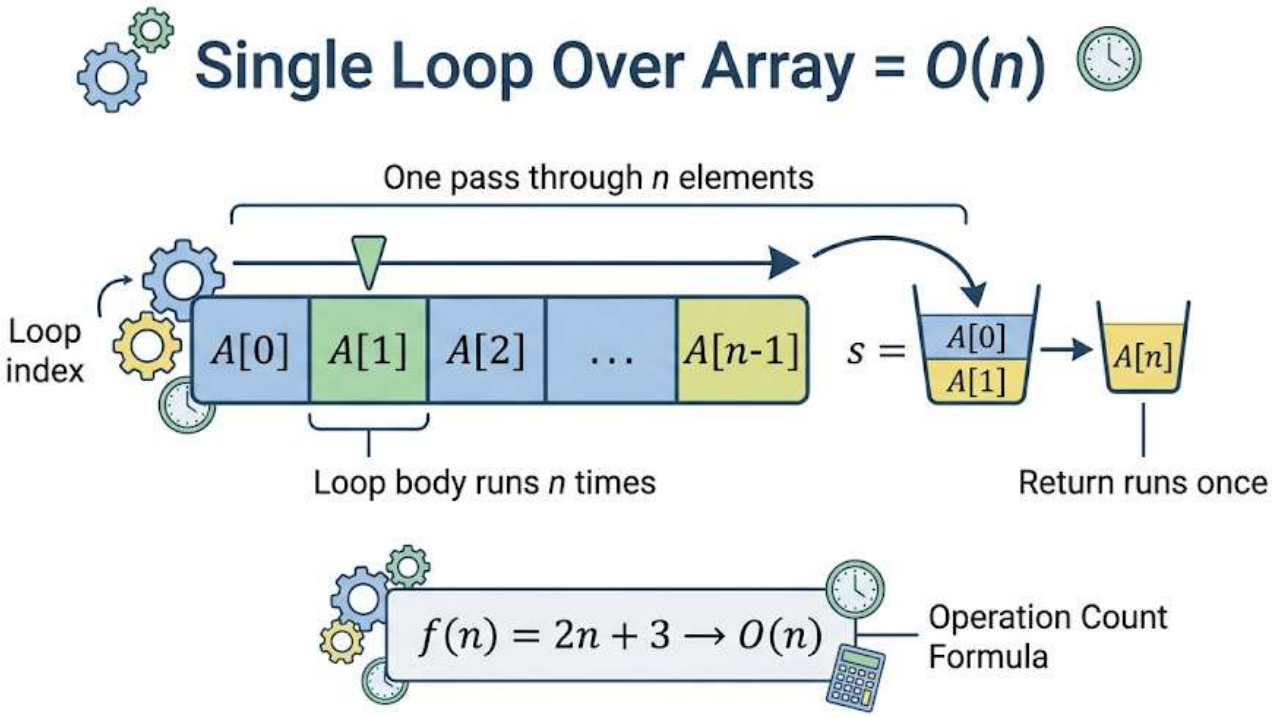
Why this becomes O(1)

- 3 is a constant.
- The instructor explains that this is like 3 * n^0.
- Since n^0 = 1, the degree is 0.
- A degree-0 time function is constant time.

Key lesson from this example

- If the number of operations does not depend on n, the complexity is **O(1)**.

Topic 3: Example 2 - Single loop for array sum



As the number of elements grows, the work grows in direct proportion.

Idea

The second example adds all elements of an array. The instructor says the time depends on the number of elements. If there are n elements, the loop runs n times, so the final result is O(n). He also shows how to get this result line by line.

Approximate code shown in the lesson

The transcript describes the statements, but not the full exact code. This is a clean reconstruction of the logic explained.

```
sum(A, n)
{
    s = 0
    for (i = 0; i < n; i++)
    {
        s = s + A[i]
    }
```

```
    return s
}
```

What this code does

- It starts a sum variable at 0.
- It visits each array element once.
- It adds each element into s.
- It returns the final sum.

Why the instructor used this code

- To show how a single loop usually gives linear time.
- To show how to count loop-related operations carefully.

Step-by-step time analysis

1) Initialization before the loop

- s = 0 ⟶ 1 unit

2) Loop control

The instructor discusses the loop header in detail.

For a loop like:

```
for (i = 0; i < n; i++)
```

- Initialization happens once
- Increment happens n times
- Condition is checked repeatedly
- One extra condition check happens when the loop stops

So the condition-related count is treated as:

```
n + 1
```

The instructor says many textbooks focus mainly on the **condition check** when counting the loop header. If someone wants to include more loop-header parts, constants may change, but the final order stays the same.

3) Statement inside the loop

- s = s + A[i] is a simple statement with an addition and assignment
- It runs once per iteration
- Since the loop runs n times, this contributes n

4) Return statement

- return s runs once
- So it contributes 1

Time function

The instructor gives:

f(n) = 2n + 3

Final complexity

- The highest-degree term is n
- So the time complexity is:

O(n)

Important meaning

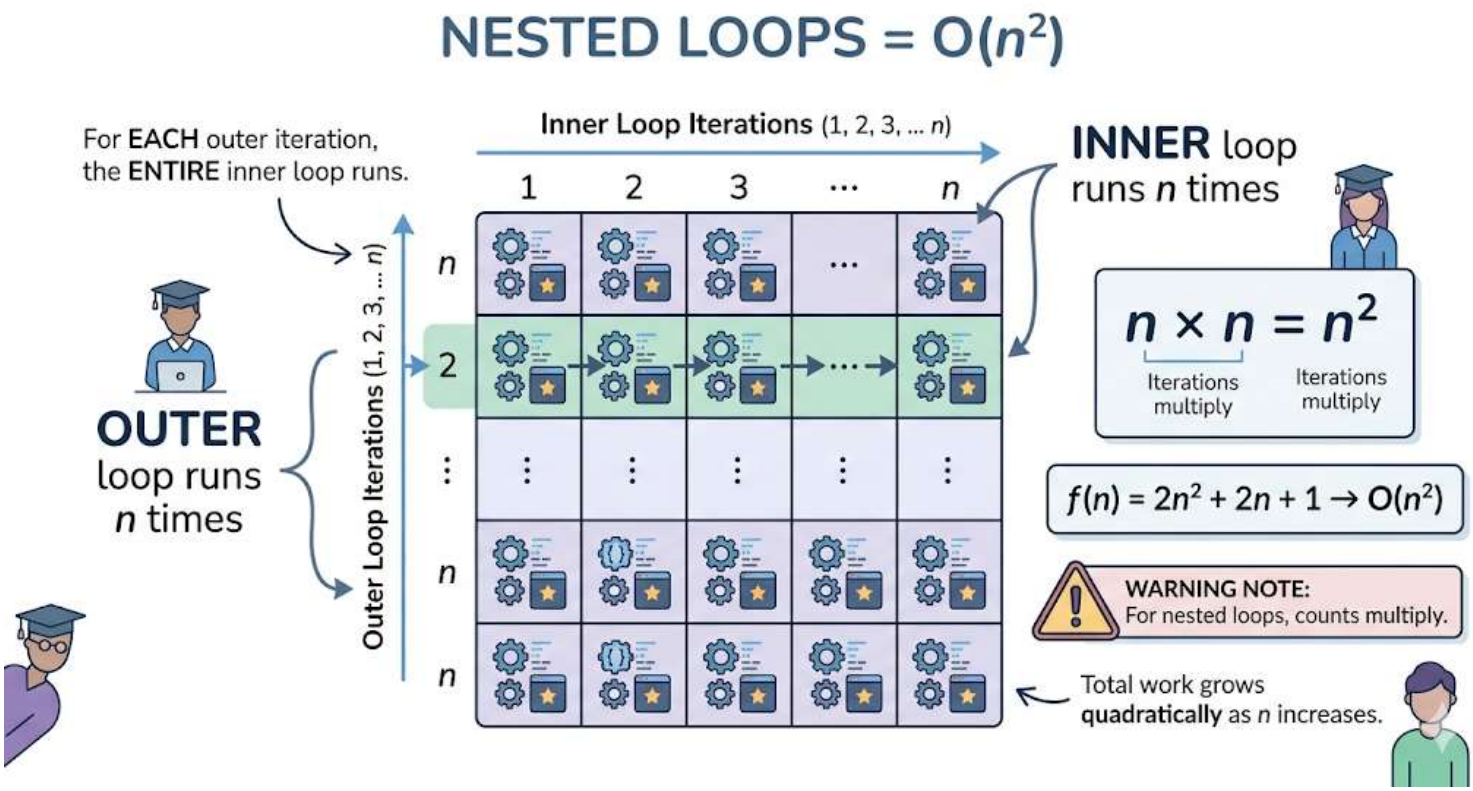
- The exact constants do not matter for the final growth rate.
- Whether you count a few extra loop-header operations or not, the result is still linear.

Instructor comparison

- You can do full line-by-line counting.

- Or, once you understand the pattern, you can look at the processing and directly say it is O(n).

Topic 4: Example 3 - Nested loops



Idea

The third example contains a loop inside another loop. The instructor uses it to show the main rule for nested loops: **counts multiply**. This leads to a quadratic time function and a final result of O(n^2).

Approximate code shown in the lesson

The exact full code was not visible in the transcript. This is a minimal reconstruction of the pattern the instructor describes.

```
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
```

```
    {
        statement
    }
}
```

What this code does

- The outer loop runs across n iterations.
- For each outer-loop iteration, the inner loop runs again.
- The inner work repeats many times because it is inside another loop.

Why the instructor used this code

- To show that nested loops increase complexity faster than a single loop.
- To show why multiplication appears in time analysis.

Step-by-step time analysis

1) Outer loop

- The instructor says the outer loop condition is checked n + 1 times.

2) Inner loop

- The inner loop is inside the outer loop.
- So its cost must be multiplied by the number of outer iterations.
- The inner condition contributes:

n * (n + 1)

3) Statement inside the inner loop

- A normal statement inside the inner loop runs n times for each outer iteration.
- So it contributes:

n * n = n^2

Time function

The instructor gives:

f(n) = 2n^2 + 2n + 1

Final complexity

- The highest-degree term is n^2
- So the time complexity is:

O(n^2)

Main lesson from this example

- In nested loops, repeated work grows quickly.
- If one loop runs n times and another inside it also runs n times, total work is usually proportional to n^2.

Important rule

- For nested loops, use **multiplication**, not addition.
- The instructor directly asks whether to add or multiply, then explains that nested loops require multiplication.

Topic 5: Function calls and hidden cost

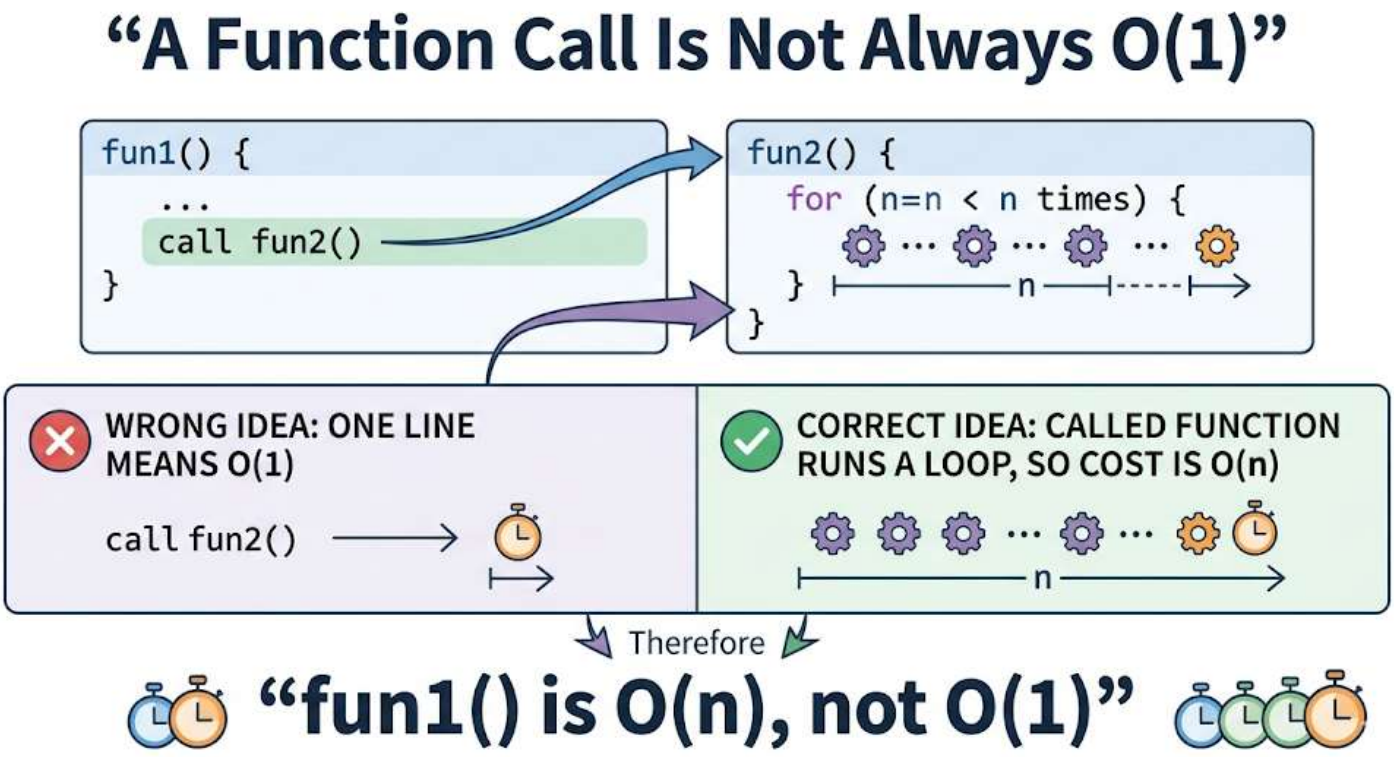
Idea

The instructor gives a warning: do not say a function is O(1) just because it contains only one statement if that statement is a **function call**. You must analyze the called function too.

Approximate code shown in the lesson

The transcript only describes a skeleton of two functions. This is a clean reconstruction of that example.

```
fun1()
{
    fun2()
}
fun2(n)
{
    for (i = 0; i < n; i++)
    {
        statement
    }
}
```



What this code does

- fun1() calls fun2()

- fun2() contains a loop that runs n times

Why the instructor used this code

- To correct a common student mistake
- To show that the cost of a function call includes the cost of the called function

Wrong conclusion

A student may say:

- "fun1() has only one statement"
- "So it takes 1 unit of time"
- "So it is O(1)"

The instructor says this is **wrong**.

Correct conclusion

- The statement inside fun1() is not a simple assignment or arithmetic step
- It is a function call
- That called function contains a loop
- So fun2() takes O(n)
- Therefore, fun1() also takes O(n), because it depends on the work done by fun2()

Main lesson from this example

- A function call is not automatically constant time
- Always inspect the called function before deciding the complexity of the caller

---

Topic 6: General rules and instructor advice

Rule 1: Start with simple statements

- If a statement is simple, count it as 1
- Examples:
  - assignment
  - arithmetic operation
  - conditional check

Rule 2: Inspect complex statements

- If the statement is a loop, count how many times it runs
- If the statement is a function call, inspect the called function
- Do not blindly label complex statements as 1

Rule 3: Loops often decide the complexity

- The instructor says loops usually create terms like:
  - n
  - n^2
  - n^3
- So loops are often the biggest reason time grows with input size

Rule 4: For loop and while loop are handled in a similar way

- Some students understand for loops but worry about while loops
- The instructor says both are similar in idea
- What matters is how many times the loop repeats
- You must read the code carefully to know that count

Rule 5: Not every loop is O(n)

- A normal counter-controlled loop from 1 to n is usually O(n)

- But if the loop behaves differently, the result may be:
    - log n
    - root n (the instructor mentions this informally)
    - something else
- So do not assume every loop is linear
- Read the update pattern of the loop variable carefully

Rule 6: Highest-degree term gives the final order

- First build the full time function
- Then find the highest-degree term
- That term determines the growth rate
- The instructor calls this the “order of” the function
- He also mentions that people may say:
    - Big-O
    - Theta
    - Omega
- He says the exact use of these will be discussed later, but for now he often says “order of” in the videos

Rule 7: Space complexity can also be written as a function

- The instructor briefly says that space complexity can also be expressed as a function
- He mentions it was explained in a previous video
- This lesson mainly focuses on time complexity

---

6. Code examples related to each topic

Code example 1: Swap function

```
swap(x, y)
{
    temp = x
    x = y
    y = temp
}
```

Simple explanation

- This swaps two values.
- It has three simple assignments.
- Each takes 1 unit.
- Total time = 3.
- Complexity = O(1).

Concept linked to this code

- Constant time
- Simple-statement counting

Code example 2: Sum of array elements

```
sum(A, n)
{
    s = 0
    for (i = 0; i < n; i++)
    {
        s = s + A[i]
```

```
    }
    return s
}
```

Simple explanation

- This adds all n elements of an array.
- The main work is the loop.
- The loop body runs n times.
- So the time grows linearly with n.

Concept linked to this code

- Linear time
- Loop counting
- Time function f(n) = 2n + 3

Code example 3: Nested loops

```
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        statement
    }
}
```

Simple explanation

- The outer loop runs many times.
- The inner loop runs again for every outer-loop pass.

- Repetition multiplies.
- That produces quadratic growth.

Concept linked to this code

- Nested loops
- Multiplication of counts
- Time function f(n) = 2n^2 + 2n + 1
- Complexity O(n^2)

Code example 4: Function call complexity

```
fun1()
{
    fun2()
}
fun2(n)
{
    for (i = 0; i < n; i++)
    {
        statement
    }
}
```

Simple explanation

- fun1() looks short.
- But its single line calls fun2().
- fun2() contains a loop.
- So fun1() is not O(1).

- It inherits the cost of fun2(), which is O(n).

Concept linked to this code

- Hidden cost of function calls
- Caller complexity depends on callee complexity

---

7. Important instructor tips, warnings, or common mistakes

Tips

- Start by assuming each simple statement takes 1 unit.
- Build the full time function first.
- Then keep the highest-degree term.
- Practice line-by-line analysis until it becomes easy.
- Once you understand common patterns, you can often identify O(n) or O(n^2) faster.

Warnings

- Do not treat a loop as one simple statement.
- Do not treat a function call as automatically O(1).
- Do not assume every loop is linear.
- Do not add counts for nested loops when they should be multiplied.

Common mistakes

- Saying “this line is one statement, so it is O(1)” without checking what that line does
- Forgetting the extra failed condition check in a loop
- Ignoring that nested loops multiply work

- Focusing only on syntax and not on actual repetition
- Thinking while loops must be analyzed differently from for loops in principle

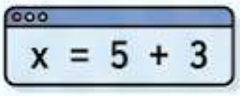
---

8. Final recap

Quick Rules for Finding Time Complexity

Correct Rules

1. Simple Statement = 1 Unit



Constant time operations (e.g., assignment, arithmetic). O(1).

2. Count Loop Repetitions

E.g.,
`for i in range(n)`

Determine how many times a loop body executes based on input size. E.g., for i in range(n)

3. Nested Loops Multiply

× O(n)
× O(m)

Multiply outer iterations by inner iterations. O(n * m) or O(n^2)

4. Inspect Function Calls

Check the internal code complexity of custom or library functions. Not always O(1).

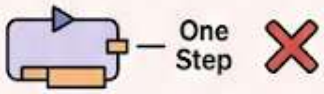
5. Keep Highest-Degree Term

$3n^3 + 5n^2 + 10$

Focus on the dominating term as 'n' grows large. Drop constants and lower terms.

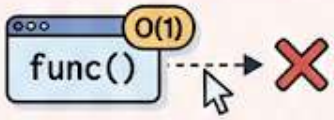
Common Mistakes

1. Treating a Loop as One Step



Overlooking the multiple iterations of loop bodies. O(n) is not O(1).

2. Treating a Function Call as O(1)



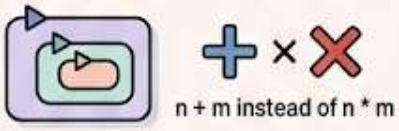
Assuming all function calls take constant time without inspecting them. Hidden O(n) or worse.

3. Forgetting the Extra Loop Condition Check

```
while i < n (+1)      n+1
    Body: n           Body: n
end
```

Condition runs once more to fail and terminate the loop. (e.g., n+1 checks)

4. Adding Nested Loop Counts

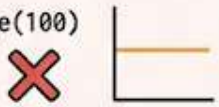


n + m instead of n * m

Incorrectly calculating total iterations for nested structures. Leads to wrong complexity like O(n).

5. Assuming Every Loop is O(n)

```
for i in range(100)
    O(n)
end
```



Loops with fixed iterations regardless of input size are constant time. O(1).

Educational Revision Poster

- Time complexity is found by counting how many times code runs.
- First build a **time function**.
- Simple statements are counted as 1 unit each.
- A swap function with three assignments gives a constant function, so it is **O(1)**.
- A loop that processes n elements gives a linear function, so it is **O(n)**.
- Nested loops often multiply counts, which can give **O(n^2)**.
- A function call must be analyzed based on the work done inside the called function.
- The final time complexity comes from the **highest-degree term** of the time function.
- Loops are usually the main reason time grows.
- Some loops may be O(log n) or other forms, so always read the code carefully.

---

9. Unclear or incomplete transcript parts

- The exact written code for the examples was not fully visible in the transcript. The code blocks above are **clean reconstructions based on the instructor's spoken explanation**.
- The instructor refers to textbooks focusing mainly on the loop condition count, but the exact wording around counting all parts of the for loop is slightly unclear.
- The instructor briefly mentions **O**, **Theta**, and **Omega**, but says their exact use will be explained later. So this transcript does not fully define the difference between them.

- The instructor mentions space complexity from a previous video, but does not explain it in detail here.
- The phrase “rootn” appears in the transcript informally. It likely means a loop whose running time could be proportional to something like √n, but no code example is given here. [unclear]